# Agentic AI

# Contents


Foundations & The Autonomous Agent Loop

Single-Agent Frameworks & Tool Calling (ADK, LangChain, SDK)

Multi-Agent Orchestration (Swarm, AutoGen, CrewAI)

Stateful Graph Workflows & Human-in-the-Loop (LangGraph)

Guardrails, Observability, Evaluation & Production Deployment

# Foundations & The Autonomous Agent Loop

- **Generative AI vs. Agentic AI:** Moving from prompt-response text generation to multi-step perception-action loops
- **Core Anatomy of an AI Agent:** Brain (LLM), Sensors/Inputs, Working Memory, Tools, and Actuators
- **The ReAct Pattern Deep Dive:** Interleaving Thought → Action → Observation → Reflection
- **Function Calling & Tool Execution Mechanics:** How LLMs output JSON schemas and execute local Python functions using Agent
- **Structured Outputs & Schema Enforcement:** Eliminating hallucinations using Pydantic and JSON validation
- **Local LLM Inference with Ollama:** Running LLM locally, setting LLM parameters
- **Capstone Projects:**
  - Portfolio Risk & Asset Intelligence (ReAct)

# Single-Agent Frameworks, Tools & Knowledge Systems



- **Framework Ecosystem:** Comparing Google GenAI SDK (ADK), LangChain, and OpenAI Agent SDK
- **LangChain LCEL:** Declarative pipelines (prompt | model | parser) and tool binding
- **Multi-Tool Orchestration:** Binding multiple tools (Forex APIs, Database queries, Calculator) dynamically
- **Agentic Memory Architectures:** Short-term sliding context buffers vs. Long-term semantic episodic memory
- **Agentic RAG vs. Traditional RAG:** Dynamic query rewriting, self-correction, and hybrid keyword-vector search
- **Grounding & Evidence Citations:** Extracting verified document IDs (HR-POL-042) for compliance
- **Plan-and-Execute Pattern:** Decoupling sequential task decomposition from deterministic tool execution
- Capstone Projects:
  - Expense Auditor (Tool calling)
  - Conversational Buffer (Memory)
  - Enterprise Policy Synthesizer (Agentic RAG)

# Multi-Agent Systems, Swarms & Collaborative Debates

- **Why Multi-Agent Systems?:** Specialization, blast-radius isolation, and domain expert personas.
- **Decentralized Swarms & Dynamic Handoffs:** Peer-to-peer delegation without a central coordinator (OpenAI Swarm pattern).
- **Microsoft AutoGen Architecture:** ConversableAgent, GroupChat, and GroupChatManager.
- **Multi-Agent Debate & Self-Consistency:** Having Product, Architecture, Dev, and QA challenge each other to reach consensus.
- **Sandboxed Code Execution:** Safe local Python execution and automated unit testing in agent loops.
- **Hierarchical Crew Orchestration:** Roles, Goals, Backstories, Tasks, and Manager LLM delegation.
- **Actor-Critic & Reflection (Reflexion):** Generating artifacts, running AST linting/critiques, and synthesizing self-correcting patches.
- Capstone Projects:
  - PR Reviewer (Reflection)
  - AutoGen Software Team (Self consistency)
  - Support Swarm (Multi-Agent Collaboration)
  - CrewAI Inventment Crew (Hierarchical Agents)

# Stateful Graph Workflows, Human-in-the-Loop & Events

- **State Machine Foundations:** Why acyclic DAGs fail and cyclical state graphs (StateGraph) succeed
- **LangGraph Core:** State schemas, Nodes (actions), Edges (transitions), and Conditional Routers
- **The Hierarchical Supervisor Pattern:** Fan-out parallel subgraphs and state aggregation (operator.add)
- **State Persistence & Checkpoint Recovery:** Saving intermediate snapshots to JSON for failure recovery
- **Human-in-the-Loop (HITL) Architecture:** Pausing execution at high-risk decision gates and resuming
- **Event-Driven AI Pipelines:** Reactive workflows triggered by HTTP webhooks and message queues
- **The Evaluator–Optimizer Pattern:** Two-component generator-discriminator refinement loops for clinical/medical matching
- Capstone Projects:
  - Due Diligence Supervisor (Supervisor)
  - Stateful Loan Underwriting (HIL)
  - Clinical Trial Matcher (Evaluator Optimizer)
  - B2B Lead Enrichment Flow (Event Driven Agents)

# Guardrails, Evaluation, Observability & Deployment

- **Production Agent Challenges:** Hallucinations, runaway loops, rate limits, and unauthorized tool calls
- **Deterministic Guardrails & Blast-Radius Containment:** Input/output filtering, pre-flight safety gates, and budget caps
- **Model Context Protocol (MCP):** Universal standardized interface connecting agents to external databases, files, and tools
- **Agent Observability & Tracing:** Step-by-step telemetry, token tracking, and logging with LangSmith, Helicone, and Arize
- **Evaluation & LLM-as-a-Judge:** Evaluating agent trajectory success, tool call accuracy, and RAG faithfulness (RAGAS)
- **Production API Architecture:** Wrapping agents in high-throughput **FastAPI** microservices and Docker containers
- **Interactive UI Dashboards:** Building interactive user interfaces using Streamlit and Rich terminal dashboards
- **Capstone Projects:**
  - Cyber Incident Responder (Plan and Execute)
  - DevOps SRE Self-Healing Engine (Checkpoint and Recovery)

# Agentic Frameworks

- OpenAI Agent SDK & Swarms
- Google ADK & GenAI SDK
- Microsoft AutoGen
- LangChain LCEL & Modern Tools
- LangGraph Stateful Graphs & Human-in-the-Loop
- CrewAI Crews and Flows
- n8n workflows

# About Instructor

- With **18+ years of crafting technology solutions**, I bring a blend of innovation, leadership, and hands-on expertise
- As the **Associate Technical Director at Sunbeam** and a passionate **Freelance Developer**, I've explored diverse domains, solving complex problems with modern tools and technologies.
- I've had the privilege of developing **many mobile applications** that power experiences on **iOS** and **Android** platforms and crafting dynamic websites with **PHP**, **MEAN**, **and MERN stacks**
- My journey has been fuelled by a deep love for programming languages like **C, C++, Python, JavaScript, TypeScript, Go, Swift, Kotlin and PHP**
- My DevOps journey includes building **CI/CD pipelines using Jenkins, GitHub Actions & ArgoCD**, automating infrastructure with tools like **Terraform & Ansible**, leveraging cloud platforms like **AWS** to create robust, scalable systems and containerizing applications using **Docker** and orchestrating them using **Kubernetes and Docker Swarm**













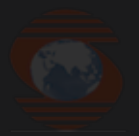
# Rules and regulations



- The class will begin at 9:00 pm every day  (Please join at least 5 minutes before)
- The classes will be conducted from Monday to Friday
- All the classes will be recorded and will be available in the student's portal next day around 10am
- When you are having the doubts, write them down somewhere
- To avoid the class disturbance, at the end of the theoretical discussion, we will spend time taking the questions all at once
- All the source code will be shared in a GitHub repository

# Generative AI

# Generative AI



- Generative AI refers to AI systems that can create new content from scratch based on patterns learned from training data
- Rather than just analyzing or classifying existing information, these systems generate entirely new outputs
- **Key capabilities**
  - Creating original content (essays, code, designs) → text | video | audio | image
  - Answering questions in conversational style
  - Summarizing or simplifying complex information
  - Translation between languages
  - Brainstorming and ideation
  - Problem-solving through reasoning

# Key characteristics

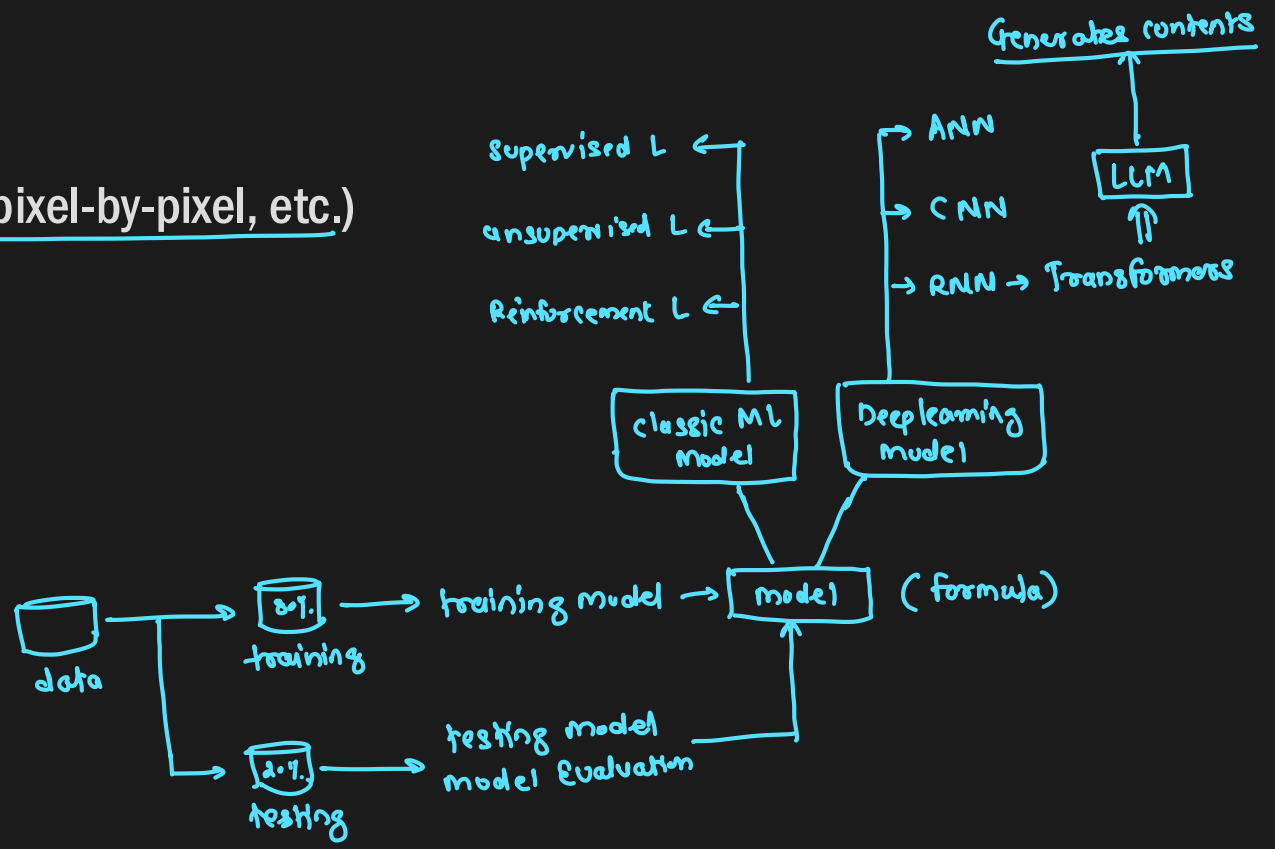
- **Creates new content**
  - The system produces novel outputs—text, images, code, video, audio, etc.—that didn't exist before, rather than retrieving or classifying existing data
- **Learns patterns**
  - It's trained on large datasets to understand patterns, structures, and relationships in that domain
- **Probabilistic**
  - It predicts what should come next (next word, next pixel, next note) based on what came before
- **Flexible**
  - A single model can often generate different outputs for the same input, with some variation

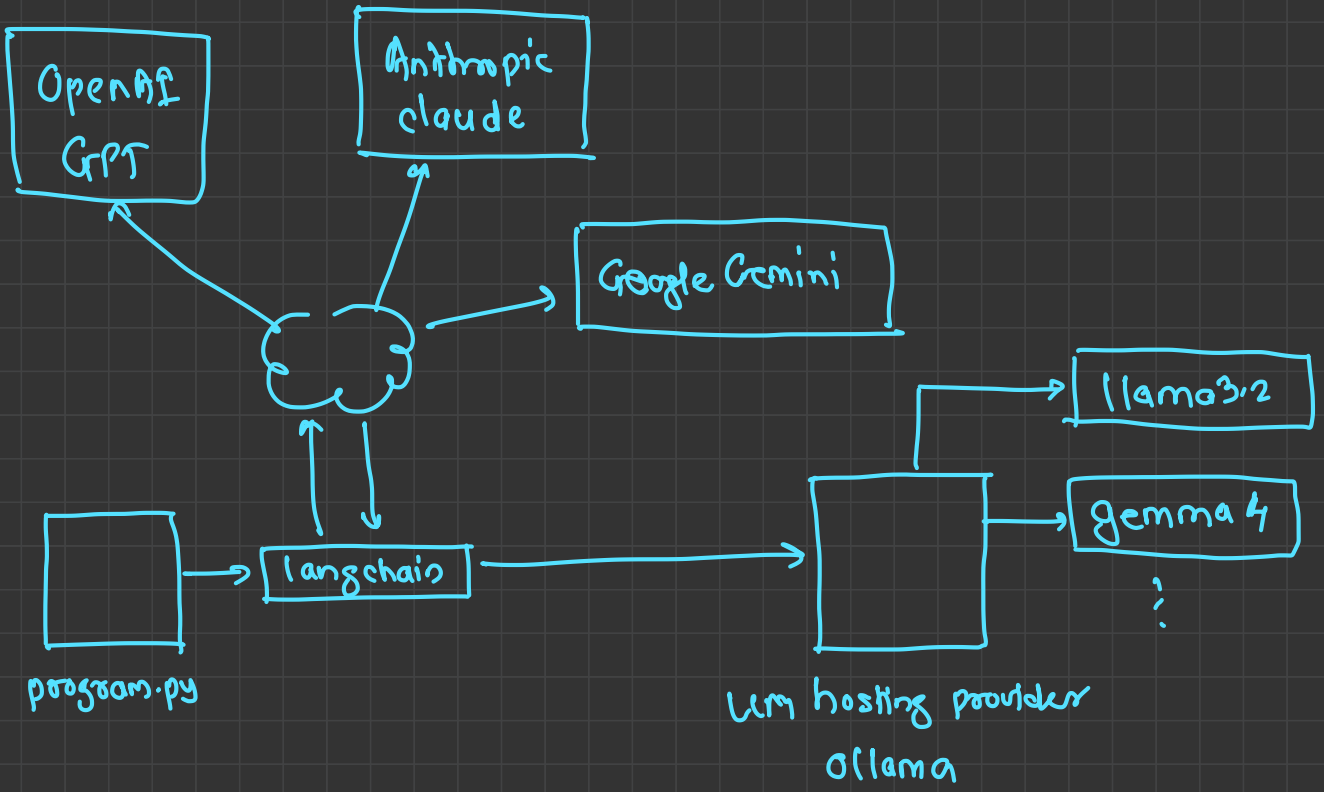
# Common types

- **Large Language Models (LLMs)**
  - Generates text
  - Used for writing, answering questions, summarizing, translation, coding
  - Examples: Claude, ChatGPT, Gemini
- **Image generation**
  - Creates images from text descriptions
  - Used for art, design, illustration
  - Examples: DALL-E, Midjourney, Stable Diffusion
- **Code generation**
  - Writes functional code
  - Used for programming assistance
  - Examples: GitHub Copilot, Claude Code
- **Audio/Music generation**
  - Creates speech or music
  - Examples: ElevenLabs, MusicLM

# How it works

- **Training**
  - Model learns patterns from massive datasets
- **Encoding**
  - Converts input into mathematical representations
- **Generation**
  - Predicts and generates output token-by-token (or pixel-by-pixel, etc.)
- **Output**
  - Produces novel, contextually relevant content




OpenAI
GPT
Anthropic
claude
Google Gemini
llama3.2
gemma 4
langchain
program.py
LLM hosting provider
ollama

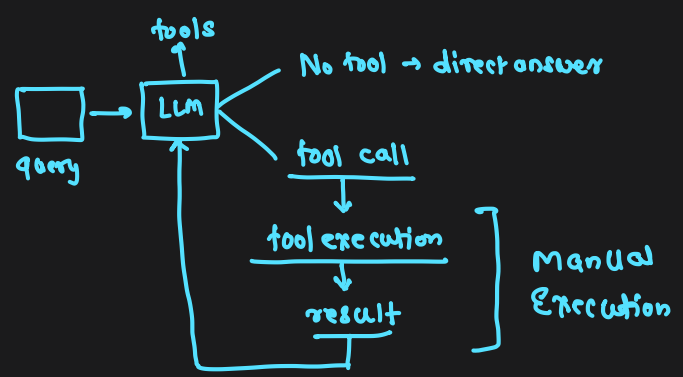
# Agentic AI

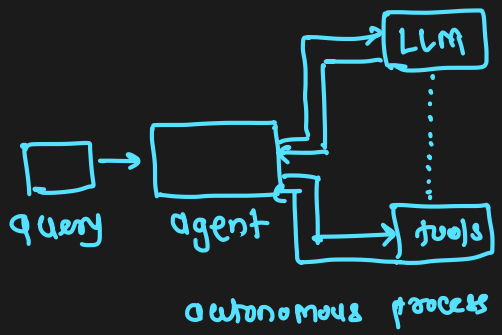
agents

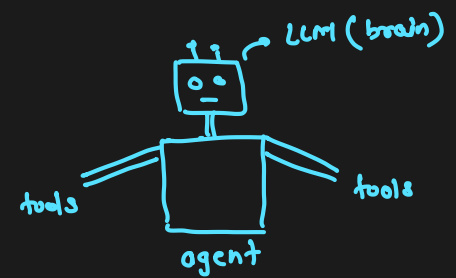
# Introduction

automatically

- Agentic AI refers to AI systems that operate autonomously to accomplish specific goals with minimal human intervention
- Rather than just responding to prompts, agentic systems take independent actions over time to solve problems
- Think of the AI assistance as your digital employee
- **Examples**
  - An AI assistant that can autonomously research a topic, organize findings, create a presentation, and email it to you
  - A coding agent that reads your requirements, writes tests, implements features, debugs failures, and pushes code
  - An AI that manages your calendar by reading emails, identifying scheduling needs, and booking meetings
  - A research agent that searches papers, extracts information, synthesizes findings, and flags contradictions



# Key Characteristics

- **Goal-oriented**
  - The AI is given an objective and works to achieve it without step-by-step human direction
- **Autonomous decision-making**
  - The system decides what actions to take next, which tools to use, and how to sequence them—rather than waiting for new instructions after each response
- **Tool use**
  - Agentic systems interact with external tools, APIs, and systems to gather information or take actions (search, code execution, database queries, API calls, etc.)
- **Planning & reasoning**
  - They break down complex tasks into subtasks, maintain context across multiple steps, and adapt their approach based on results
- **Iterative refinement**
  - They can evaluate outcomes, adjust strategy, and retry when something doesn't work as intended

# Generative vs Agentic AI

- **Generative AI**
  - Focuses on: Output quality and creativity
  - Key question: "Can it produce good content?"
  - Limitation: Produces output and stops—doesn't check if goal was achieved or adapt
  - Example: You ask ChatGPT to "write a blog post about AI," it writes one and waits for your next command
  - Use when: You want creativity, ideas, or content; you want to guide the process

- **Agentic AI**
  - Focuses on: Achieving real-world goals through action
  - Key question: "Can it accomplish the objective autonomously?"
  - Strength: Loops continuously—takes action, evaluates result, adapts, retries until goal is met
  - Example: You tell an agent "write a blog post about AI, optimize it for SEO, create matching social media posts, and schedule them"—it does all that without waiting for instructions between steps
  - Use when: You have a clear goal; you want hands-off execution; the task is complex and multi-step

**Agentic AI = Generative AI + Planning + Tool Use + Feedback Loop**

# Limitations

- **Hallucinations & Accuracy**
  - The AI generates plausible-sounding but false information
- **Cost & Resource Consumption**
  - Agentic systems make many API calls and process lots of tokens
  - **Example**: A simple task might cost $0.02, but an agent looping 5 times to get it right costs $0.10
- **Lack of True Understanding**
  - Agents don't truly understand problems—they pattern-match
  - **Example**: An agent might optimize for the wrong metric because it misunderstood the goal
- **Error Propagation & Brittleness**
  - Errors compound across steps
  - **Example**: If an agent misreads a date format, it schedules everything on the wrong day, and doesn't catch it
- **Difficulty with Complex Reasoning**
  - LLMs are still relatively weak at Multi-step logical reasoning, Complex mathematics and precise calculations etc.
- **Tool Dependency & API Brittleness**
  - Agents depend on external tools
  - **Example**: If the email API is down, your outreach agent stops completely

# Challenges

- **Unreliability & Non-Determinism**
  - Same agent, same input, different output. Inconsistent performance makes production deployment risky
- **Cost & Token Inefficiency**
  - Multi-step agentic loops consume far more tokens than necessary, making them economically unviable for many use cases
- **Safety & Uncontrolled Execution**
  - Autonomous systems can take harmful actions at scale without human oversight. Hard to stop or redirect mid-execution
- **Lack of True Reasoning & Understanding**
  - Agents pattern-match rather than actually understand. They struggle with novel problems, complex logic, and real-world constraints
- **Hallucinations & Error Propagation**
  - The AI generates false information or misinterprets real data, and errors cascade through multi-step workflows

# Agents

# What is Agent ?

- **Traditional AI Agent:** A program that follows a set of rules (if X happens, do Y) to complete a task. It is rigid and reactive
- **Agentic AI Agent:** An AI system that uses a **Large Language Model (LLM)** as its "brain" to reason, plan, and take action autonomously to achieve a complex goal
- **An agent is an autonomous digital entity that perceives its environment, makes independent decisions using generative AI, and takes actions to achieve a specific objective—without needing step-by-step human instructions**
- Examples:
  - **Coding Agent:** Given a GitHub issue, it plans, writes code, debugs errors, tests it, and submits a pull request
  - **Research Agent:** You ask it to write a market report. It autonomously searches the web, reads PDFs, extracts key data, creates a spreadsheet, and writes a summary—all without you telling it which websites to visit
  - **IT Operations Agent:** It monitors server logs, detects a spike in errors, reasons that the database is overloaded, automatically spins up a new cloud server, and reroutes traffic

# The Core Components

- Every agentic AI agent is built on a loop of four key functions:
- **Perception**
  - It takes in data from its environment
  - This isn't just text; it can include images, audio, sensor data, database records, or live web searches
- **Brain (The LLM)**
  - This is the engine
  - Instead of just predicting the next word, the LLM acts as the "orchestrator"
  - It engages in **reasoning** (e.g., chain-of-thought) to break a large goal into smaller sub-tasks
- **Planning**
  - The agent dynamically creates a step-by-step plan to achieve the goal. Crucially, it can re-plan mid-course if it encounters an error or new information
- **Action (Tools)**
  - The agent doesn't just talk; it does
  - It connects to external **tools** via APIs
  - This allows it to send emails, update spreadsheets, control robotic arms, query databases, or browse the interne

# The Agentic Workflow



- Unlike a standard chatbot that gives one answer and stops, an agentic agent runs in a continuous **"Sense-Think-Act" loop**
- **User gives a goal** (e.g., "Find the cheapest weekend flights to Pune, book a hotel within 2 KMs of the venue, and add it to my calendar")
- **The Agent reasons** and creates a multi-step plan
- **It takes action** (searches flights)
- **It perceives the result** (flight prices)
- **It evaluates** (are these under my budget?)
- **It adjusts its plan** (if not, it searches for alternative dates)
- **It takes the next action** (books the flight, then searches hotels)
- **It stops only when the final goal is achieved**

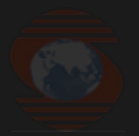
# Key Characteristics of an Agentic Agent



- **Autonomy**
  - It operates without human hand-holding. You give it the *what*, and it figures out the *how*.
- **Tool Use**
  - It is "tool-calling." Without tools, an LLM is just a brain in a jar. With tools, it becomes a doer.
- **Reflection & Self-Correction**
  - It can critique its own outputs. For example, it might write code, run it, see that it threw an error, rewrite the code, and run it again until it works
- **Memory**
  - It has **short-term memory** (the current conversation context) and **long-term memory** (a vector database where it stores past interactions, user preferences, or project data to reference later)

# Agent Memory

# Agent Memory



- **Agent memory** is the mechanism by which an AI agent retains and reuses information beyond a single, immediate input-output exchange
- It's what lets an agent behave with continuity and context-awareness rather than treating every interaction as a blank slate
- An LLM by itself is stateless — each call only "knows" what's in the prompt you send it. Agent memory is the system built around the model that decides:
  - What information to capture from interactions
  - Where to store it
  - When and how to bring it back into context so the agent can use it
- **Why It Matters for Agents Specifically**
  - Unlike a simple chatbot, an agent typically:
    - Executes multi-step tasks (needs to remember what it already tried)
    - Uses tools (needs to remember tool outputs to reason over them later)
    - Operates across sessions (needs to remember user preferences, past decisions, ongoing project state)
    - Sometimes coordinates with other agents (needs shared memory)
  - Without memory, an agent would re-plan from scratch every turn, forget tool results moments after generating them, and have no way to build on past interactions

# Types By Persistence / Scope

- **Short-term (working) memory**
  - Lives within a single session or task execution
  - Usually just the conversation/context window itself
  - Includes intermediate reasoning, tool outputs, scratchpad notes
  - Cleared when the session ends
- **Long-term memory**
  - Persists across sessions
  - Requires external storage (vector DB, key-value store, graph DB, relational DB)
  - Needs explicit write/read logic — the agent decides what's worth saving
- **Episodic memory**
  - Stores specific past events/interactions ("what happened")
  - Useful for recalling prior task attempts, user preferences stated once, past decisions and their outcomes
- **Semantic memory**
  - Stores generalized facts/knowledge extracted from experience ("what is true")
  - Often built by summarizing/abstracting episodic memories over time
- **Procedural memory**
  - Stores learned skills, workflows, or successful action sequences
  - E.g., "here's the tool-call sequence that worked for this type of task last time"

# Short-term (working) memory — checkpointer

```python
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),   # persists messages per thread_id
)

config = {"configurable": {"thread_id": "session-1"}}
agent.invoke({"messages": [{"role": "user", "content": "Hi, I'm Amit."}]}, config)
```

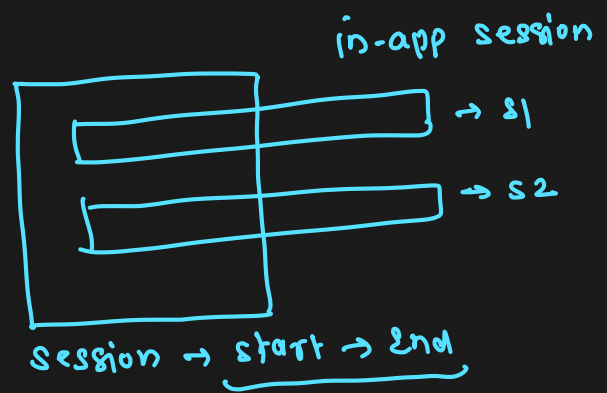
# Long-term / persistent memory — store (cross-session)

↳ thread

```python
from langgraph.store.memory import InMemoryStore   # swap for PostgresStore in prod

store = InMemoryStore()

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),  → short term
    store=store,   # accessible inside tools via get_store()
)
```



# Episodic / semantic memory — namespaced facts in the store



```python
from langgraph.store.base import BaseStore
from langchain.tools import tool

@tool
def remember_preference(user_id: str, fact: str, store: BaseStore):
    """Save a fact learned about the user (episodic/semantic memory)."""
    store.put(("user_memories", user_id), key=fact[:20], value={"fact": fact})
    return "Saved."
```

# Procedural memory — reusable tool/skill definitions



```
# Procedural memory = the agent's callable tools/skills themselves,
# often updated by appending a newly-learned successful routine
agent = create_agent(
    "gpt-5",
    tools=[remember_preference, search_db, book_meeting],  # the "how-to" library
    checkpointer=InMemorySaver(),
)
```

# Types by Storage Mechanism



- **Buffer memory** — simple sliding window of recent turns, sent raw
- **Summarization memory** — older turns get compressed into running summaries to save context budget
- **Vector store retrieval (RAG-style)** — embeddings of past interactions/facts, retrieved by semantic similarity at query time
- **Key-value / structured stores** — explicit facts stored under keys (like your own memory filesystem here), good for deterministic lookup
- **Knowledge graphs** — entities and relationships, good for multi-hop reasoning over accumulated facts
- **Hybrid** — most production agents combine a few of these (e.g., buffer + vector store + structured profile)

# Summarization memory — trims/compresses long context



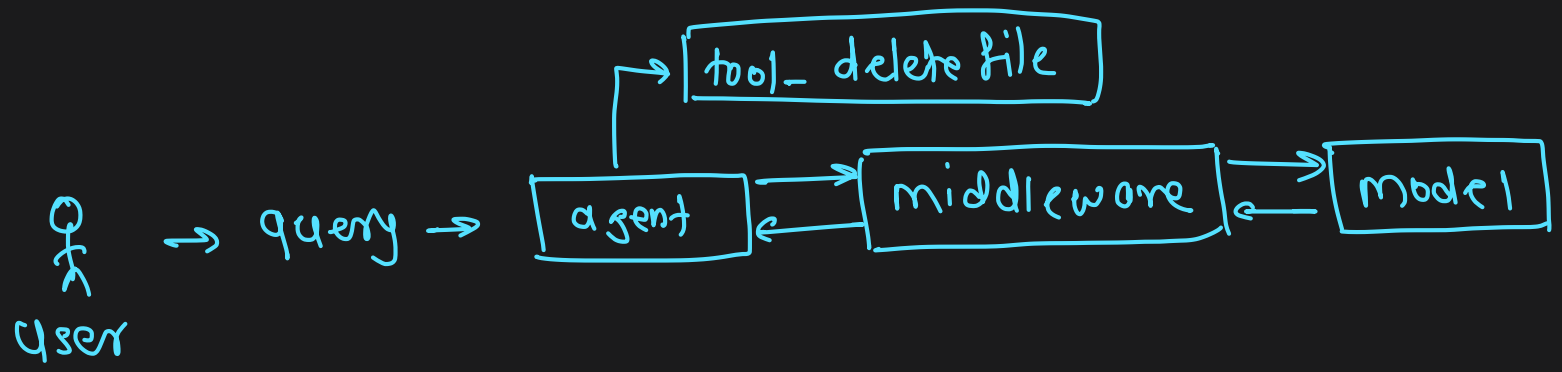
```python
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger={"tokens": 4000},    # summarize once history exceeds this
            keep={"messages": 20},       # retain last 20 raw messages
        )
    ],
)
```

# Types by Retrieval Strategy



- **Always-inject** — certain memory (user profile, system config) is loaded into every prompt regardless of relevance
- **Retrieval-on-demand** — agent calls a "search memory" tool when it decides it needs past context
- **Automatic relevance filtering** — a retrieval step runs before the agent responds, injecting only top-k relevant memories
- **Reflection-triggered writes** — a separate process (sometimes a second LLM call) reviews a finished interaction and decides what's durable enough to write to long-term storage (rather than the agent deciding inline)




# Middleware

# What is middleware?

- **Middleware is code that plugs into specific points of that loop and runs automatically at each cycle**, without you having to rewrite the agent's core logic
- It's the same concept as HTTP middleware in a web framework (like Express or Django) — a piece of logic that intercepts a request/response pipeline and can inspect, modify, block, or redirect it before it reaches its destination
- Agent middleware does the same thing, but to the model-call / tool-call loop instead of an HTTP request
- Without middleware, if you wanted to, say, trim old messages before every model call, you'd have to fork and rewrite the agent's internal execution graph yourself
- Middleware gives you defined "hook points" to attach that logic instead:
  - **Before the model is called** — e.g., trim/summarize history, inject extra context
  - **After the model responds** — e.g., check the output for policy violations, block a bad answer
  - **Before agent starts / after agent finishes** — e.g., one-time setup, final output validation
  - **Around a tool call** — e.g., catch errors, ask for human approval, redact sensitive data before it reaches a tool

# SummarizationMiddleware

- Compresses old messages once a token threshold is hit

```python
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

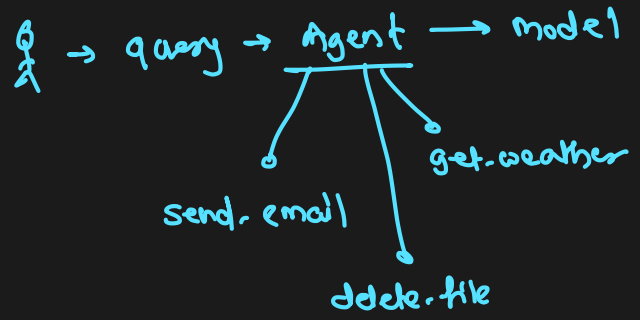
agent = create_agent(
    "gpt-4o",
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger={"tokens": 4000},
            keep={"messages": 20},
        )
    ],
)
```

# HumanInTheLoopMiddleware

- Pauses before risky tool calls, waits for approval

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    "claude-sonnet-4-6",
    tools=[read_email, send_email],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"send_email": True})],
)
```



# PIIMiddleware



- Detects/redacts/masks/blocks PII (email, phone, SSN, custom regex)

```python
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    "gpt-4o",
    middleware=[
        PIIMiddleware("email", strategy="redact"),
        PIIMiddleware("credit_card", strategy="mask"),
    ],
)
```

# LLMToolEmulator



- Fakes tool outputs via an LLM — for testing without real side effects

```python
from langchain.agents.middleware import LLMToolEmulator

agent = create_agent(
    "gpt-4o",
    tools=[get_weather, send_email],
    middleware=[LLMToolEmulator(tools=["get_weather"])],  # fake this tool's output
)
```

# ModelCallLimitMiddleware



- Caps how many model calls an agent can make

```python
from langchain.agents.middleware import ModelCallLimitMiddleware

agent = create_agent(
    "gpt-4o",
    middleware=[ModelCallLimitMiddleware(max_calls=10)],
)
```

# ToolCallLimitMiddleware



- Caps how many tool calls an agent can make

```python
from langchain.agents.middleware import ToolCallLimitMiddleware

agent = create_agent(
    "gpt-4o",
    tools=[search_db, book_meeting],
    middleware=[ToolCallLimitMiddleware(max_calls=5)],
)
```

# Guardrail



- A **guardrail** is a rule or check that constrains what an AI system is allowed to do or say — it sits between the model's raw capability and the real-world action, catching things that shouldn't happen before they cause harm
- **They typically enforce**
  - Content safety — block toxic, biased, or harmful output
  - Data privacy — prevent leaking PII, secrets, credentials
  - Scope control — keep the agent from answering things it shouldn't (e.g. medical/legal advice)
  - Action safety — require approval before risky operations (deleting data, sending money, sending emails)
  - Format/quality control — reject malformed or hallucinated structured outputs
  - Input validation — filter malicious or manipulative prompts before they reach the model (prompt injection defense)
- **Two Broad Categories**
  - **Input guardrails** — checks applied to what goes into the model (user prompts, retrieved documents, tool results)
  - **Output guardrails** — checks applied to what comes out of the model (its response, or a tool call it wants to make)

# Built-in Guardrail Middleware



```python
from langchain.agents.middleware import PIIMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    "gpt-4o",
    tools=[send_email, delete_record],
    middleware=[
        PIIMiddleware("credit_card", strategy="block"),   # hard-block instead of reda
        HumanInTheLoopMiddleware(interrupt_on={"delete_record": True}),
    ],
)
```

# Custom Guardrail — validate output before returning

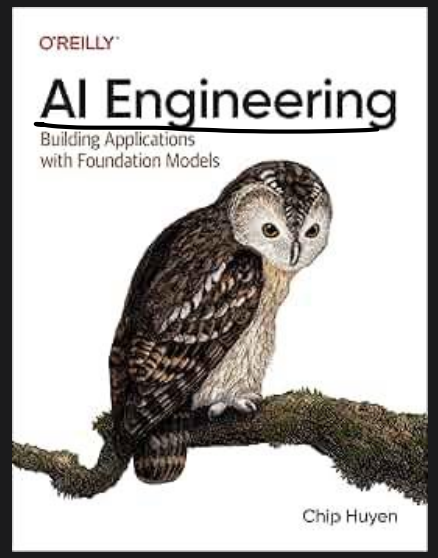


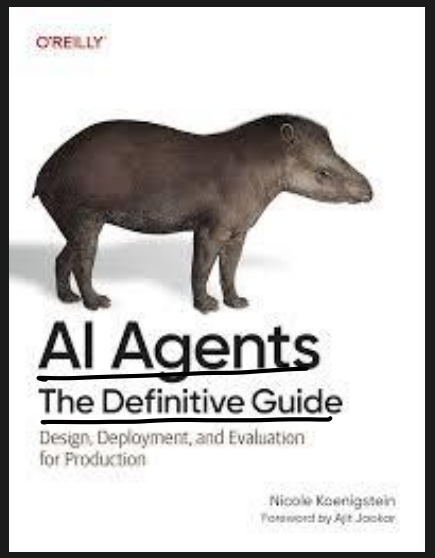
```python
from langchain.agents.middleware import after_agent, AgentState, hook_config
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model

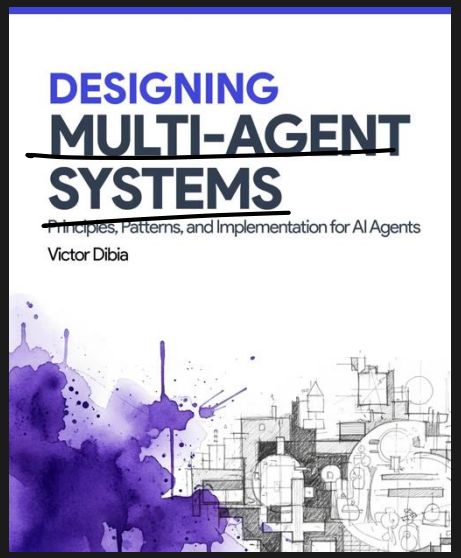
@after_agent
def validate_output(state: AgentState, runtime) -> dict | None:
    last = state["messages"][-1]
    if isinstance(last, AIMessage) and "confidential" in last.content.lower():
        return {"messages": [AIMessage("I can't share that information.")]}
    return None

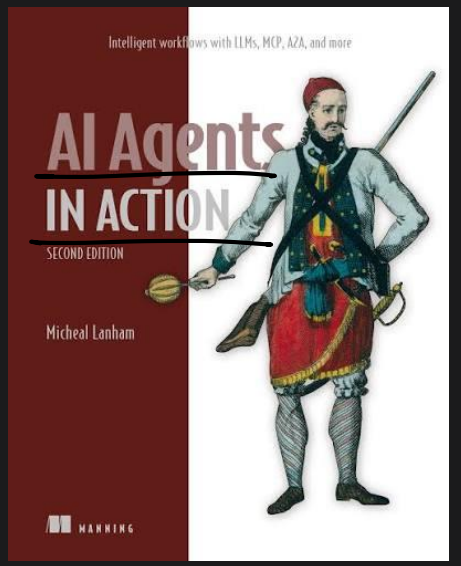
agent = create_agent(
    "gpt-4o",
    tools=[...],
    middleware=[validate_output],
)
```

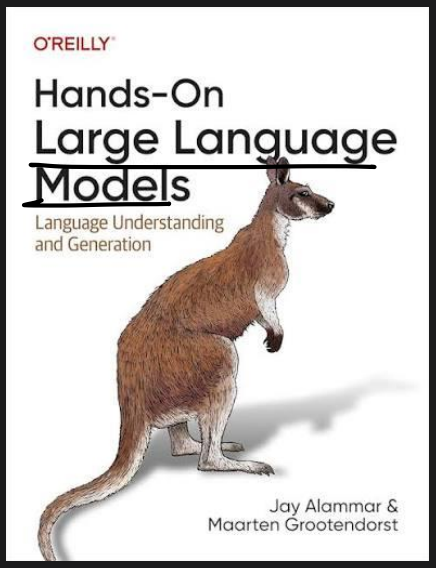
# Books


O'REILLY
AI Engineering
Building Applications with Foundation Models
Chip Huyen


O'REILLY
AI Agents
The Definitive Guide
Design, Deployment, and Evaluation for Production
Nicole Koenigstein


DESIGNING
MULTI-AGENT SYSTEMS
Principles, Patterns, and Implementation for AI Agents
Victor Dibia


Intelligent workflows with LLMs, MCP, A2A, and more
AI Agents IN ACTION
SECOND EDITION
Micheal Lanham
MANNING


O'REILLY
Hands-On Large Language Models
Language Understanding and Generation
Jay Alammar &
Maarten Grootendorst

# Use Cases

# Software Development



- **Code Review & Refactoring Agent**
  - Takes a codebase, identifies issues (bugs, performance, security), refactors code, runs tests, and generates a report
  - Without agency: You'd manually review each file, suggest fixes, wait for feedback
  - With agency: Autonomous improvement cycle
- **Bug Fixing Agent**
  - Reads error logs, reproduces bugs, explores codebase, writes fixes, runs tests, validates solution
  - Saves developers hours of manual debugging

# Research & Data Analysis



- **Market Research Agent**
  - Autonomously searches for competitor info, analyzes pricing, collects customer reviews, synthesizes findings into a report
  - Without agency: Researcher manually visits 20+ sites, takes notes, synthesizes
  - With agency: Comprehensive analysis in minutes
- **Academic Paper Summarizer**
  - Takes 50 papers in your field, extracts key findings, identifies gaps, organizes by theme, flags contradictions
  - Saves months of literature review work
- **Financial Analysis Agent**
  - Monitors stock news, extracts market sentiment, correlates with price movements, generates trading signals

# Business Operations

- **Sales Lead Qualification & Outreach**
  - Receives leads, researches companies, personalizes outreach emails, tracks responses, and schedules follow-ups
  - Multiplies sales team productivity
- **HR Recruitment Agent**
  - Screens resumes, schedules interviews, sends rejection emails, tracks candidate pipelines
  - Reduces hiring cycle time dramatically
- **Customer Support Agent**
  - Handles inquiries by searching knowledge bases, troubleshooting, escalating complex issues, and closing tickets
  - Works 24/7 without burnout

# Education

- **Curriculum Design Agent**
  - Takes learning objectives, researches best practices, designs course structure, creates lesson plans, generates assessments
  - Could dramatically reduce curriculum development time
- **Student Learning Agent**
  - Monitors student progress, identifies knowledge gaps, generates personalized practice problems, provides explanations, tracks improvement
  - Adaptive learning at scale
- **Course Content Generator**
  - Converts textbook chapters into study notes, creates practice questions, generates flashcards, produces video scripts
  - (Similar to your work creating study notes and course materials)

# Design Patterns

# ReAct (Reason + Act)



- **Concept:** An iterative loop where the agent interleaves reasoning (Thought), action execution (Tool Call), and environment observation (Tool Output) until it arrives at a final answer
- **Flow:** User Request ⟶ Thought (Reasoning) ⟶ Action (Tool Call) ⟶ Observation (Output) ⟶ Final Answer
- **Best used in:** Interactive problem solving, financial ratio calculations, live database queries, and general-purpose tool-assisted reasoning
- **Best Frameworks:** Google GenAI SDK / ADK, LangChain Agents, LlamaIndex
- **Capstone Project:**
  - **Title:** Structured Financial Portfolio Analyst & Risk Assessor
  - **Summary:** Analyzes stock portfolios against real market metrics from JSON files, computes Sharpe Ratios and daily Value-at-Risk (VaR), and generates structured rebalancing plans
  - **Framework Used:** Google GenAI SDK / ADK
  - **Framework Features used:** google-genai function calling, Pydantic structured output validation, and local model inference (llama3.2:3b)

# Plan-and-Execute

- **Concept:** Separates planning from execution by having a planner agent generate a comprehensive, sequential execution plan before worker agents execute each phase deterministically
- **Flow:**

  Goal ⟶ [Planner Agent] ⟶ Step 1 ⟶ Step 2 ⟶ Step 3 ⟶ Execution Verification
- **Best used in:** Cybersecurity incident response, multi-step cloud migrations, and structured data extraction
- **Best Frameworks:** Microsoft AutoGen, LangChain Plan-and-Solve
- **Capstone Project:**
  - **Title:** Cybersecurity SOC Incident Response Orchestrator
  - **Summary:** Generates and executes a 4-phase NIST incident response plan: SIEM log detection, threat intel correlation, firewall isolation, and CISO executive briefing
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** Multi-stage sequential execution pipeline, tool calling for firewall and IAM actions, and NIST incident logging

# Reflection



- **Concept:** An Actor agent generates an output (e.g., code or text), and an independent Critic agent (or deterministic linter) reviews it against quality/security rules, feeding back specific fixes until the output meets quality standards
- **Flow:**

$$\text{Draft Artifact} \longrightarrow [\text{Critic / AST Scanner}] \xrightarrow{\text{Issues Found}} \text{Actionable Critique} \longrightarrow \text{Refined Final Artifact}$$

- **Best used in:** Automated code generation, AST security auditing, SQL query tuning, and contract drafting.
- **Best Frameworks:** OpenAI Agent SDK, LangGraph, AutoGen.
- **Capstone Project:**
  - **Title:** Autonomous Pull Request Reviewer & Security Bug Fixer
  - **Summary:** Scans PR code using AST analysis to detect SQL injections and unsafe eval calls, provides constructive critiques, and synthesizes a verified git patch.
  - **Framework Used:** OpenAI Agent SDK Pattern
  - **Framework Features used:** AST Python visitor, deterministic security rules engine, and unified diff generator.

# Self-Consistency

- **Concept:** A reasoning optimization pattern where the system samples multiple distinct reasoning paths / solutions for the same problem (using varied sampling temperatures, prompts, or diverse agent personas) and selects the most reliable answer via **majority voting, consensus scoring, or automated validation**
- **Flow:**

$$\text{Complex Problem} \longrightarrow \begin{cases} \textit{Reasoning Path 1} \longrightarrow \textit{Candidate Solution A} \\ \textit{Reasoning Path 2} \longrightarrow \textit{Candidate Solution B} \longrightarrow [\textit{Majority Voting / Consensus Engine}] \\ \textit{Reasoning Path 3} \longrightarrow \textit{Candidate Solution A} \end{cases} \longrightarrow \textit{Winning Solution A}$$

- **Best used in:** Complex mathematical calculations, multi-constraint software architecture design, mission-critical code generation, and multi-agent debate consensus
- **Best Frameworks:** Microsoft AutoGen, LangChain / LangGraph, DSPy
- **Capstone Project:**
  - **Title:** Multi-Persona Self-Consistency & Consensus Engine
  - **Summary:** Generates multiple independent technical implementation strategies across distinct personas (Product Manager, Lead Architect, Developer), evaluates test suites in an isolated sandbox, and selects the consensus implementation that passes all criteria.
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** ConversableAgent, multi-turn debate consensus, and local code verification sandbox.

# Tool-Calling (Function Execution)



- **Concept:** The agent translates natural language user intents into structured function parameters, invokes local Python functions or external REST APIs, and uses the returned data to complete the task.
- **Flow:** Natural Language Intent ⟶ Schema Argument Extraction ⟶ Function Execution ⟶ Result Grounding
- **Best used in:** Currency conversions, policy rule checks, database lookups, and API integrations.
- **Best Frameworks:** LangChain, OpenAI Agent SDK, Google ADK.
- **Capstone Project:**
  - **Title:** Autonomous Travel & Expense Policy Auditor
  - **Summary:** Converts multi-currency receipts via live Forex APIs and enforces city-by-city per-diem spending caps against policy data in JSON.
  - **Framework Used:** LangChain
  - **Framework Features used:** @tool decorator, LCEL tool binding (bind_tools), and structured argument schemas

# Routing & Dynamic Dispatching

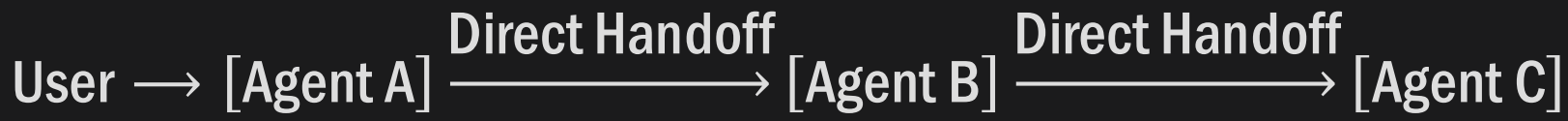
- **Concept:** An intelligent classification gate that evaluates the user prompt and routes execution to the appropriate specialist agent, tool, or downstream workflow to minimize latency and token cost
- **Flow:**

$$\text{Incoming Request} \longrightarrow [\text{Classifier Router}] \begin{cases} \longrightarrow \text{Billing Specialist} \\ \longrightarrow \text{Technical Escalation} \\ \longrightarrow \text{Standard Auto-Reply} \end{cases}$$

- **Best used in:** Multi-tier customer support portals, helpdesk triage, and omnichannel webhook dispatchers
- **Best Frameworks:** OpenAI Agent SDK, n8n AI Workflows, Semantic Router
- **Capstone Project:**
  - **Title:** Customer Support Triage Swarm
  - **Summary:** Ingests support inquiries, routes users to Front-Desk, Billing, or Technical Escalations based on intent, and updates customer accounts
  - **Framework Used:** OpenAI Agent SDK (Swarm Pattern)
  - **Framework Features used:** Swarm Agent definitions, Context Hand-off functions, and typed customer state persistence

# Multi-Agent Collaboration (Swarm & Handoffs)

- **Concept:** A decentralized architecture where specialized peer agents pass conversational execution and state directly to one another without bottlenecking through a central orchestrator
- **Flow:**




- **Best used in:** Customer service concierges, e-commerce support, and multi-department internal ticketing
- **Best Frameworks:** OpenAI Agent SDK, Google ADK Swarm
- **Capstone Project:**
  - **Title:** Multi-Tier Customer Care & Handoff Swarm
  - **Summary:** Front-Desk triage smoothly delegates to Billing (for refunds) and SRE (for cluster downtime) with complete conversation history retention
  - **Framework Used:** OpenAI Agent SDK
  - **Framework Features used:** Agent transfer callbacks (transfer_to_billing, transfer_to_tech), shared state objects, and tool registries

# Supervisor (Hierarchical Plan-and-Solve)

- **Concept:** A centralized Lead Supervisor agent decomposes a high-level task into sub-tasks, dispatches them across specialized worker agents or subgraphs in parallel, and aggregates their findings into a cohesive report
- **Flow:**

$$\text{Mandate} \longrightarrow [\text{Supervisor}] \begin{cases} \longrightarrow [\text{Macro Worker}] \\ \longrightarrow [\text{Valuation Worker}] \\ \longrightarrow [\text{Moat Worker}] \end{cases} \longrightarrow \text{Synthesized Executive Memo}$$

- **Best used in:** M&A due diligence, institutional equity research, medical tumor boards, and competitive intelligence
- **Best Frameworks:** LangGraph, CrewAI (Hierarchical mode)
- **Capstone Project:**
  - **Title:** Hierarchical Enterprise Due Diligence Supervisor
  - **Summary:** Coordinates three parallel subgraphs analyzing market trends, DCF valuations, and competitor moat data from JSON records into an investment thesis
  - **Framework Used:** LangGraph
  - **Framework Features used:** Supervisor routing nodes, parallel subgraph invocation, and state reduction (operator.add)

# Human-in-the-Loop (HITL)

- **Concept:** The agent workflow executes autonomously until it encounters a high-risk decision threshold, where it automatically pauses, persists state, notifies a human operator for review/approval, and resumes upon input
- **Flow:**

$$\text{Auto-Processing} \longrightarrow \text{Threshold Check} \xrightarrow{\text{High Risk}} [\text{PAUSE: Awaiting Human Decision}] \longrightarrow \text{Resume Execution}$$

- **Best used in:** Mortgage loan underwriting, medical prescriptions, wire transfer authorizations, and live database migrations
- **Best Frameworks:** LangGraph, Temporal AI
- **Capstone Project:**
  - **Title:** Stateful Loan Underwriting with Human-in-the-Loop
  - **Summary:** Calculates credit metrics and Debt-to-Income (DTI), triggers a breakpoint interrupt when DTI exceeds safe limits, accepts human underwriter adjustments, and finalizes the loan state
  - **Framework Used:** LangGraph
  - **Framework Features used:** StateGraph, interrupt(), persistent JSON state checkpoints, and conditional resumption edges

# Retrieval-Augmented Generation (RAG) & Memory



- **Concept:** Combines conversational memory buffers with semantic/keyword retrieval from knowledge bases to ground agent reasoning in verified enterprise source documents
- **Flow:**
  User Query + Conversation Memory ⟶ Document Retrieval ⟶ Grounded Answer + Official Citations
- **Best used in:** Enterprise HR assistants, compliance policy bots, customer onboarding, and technical documentation lookup
- **Best Frameworks:** LangChain v0.3 LCEL, LlamaIndex
- **Capstone Project:**
  - **Title:** Multi-Source Enterprise Knowledge Synthesizer
  - **Summary:** Answers employee questions regarding remote office stipends, AI security, and travel rules by querying JSON policy documents and attaching verified document citations (HR-POL-042).
  - **Framework Used:** LangChain LCEL
  - **Framework Features used:** ChatPromptTemplate, sliding-window conversation memory buffer, and hybrid document retriever

# Evaluator–Optimizer Pattern

- **Concept:** An iterative refinement pattern where one component (the **Optimizer / Generator**) produces a candidate output, while a second independent component (the **Evaluator / Critic**) assesses it against strict criteria, rubrics, or safety constraints. If defects are found, actionable feedback is returned to the Optimizer to refine the output in a closed-loop cycle until the quality threshold is satisfied.
- **Flow:**

Task Request ⟶ [*Optimizer (Generates Candidate)*]

⟶ [*Evaluator (Scores & Screens)*] { *Issues Found (Feedback)* ⟶ [*Optimizer (Refines Artifact)*] ; *Passes All Criteria* ⟶ *Final Validated Output* }

- **Best used in:** High-accuracy code generation with test suites, medical/clinical trial screening against contraindication rules, legal contract drafting, translation refinement, and complex mathematical problem solving.
- **Best Frameworks:** Google GenAI SDK / ADK, LangGraph (Looping Graphs), OpenAI Agent SDK, LlamaIndex Reflexion.
- **Capstone Project:**
  - **Title:** Multimodal Clinical Trial & Pharmacovigilance Matcher
  - **Summary:** The Optimizer agent identifies candidate clinical trials for an oncology patient based on primary diagnosis, while an independent Evaluator screens the candidate list against strict genetic biomarker alterations (EGFR, PD-L1) and drug-drug contraindications (e.g., CYP3A4 inducers) from JSON files to optimize trial eligibility and eliminate patient risk.
  - **Framework Used:** Google GenAI SDK / ADK Pattern
  - **Framework Features used:** Pydantic typed schemas (ClinicalDocument, ClinicalMatchReport), multi-step rule evaluation gates, and local model inference

# Panel of Experts (Multi-Agent Debate)



- **Concept:** Multiple agents with distinct personas and conflicting objectives participate in a shared discussion channel to critique, challenge, and collaboratively solve a complex problem
- **Flow:**

  Product Manager ⟷ System Architect ⟷ Senior Developer ⟷ QA Engineer
- **Best used in:** End-to-end software development, architectural reviews, and cross-functional simulations
- **Best Frameworks:** Microsoft AutoGen, CrewAI
- **Capstone Project:**
  - **Title:** Autonomous Agile Software Engineering Team
  - **Summary:** Product Manager writes user stories, Architect creates technical design, Developer writes Python rate limiter code, and QA verifies in an isolated sandbox
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** ConversableAgent, GroupChat, GroupChatManager, and local Python code execution sandbox

# Hierarchical Agents (Delegated Role Specialization)

- **Concept:** A structured organization of agents where senior agents delegate domain-specific tasks to specialized junior analyst agents and consolidate their output
- **Flow:**

$$[\textit{Chief Investment Officer}] \begin{cases} \longrightarrow [\textit{Financial Filing Analyst}] \\ \longrightarrow [\textit{DCF Valuation Modeler}] \longrightarrow \textit{Investment Rating} \\ \longrightarrow [\textit{Chief Risk Officer}] \end{cases}$$

- **Best used in:** Hedge fund research, deep market intelligence, complex audit investigations, and enterprise reporting
- **Best Frameworks:** CrewAI, LangGraph
- **Capstone Project:**
  - **Title:** Institutional Market Intelligence & Equity Research Crew
  - **Summary:** Coordinates a Chief Investment Officer, SEC Financial Filing Analyst, and Chief Risk Officer to analyze 10-K data from JSON records and generate a buy/sell rating
  - **Framework Used:** CrewAI
  - **Framework Features used:** Agent, Task, Crew(process=Process.hierarchical), role-based persona prompting, and custom tool bindings

# Event-Driven Agents & Reactive Pipelines



- **Concept:** Agents that activate in response to asynchronous triggers (HTTP Webhooks, file events, message queues) to execute chained steps without requiring manual user intervention
- **Flow:**

  Webhook Event ⟶ State Initialization ⟶ Enrichment Step ⟶ Action / Notification
- **Best used in:** Marketing automation, CRM lead qualification, continuous integration pipelines, and real-time fraud alerts.
- **Best Frameworks:** CrewAI Flows, n8n AI Workflows, Temporal.
- **Capstone Project:**
  - **Title:** Event-Driven B2B Prospect Enrichment & Outreach Flow
  - **Summary:** Listens for inbound business leads, scrapes tech signals from Hacker News and JSON records, calculates lead scores, and generates personalized executive email outreach.
  - **Framework Used:** CrewAI Flows
  - **Framework Features used:** @start, @listen decorators, typed state models (Flow[LeadState]), and Pydantic validation.

# Checkpoint & State Recovery

- **Concept:** Persisting intermediate execution state at every node transition, allowing long-running multi-step agents to recover from failures or restart exactly where they left off without losing work
- **Flow:**




- **Best used in:** Long-running workflows, multi-day research agents, asynchronous human approvals, and fault-tolerant pipelines
- **Best Frameworks:** LangGraph (SqliteSaver, MemorySaver, JSON State Store), Temporal
- **Capstone Project:**
  - **Title:** Stateful Loan Underwriting Pipeline (code-demos/05_langgraph/app_09_human_in_the_loop_loan_approval)
  - **Summary:** Persists complete underwriting telemetry snapshots into data/loan_applications.json at each stage, ensuring full auditability and zero state loss across interruptions
  - **Framework Used:** LangGraph
  - **Framework Features used:** Durable JSON state checkpointing, thread tracking, and audit trail accumulation

# ReAct

# Why Do We Need ReAct?



- **Vanilla LLMs have fatal flaws**
  - Hallucination
    - LLMs guess when they don't know
    - E.g."The CEO of Acme Corp is John Smith" (made up)
  - No External Data
    - Can't access live information
    - Can't tell you today's weather or stock price
  - Single Pass
    - One chance to get it right
    - Can't adapt when initial approach fails
- Some tasks can't be solved in a single LLM call
- When a question requires looking up data, processing it, and making a decision based on the result, a one-shot response will either hallucinate the answer or give a shallow one

# The ReAct Analogy



- **Think Like a Detective**
  - The detective doesn't just stare at the evidence and guess the answer
  - And they don't just randomly run around town looking for clues without thinking.
- **The Detective's Process:**
  - **Think** about what they know and need to find out
  - **Act** by looking up a record or visiting a location
  - **Observe** the new information
  - **Think** again about how this changes the theory
  - **Act** by interrogating a suspect
  - **Observe** the alibi
  - **Think** again... and finally conclude
- **ReAct does exactly this, but for AI agents!**

# What is ReAct?



- **ReAct** (Reasoning + Acting) is an architectural pattern that interleaves **chain-of-thought reasoning** with **tool-calling actions** in a tight, synergistic feedback loop
- **Introduced by:** Yao et al., 2022 - "Synergizing Reasoning and Acting in Language Models"
- It is a behavior triggered by how you write your prompt, not by direct changes you make to the model
- This setup turns a basic chat into a structured process where the model explains its plan before it ever touches a tool

# Step-by-Step

- **Thought:** AI analyzes the problem and plans next step
- **Action:** AI calls an external tool or API
- **Observation:** System feeds the tool result back
- **Repeat** until enough information is gathered
- **Final Answer:** Deliver the response

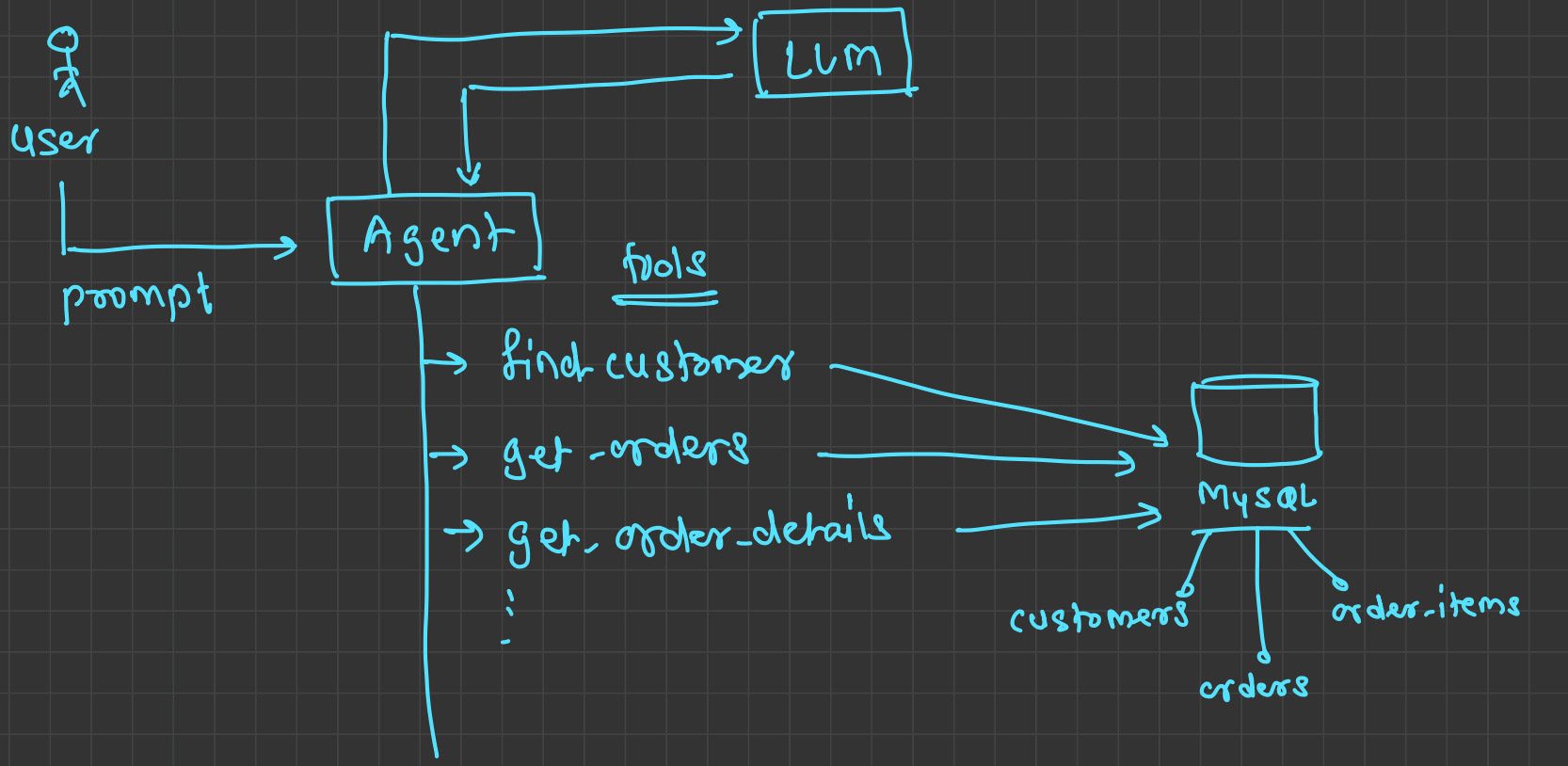


# Detailed Example



- **User:** "What is the weather in Tokyo right now, and should I pack an umbrella?"
- **Thought 1:** I need the current weather in Tokyo. I will call the weather API.
- **Action 1:** get_weather(city="Tokyo")
- **Observation 1:** "Current conditions: 22°C, Overcast, 80% humidity, light rain expected in 2 hours."
- **Thought 2:** Rain is expected in 2 hours. I should advise packing an umbrella. Also check if a jacket is needed.
- **Action 2:** compare_temp(22°C)
- **Observation 2:** "22°C is mild. Jacket not required."
- **Thought 3:** I have all the information needed.
- **Final Answer:** "It is currently 22°C and overcast in Tokyo. Light rain is expected in about 2 hours, so yes, you should pack an umbrella. A jacket is not necessary for this temperature."

# Customer support Agent



# Structural Components

- Reasoning Engine (LLM)
  - Interprets prompts, creates plans
  - Generates Thought-Action traces
- Tool Layer (External Functions/APIs)
  - Search engines, databases, APIs
  - Executes actions and returns observations
- Working Memory (Context Window)
  - Stores thoughts, actions, observations
  - Maintains conversation history
- Scratchpad (Reasoning Trace)
  - Visible step-by-step logic
  - Auditable decision trail
- Orchestrator (Control Loop)
  - Manages Think-Act-Observe cycle
  - Passes outputs to tools, feeds results back

# Advantages of ReAct



- **Enhanced Reasoning**
  - Transparent thinking process
  - Better decisions through thoughtful consideration
  - Error recognition and correction
- **Improved Accuracy**
  - Grounded in external facts, not hallucinations
  - Verification loops for double-checking
  - Adaptive strategies based on observations
- **Debuggability**
  - Reasoning traces show why decisions were made
  - Step-by-step inspection
  - Failure analysis capabilities
- **Dynamic Adaptation**
  - Context-aware adjustments
  - Flexible, non-predetermined sequences
  - Recovers from errors with alternative approaches

# Limitations



- **Increased Latency**
  - Issue: Multiple LLM calls add time
  - Impact: 3-10x slower than single-shot responses
  - Each cycle requires a separate forward pass
- **Higher Token Costs**
  - Issue: Reasoning traces consume tokens without direct output
  - Impact: 2-5x more tokens than direct tool use
  - One unbounded query can cascade into 50-500 LLM calls costing $5-$50
- **Potential for Infinite Loops**
  - Agent can get stuck in unproductive cycles
  - Example: "Page not found" → "I'll search again" → "Page not found"...
  - Must implement maximum iteration limits
- **Context Window Limitations**
  - All thoughts, actions, observations must fit in context
  - Complex tasks with many steps can exceed token limits
  - Risk of context loss and inaccurate results

# When to use ReAct?



- **Interactive tasks** where the goal isn't well-defined upfront
- **Dynamic user queries** that require adaptation (e.g., document chat, follow-up questions)
- **Exploratory data analysis** - "What patterns do you see in this report?"
- **Zero-shot reasoning** with unpredictable inputs
- **Need to learn** what tools your agent actually requires
- **Debuggability** and auditability are important

# When to avoid?

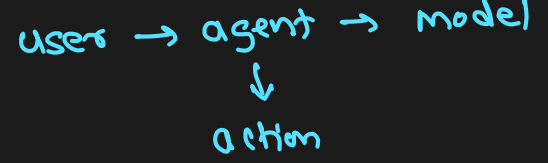


- Control flow is known at design time (use a workflow instead)
- Latency SLOs are strict
- Short-horizon tasks (fewer than 10 steps)
- Domain requires auditable deterministic branches
- Compliance and auditability must be strictly enforced

# Human In the Loop

# What is HITL?

- Human-in-the-Loop (HITL) is a design pattern in AI systems where human judgment is integrated into the automated decision-making process at critical points
- Instead of allowing AI to operate completely autonomously, HITL creates a collaboration where:
  - AI proposes actions, recommendations, or decisions
  - Humans review, validate, or modify these proposals
  - Only then are actions executed
- AI has fundamental limitations
  - **Hallucinations:**
    - AI confidently generates false information
    - It cannot distinguish between fact and fiction
  - **Lack of True Understanding**
    - AI processes patterns, not meaning
    - It doesn't truly "understand" context, nuance, or consequences
  - **Bias Amplification**
    - AI inherits biases from training data
    - It amplifies historical inequalities



# The Problem with Agentic AI

- Agentic AI doesn't just answer questions—it takes actions too

- Delete database records
  - IRREVERSIBLE
  - Data loss, business disruption
- Send emails
  - IRREVERSIBLE
  - Reputation damage, legal liability
- Process payments
  - IRREVERSIBLE
  - Financial loss, fraud
- Update production code
  - IRREVERSIBLE
  - System crashes, security breaches
- Modify legal documents
  - IRREVERSIBLE
  - Contract violations, lawsuits
- Schedule appointments
  - Partially reversible
  - Wasted time, customer dissatisfaction

# With or Without HITL

- **Without HITL**




- **With HITL**



# Why is it required?



- **AI is not reliable enough** to act without oversight
- **AI is not accountable** for its actions
- **AI lacks context** that only humans possess
- **AI cannot evolve** with changing intent
- **Laws demand** human oversight
- **Trust requires** human involvement
- **The cost of failure** is too high

# The Design Pattern

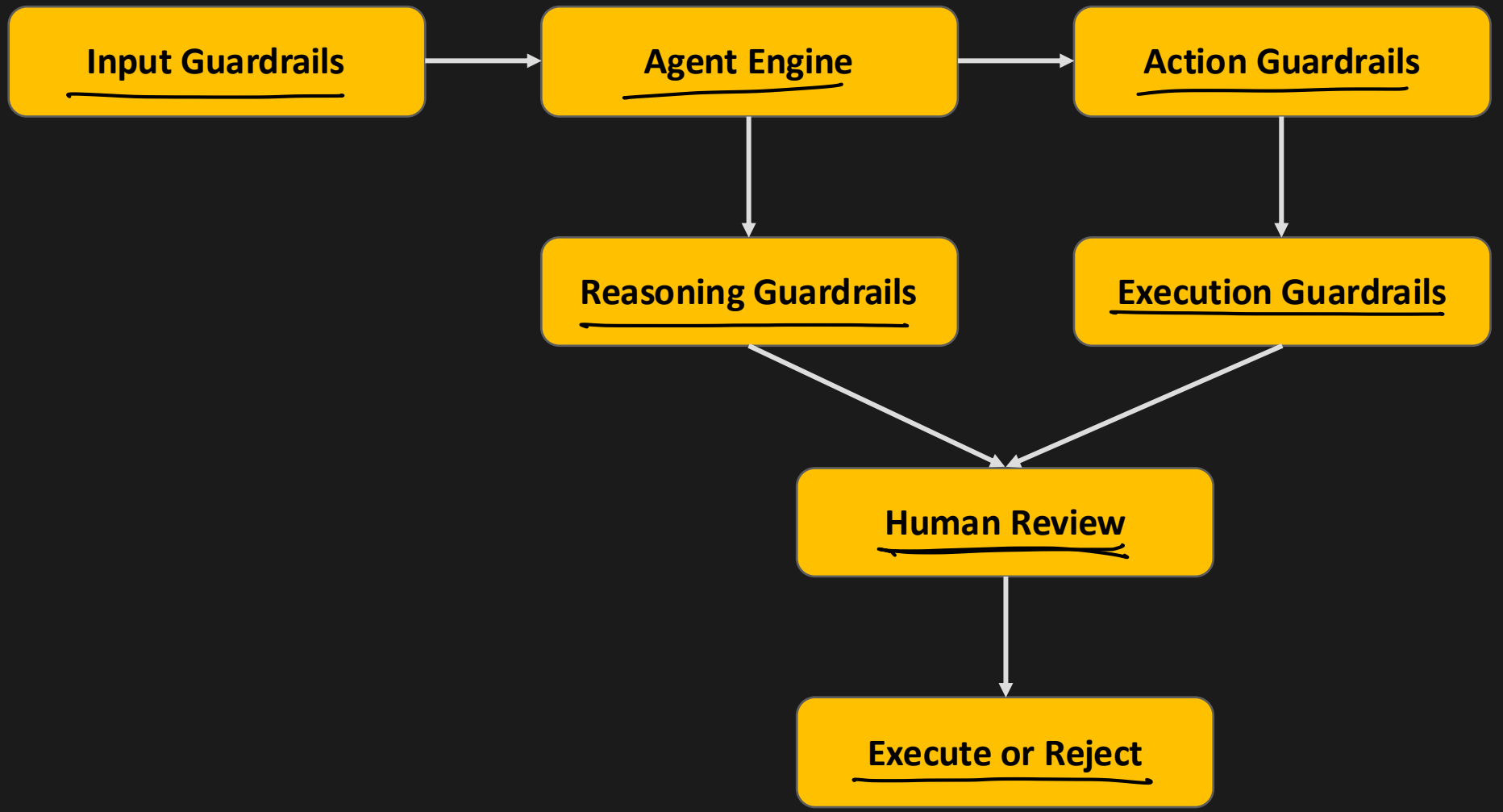
- HITL is not a single "approval button." It exists on a spectrum of human involvement, from high control to high automation
- Modern architectures have evolved to implement these patterns effectively

| Pattern | Agent Autonomy | Human Role | Best For |
|---|---|---|---|
| **Human-in-Command** | Low | Drives every step | High-risk Legal/Medical decisions |
| **Human-as-Filter** | Medium | Approves/Edits final output | Content Generation (e.g., marketing materials) |
| **Human-as-Backup** | High | Only intervenes on error/uncertainty | Customer Support, most production workflows |
| **Human-on-the-Loop** | Max | Audits logs after completion | High-volume analysis with low risk |

# Specific Implementation Patterns

- **Approval based**
  - Agent proposes action → Human approves or rejects
  - Binary decision point
  - Use case: Loan approvals, compliance checks, high-stakes binary choices
- **Review and Edit**
  - Humans actively modify AI-generated content
  - Iterative refinement process
  - Use case: Report drafting, marketing copy, creative content
- **Escalation based**
  - System dynamically hands over to human during interaction
  - Triggered by: User request, negative sentiment, low confidence
  - Use case: Customer service, complex problem-solving
- **Confirm and then execute**
  - Agent prepares action → User confirms → Execution
  - Simple, low-risk pattern
  - Use case: Scheduling meetings, sending emails
- **Propose and then review**
  - Action sent to internal reviewer for sign-off
  - Multiple approval levels possible
  - Use case: Refunds, financial adjustments, change requests

# Architectural Components



# Guardrails

- **Input Guardrails** (PII)
  - Check user requests for prompt injection
  - Detect out-of-scope content
  - Filter inappropriate language
  - Validate input format
- **Reasoning Guardrails**
  - Control how agent decides what to do
  - Prevent unsafe tool combinations
  - Enforce reasoning constraints
  - Block disallowed reasoning paths
- **Output Guardrails**
  - Check final responses for hallucinations
  - Verify policy compliance
  - Detect sensitive data leakage
  - Validate output format
- **Action Guardrails**
  - Check what agent is about to do
  - Validate action against policies
  - Require confirmation for high-risk actions
  - Log all actions for audit

# State Management & Interrupts

- **Interrupts and breakpoints**
  - Hardcoded pause points before sensitive tools
  - Agent state is "frozen" at breakpoint
  - Human sends approve/reject signal
  - Execution resumes only after signal received
  - **Example:** delete_user() tool triggers breakpoint
- **State Injection**
  - Reviewer can "go back" to previous step
  - Edit the agent's state (observations, thoughts)
  - Resume execution from edited state
  - Allows precise steering without starting over
- **Confidence based Escalation**
  - Agent calculates uncertainty score
  - If score exceeds threshold → Pause
  - Escalate to human for decision
  - Works with: Probability scores, confidence intervals

# LangGraph

# What is LangGraph?



- A low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful AI agent
- Provides control and infrastructure for complex agent workflows beyond simple loops. It allows you to mix deterministic code with LLM-driven steps reliably
- Position in the LangChain Ecosystem :
  - LangGraph is the orchestration runtime (durable execution, streaming, HITL).
  - LangChain is the higher-level agent framework (abstractions, integrations).
  - LangSmith is the platform for tracing, evaluation, and deployment
- Inspired by Google's Pregel and Apache Beam; public interface draws from NetworkX

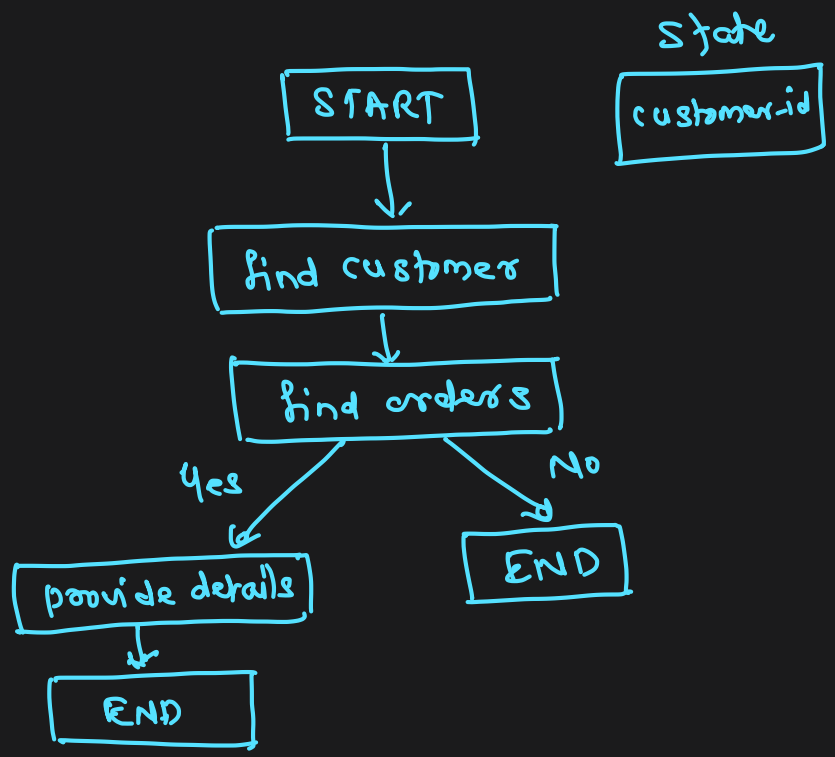
# Benefits

- **Durable Execution (Persistence):** Agents can survive failures and be paused/resumed, crucial for long-running tasks
- **Human-in-the-Loop (HITL):** Ability to pause the graph and wait for human input, review, or approval before proceeding (e.g., for sensitive actions like sending an email)
- **Comprehensive Memory:**
  - **Short-Term Memory (STM):** The state within a single thread, persisted via **Checkpointers** (keyed by thread_id). Used for conversation continuity and resuming after interruptions
  - **Long-Term Memory (LTM):** Information persisted across threads/sessions (keyed by user_id/namespace). Stored via the **Store** API for user preferences or facts
- **Streaming:** Allows you to see agent steps and tokens in real-time as they are generated
- **Debugging & Observability with LangSmith:** Provides full trace trees, latency breakdowns, and token/cost tracking to debug complex behavior

# Core Architecture - State

- The **shared**, **persistent data** that flows through the graph. It acts as the agent's working memory
- Typically defined as a TypedDict
- LangGraph maintains a shared **state** as the agent executes
- Nodes can read and update this state.

```
state = {
    "user_message": "Where is my order?",
    "order_id": "ORD123",
    "order_status": None,
    "needs_refund": False,
    "human_approved": False
}
```



# Core Architecture – Node

- **Units of work**
- Each node is a function (e.g., calling an LLM, using a tool, executing code)
- Nodes receive the current state, perform an action, and return an update to the state

```python
def find_order(state):
    order = get_order_from_database(state["order_id"])

    return {
        "order_status": order["status"]
    }
```

updated state

```python
def check_refund(state):
    if state["order_status"] == "cancelled":
        return {"needs_refund": True}

    return {"needs_refund": False}
```

updated state

# Core Architecture - Edges



- Define the **flow of control** between nodes
- **Normal Edges**
  - Directly connect one node to the next in a deterministic path (`graph.add_edge("node_a", "node_b")`)
- **Conditional Edges** → branching
  - Use a routing function to choose the next node(s) based on the current state (`graph.add_conditional_edges("node_a", routing_function)`)
- **Entry Point**
  - The first node to execute, defined from the special START node

# Core Architecture - Loop (workflow)



- This is one of the biggest differences between a simple chain and an agentic workflow
- The agent can **loop until it reaches a suitable stopping condition**
- This is very useful for implementing patterns such as **ReAct**, where the agent repeatedly reasons, acts, observes, and decides what to do next

# Key Concepts



- **Checkpointers** (MemorySaver, PostgresSaver, SqliteSaver)
  - The mechanism for saving graph state
  - MemorySaver is for development; use PostgresSaver (or SqliteSaver) for production to persist state across restarts
  - **Only the parent graph should be compiled with a checkpointer when using subgraphs** to avoid state duplication and bloat
- Interrupt
  - The primitive used to implement HITL. When a node calls interrupt(), execution pauses, and the graph returns the payload for the human to review. Resuming is done by passing a Command(resume=...)
- Command
  - Versatile primitive for controlling graph execution.
  - It allows a node to both update state and navigate to specific nodes (goto) in a single step
- Send
  - Used for advanced map-reduce patterns where the number of edges is unknown at compile time
  - For example, a node might generate a list of subjects and Send each one to a "generate_joke" node

# When to use?

- Multi-step workflows
- Conditional branching
- Loops and iterative workflows
- Human-in-the-loop
- Stateful applications
- **Retry and error handling**
- Multiple agents
- Long-running workflows

# When not to use?



- Simple LLM calls
- Simple chains
- Simple RAG
- Simple tool calling
- When the workflow doesn't require state
- Very small applications

# Scenarios



- Customer Support Agent
- Research Agent
- Plan-and-Execute Agent
- Coding Agent
- Document Processing
- Financial/Business Approval Workflow
- Medical/Legal Document Workflow
- Multi-Agent Research System
- Customer Onboarding

# Famous companies using LangGraph

| Company | Application | What LangGraph is doing |
|---|---|---|
| **LinkedIn** | AI Recruiter | Hierarchical agents for candidate search, matching and recruitment workflows |
| **Uber** | Developer Platform | Agents for code migration and automated unit-test generation |
| **Klarna** | AI Customer Assistant | Multi-step customer-support and payment workflows |
| **Replit** | Replit Agent | Coding agents that build software, with human-in-the-loop capabilities |
| **Elastic** | AI / Security | Agentic workflows for security and other complex tasks |
| **J.P. Morgan** | AI Copilot | Domain-specific enterprise copilot |
| **Rakuten** | AI Copilot | Domain-specific agent workflows |
| **Morningstar** | AI Research Assistant | Research and summarization |
| **Vodafone** | AI applications | Code generation and internal search |
| **Qodo** | Coding AI | Code-generation workflows |

# Langchain vs LangGraph

- **Choose LangChain when**
  - You need a simple LLM application
  - You are building basic RAG
  - You need prompt templates
  - You need model/tool integrations
  - You need a straightforward agent
  - Your workflow is mostly linear

- **Choose LangGraph when**
  - Your workflow has **multiple steps**
  - You need **shared state**
  - You have **conditional branching**
  - You need **loops**
  - You need **retry/re-evaluation**
  - You need **human approval**
  - You have **multiple agents**
  - You need **pause/resume**
  - You need more control over agent execution

# Plan and Execute

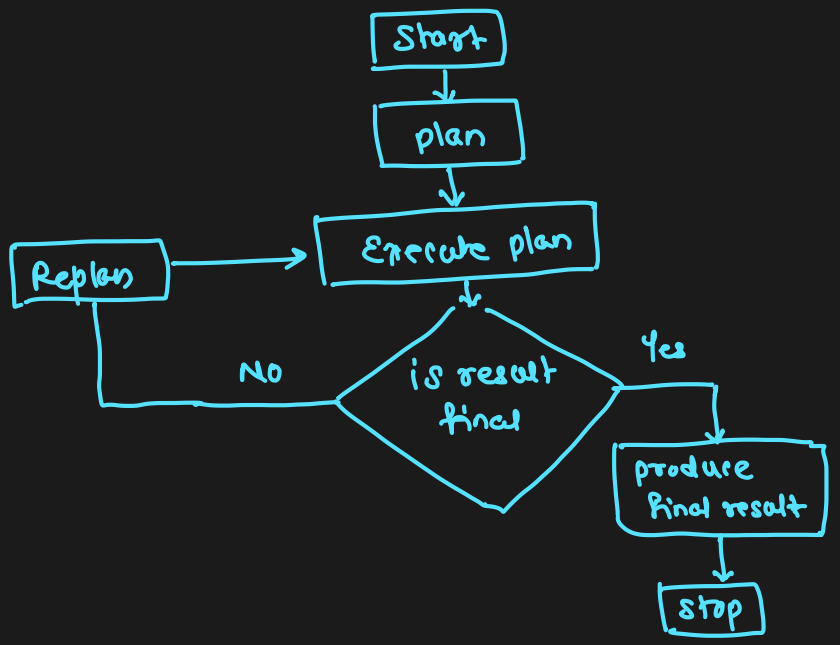
# Introduction



- The Plan-and-Execute pattern separates "decide what to do" from "do it."
- An agent first produces a plan—a structured description of the work—then executes against that plan
- The plan becomes a durable artifact that is reviewable, modifiable, checkpointable, and resumable
- This differs fundamentally from reactive patterns where the agent jumps from request to execution without explicit planning
- For complex tasks like multi-file refactors, architectural changes, or multi-step workflows, skipping the planning phase produces edits that don't account for dependencies, missing steps discovered only after partial execution, and no recovery point if execution fails partway through

# Architecture

- **Planner**
  - An LLM that parses the user request, understands intent, and generates an executable task plan—often organized as a Directed Acyclic Graph (DAG) representing task sequence and dependencies
- **Executor**
  - Executes tasks from the plan
  - May be a simple function caller or a full ReAct agent with tool access
- **Replanner**
  - Evaluates updated state after execution.
  - If results are insufficient, replans additional tasks and resubmits them
  - If sufficient, prepares final results
- The workflow follows a loop
  - User Request → Planner → Task DAG → Executor → State Update → Replanner → (loop or respond)



# Implementation Approaches

- **Basic Loop Implementation**
  - A straightforward LangGraph implementation uses plan and execute nodes in a cycle
  - The state typically tracks the task, current plan (remaining steps), and past steps with their results
  - The executor node takes the first step from the plan, executes it, and returns the updated state with that step removed and its result appended to history → state
- **Multi-Agent Implementation**
  - An alternative approach uses separate plan and execute agents without cycles
  - The plan agent's output becomes input for the execute agent, providing clear guidance through multi-step reasoning while achieving faster execution time compared to cyclic approaches
  - This is implemented by passing the plan as context in the execute agent's prompt
- **Model Tiering Strategy**
  - Production implementations often use different models for different roles
  - **Planner:** High-performance model for quality thinking and strategic decomposition
  - **Executor:** Cheaper, faster model for straightforward step execution
  - **Replanner:** Balanced model for judgment accuracy

# Advanced Variants

- **LLMCompiler**
  - This architecture speeds up agent tasks by executing them in parallel through a DAG
  - It saves redundant token use by reducing LLM calls
  - The system has three components: a Function Calling Planner that formulates execution plans, a Task Fetching Unit that dispatches function calling tasks, and an Executor that runs tasks in parallel
  - Benchmarks show consistent latency speedups of up to 3.7x with approximately 9% cost savings
- **Plan & Solve**
  - A simpler variant where planning and execution are distinct phases
  - The agent produces a plan, then executes it without the replanning loop.
  - Suitable for tasks requiring structured decomposition but not dynamic adaptation
- **Hierarchical Planning (Hybrid)**
  - The most sophisticated systems combine Plan-and-Execute with ReAct
  - A high-level planner creates a strategic plan of complex sub-goals
  - Each sub-goal is then handled by a specialized ReAct agent for tactical adaptability
  - This leverages strategic foresight from Plan-and-Execute and tactical flexibility from ReAct

# When to Choose Plan-and-Execute



- **Choose this pattern when**
  - The task is well-structured and the environment is stable
  - Cost and latency are major constraints—minimizing expensive LLM calls benefits batch processing
  - The sequence of operations is complex but deterministic
  - Long chains of dependent actions have predictable outcomes
- **Avoid when**
  - The task is exploratory and requires dynamic adaptation
  - The environment changes significantly after the plan is created (basic implementations will fail)
  - Each step's outcome fundamentally changes what should happen next

# Comparison with ReAct



- The two patterns answer one question differently Does the agent need to know the full plan before it starts?"
- ReAct (Reasoning + Acting) is an adaptive pattern
  - The agent operates in a tight loop: think → take one action → observe the result → decide the next step
  - Every action is informed by what just happened
  - As Oracle's integration team puts it: "I don't know exactly where this leads — I'll figure it out as I go."
- Plan-and-Execute is a methodical pattern
  - A Planner agent decomposes the goal into an ordered list of steps, then an Executor agent runs those steps.
  - The Planner never calls tools—it only thinks. The Executor never improvises—it only acts on what the Planner prescribed
  - As Oracle explains: "I'll figure out the full plan first, then hand it off to be executed exactly as written."

## Use ReAct When

- **Path is unpredictable**
  - The sequence of steps is unknown until the previous result is seen
  - e.g., "find all blocked orders and resolve each one"
- **Step count is variable**
  - Tasks requiring 1-5 tool calls where each call informs the next
- **Error recovery needs creativity**
  - The agent must handle unexpected tool failures creatively—try alternative lookups, reframe queries
- **Low-stakes tasks**
  - Acceptable for conversational or low-stakes tasks where the execution trace is less critical
- **Latency matters**
  - Lower latency for simple tasks—no upfront Planner call overhead

## Use Plan and Execute when

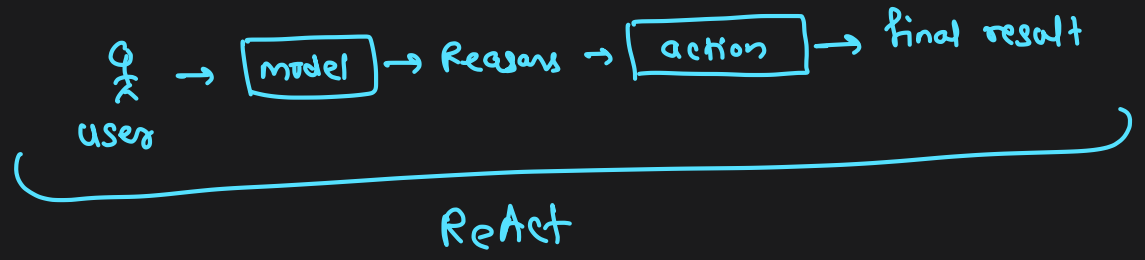


- **Steps are knowable upfront**
  - The steps required are knowable from the goal alone—
  - e.g., "generate a monthly report by pulling data from three systems"
- **Full tool sequence is predictable**
  - Tasks with a known set of sequential steps where the sequence is predictable from the goal description
- **Failures follow predictable logic**
  - Failures follow predictable retry or fallback logic that can be encoded in the plan step definition
- **Auditability is required**
  - Required for regulated, high-stakes, or compliance-sensitive workflows where every step must be logged and inspectable
- **Determinism matters**
  - The same task must execute the same way every time—deterministic, repeatable, and comparable across runs

# Reflection

# Introduction

- Reflection is a design pattern where an agent **critiques** and **improves its own output** through iterative cycles before delivering a final answer
- Instead of accepting the first response, the agent generates, evaluates, and refines — mimicking how humans review their own work
- The name comes from the idea of "reflecting on your work" — the same way a writer drafts, reviews, edits, and re-reviews before publishing
- Reflection is a feedback loop in which an agent (or set of agents) produces an output, critiques that output against a goal or rubric, and uses the critique to produce an improved output — repeating until a stopping condition is met



# Why does it exist?



- LLMs generate output **autoregressively and greedily** - each token is chosen based on prior tokens, with no internal checkpoint for correctness
- This causes predictable failure modes
  - **Hallucination:** Confident but false facts
  - **Edge-case blindness:** Misses null, empty, or boundary inputs
  - **Requirement drift:** Solves a related but wrong problem
  - **Reasoning errors:** Math, logic, or causal mistakes
  - **Style/format violations:** Wrong schema, tone, or structure
  - **Incomplete work:** Stops before finishing all steps
- A single pass has **no mechanism** to catch these. Reflection introduces one: a second look.
- LLMs are better at evaluating than generating
- Given a candidate answer, a model can often spot what's wrong even if it couldn't produce the right answer cold
- Reflection exploits this asymmetry

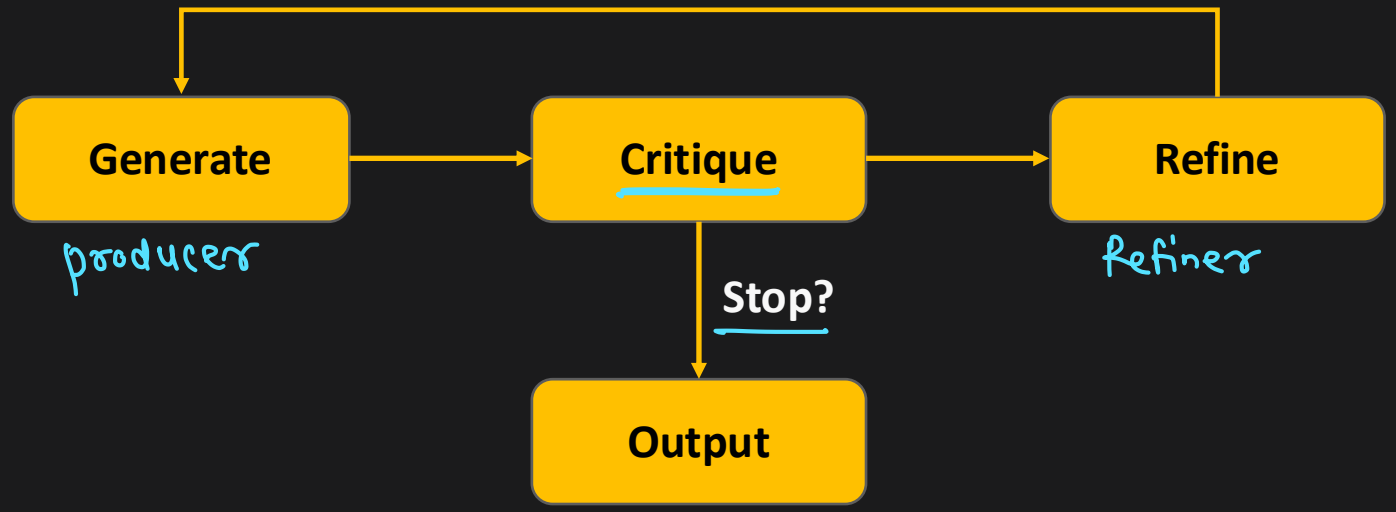
# Key Components



- Producer
  - Generates the initial output
- Critic
  - Evaluates output against criteria; returns structured feedback
- Refiner
  - Rewrites output using critic feedback
- Stopping criteria
  - Max iterations + approval threshold
- Feedback schema
  - Structured output (severity, missing, suggestions)
- Ground truth
  - Optional tool/test results fed to the critic

# The Fundamental Loop

- At its core, reflection is a **feedback control loop**




- **Three primitives**
  - **Generate** — produce a candidate output
  - **Critique** — evaluate it against criteria
  - **Refine** — produce an improved candidate
- **One decision point**
  - **Stop** — decide whether to accept, continue, or bail out

# The Four Axes of Variation

- **Axis 1: Who critiques?**
  - **Self** — same agent, same prompt
  - **Self with role-switch** — same agent, different persona ("now act as a reviewer")
  - **Separate agent** — dedicated critic, ideally different model
  - **External tool** — tests, linters, validators
- **Axis 2: How is the critique shaped?**
  - **Free-form prose** — "here's what I think"
  - **Structured** — JSON/Pydantic with fields like severity, missing, suggestions
  - **Scored** — numeric rating against a rubric
  - **Binary** — pass/fail
  - **Grounded** — tied to external evidence (test output)
- **Axis 3: How is refinement scoped?**
  - **Full regeneration** — rewrite from scratch
  - **Surgical revision** — change only what the critique flagged
  - **Additive** — append missing pieces without touching existing ones
- **Axis 4: When does it stop?**
  - **Approval flag** — critic says "good enough"
  - **Severity threshold** — no issues above minor
  - **Max iterations** — hard cap
  - **Budget exhausted** — tokens, time, or cost

# Architectural Variations

# Self-Reflection (Single Agent)



- **Mechanism**
  - One agent, one conversation. The prompt instructs the model to critique its own output, then rewrite
- **Pros**
  - Simplest to implement
  - Cheapest (one agent, no handoff)
  - Preserves full context automatically
- **Cons**
  - **Shared blind spots** — if the model didn't know it was wrong, it won't know it's wrong on review
  - **Self-approval bias** — models tend to rate their own output favorably
  - **Weak signal** — critique is often vague ("looks good, maybe check edge cases")
- **Best for**
  - Low-stakes tasks, drafting, brainstorming, cost-sensitive applications.

# Separate Critic Agent (Two Agents)



- **Mechanism**
  - A dedicated critic agent (often a different model or strongly different prompt) evaluates the producer's output
  - The producer revises based on critique
- **Pros**
  - **Breaks shared bias** — critic isn't anchored to producer's reasoning
  - **Stronger critique** — critic is prompted to be adversarial
  - **Clear separation** — one generates, one evaluates
- **Cons**
  - Higher cost (2× model calls per iteration)
  - Critic needs full context (task + output) to evaluate well
  - Handoff contract must be well-defined
- **Best for**
  - Code review, fact-checking, quality-critical content
- **Key design choice**
  - Use a **different model** for the critic when possible. Same-model critics still share training biases

# Producer–Critic–Refiner (Three Agents)

- **Mechanism**
  - Splits evaluation from revision
  - Producer makes the first draft; Critic evaluates; Refiner applies feedback. Refiner and Critic alternate
- **Pros**
  - **Cleanest responsibilities** — each agent has one job
  - **Refiner can be scoped tightly** — "only fix what the critique lists"
  - **Reduces oscillation** — refiner doesn't re-litigate the whole design
- **Cons**
  - Most expensive (3 agents)
  - More orchestration complexity
  - Diminishing returns vs. Variation B for many tasks
- **Best for**
  - High-stakes generation, long-form content, complex code changes.

# When to Use It



- **Use reflection when**
  - Output quality matters more than latency
  - The task has a **verifiable or rubric-able** success criterion
  - Errors are **costly** (production code, legal, medical, financial)
  - The model can **evaluate better than it can generate** (common for code)
  - You have **budget** for 2–3× inference cost
- **Don't use reflection when**
  - Task is simple lookup or classification
  - Latency is critical (real-time chat, autocomplete)
  - No clear quality signal exists
  - Output is cheap to fix downstream (drafts, brainstorms)
  - You can't afford the token cost

# Self Consistency

# Agentic AI

- Self-consistency is a decoding/reasoning strategy where, instead of generating a single reasoning path to an answer, the model generates **multiple independent reasoning paths** for the same problem and then selects the final answer via **majority vote (or aggregation)** across those paths
- It was introduced as an improvement over standard Chain-of-Thought (CoT) prompting
- The core insight: complex reasoning problems often admit multiple valid lines of reasoning that converge on the same correct answer, while incorrect reasoning tends to scatter across different wrong answers
- **Core Idea (Origin)**
  - Comes from the paper **"Self-Consistency Improves Chain of Thought Reasoning in Language Models"** (Wang et al., Google Research, 2022)
  - Standard CoT: greedy decoding - one reasoning chain - one answer
  - If that one chain has an error, the final answer is wrong
  - Self-consistency: sample **k** diverse reasoning chains (via temperature/top-p sampling, not greedy decoding) - extract the final answer from each - take the answer that appears most frequently

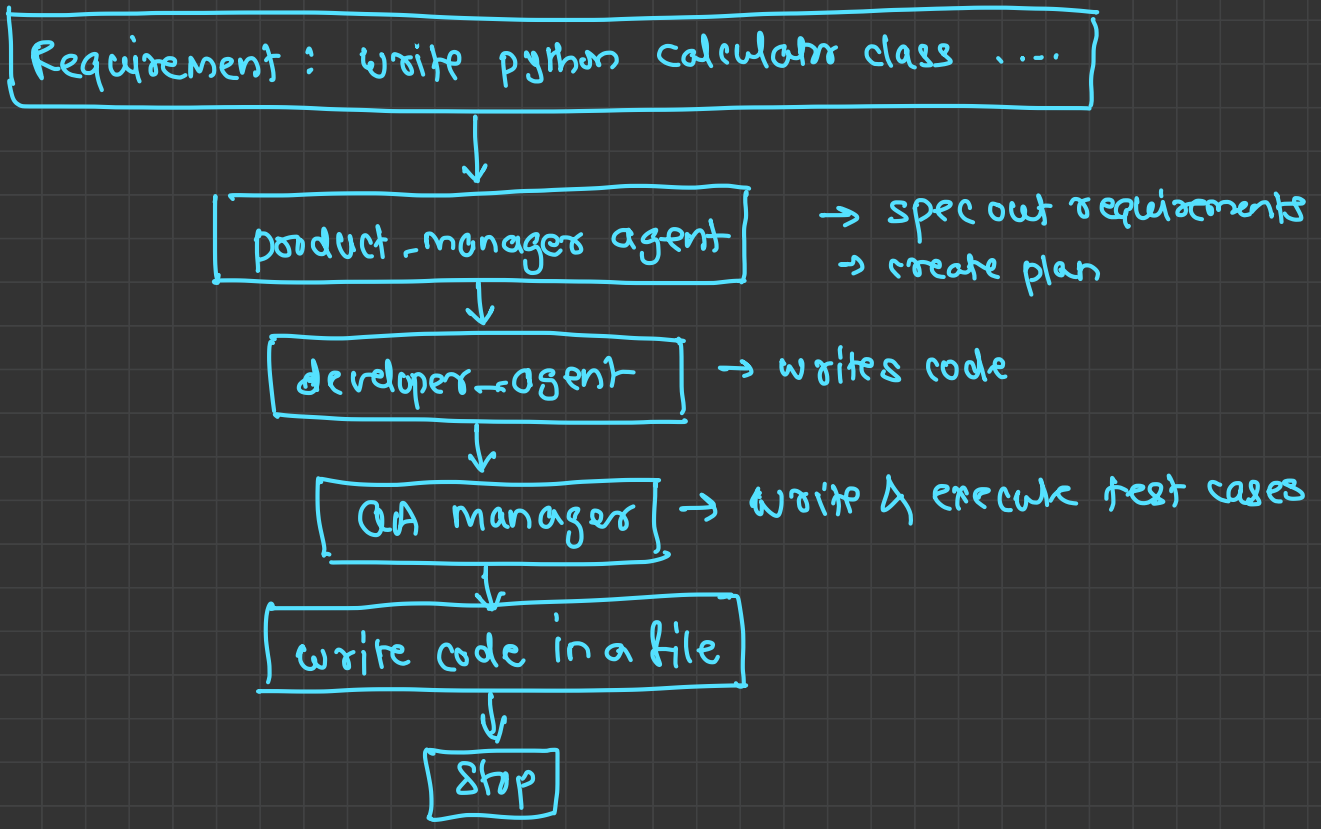
# How it works?

- **Prompt construction**
  - Use a CoT-style prompt (few-shot exemplars showing step-by-step reasoning, or a zero-shot "think step by step" instruction)
- **Diverse sampling**
  - Instead of decoding once with temperature=0, sample the model N times (e.g., N = 5, 10, 20, 40) with temperature > 0 to get varied reasoning paths
- **Independent reasoning**
  - Each sample independently reasons through the problem in its own way — different decomposition, different intermediate steps, different order of operations
- **Answer extraction**
  - Parse out the final answer from each of the N generated chains
- **Aggregation / voting**
  - Take the majority answer (mode) across the N samples as the final output
  - Ties can be broken by confidence scores, weighted voting, or re-prompting.

- Question ⟶
  - [Sample 1: reasoning path A ⟶ answer Y]
  - [Sample 2: reasoning path B ⟶ answer X]
  - [Sample 3: reasoning path C ⟶ answer Y]
  - [Sample 4: reasoning path D ⟶ answer X]
  - [Sample 5: reasoning path E ⟶ answer X]
- Majority vote ⟶ Answer X

# Why It Works

- **Error decorrelation**
  - Wrong reasoning tends to fail in different ways each time (different mistaken assumptions, arithmetic slips, missed steps), so wrong answers rarely cluster
- **Correct reasoning converges**
  - Multiple valid reasoning strategies for the same problem tend to arrive at the same correct answer, so correct answers cluster
- It's essentially an ensembling technique at the reasoning-path level, analogous to bagging/ensemble methods in classical ML, but applied to generated reasoning traces instead of trained models
- It doesn't require any additional training, fine-tuning, or auxiliary verifier model — it's purely an inference-time strategy


Requirement : write python calculator class ....
product_manager agent
→ spec out requirements
→ create plan
developer_agent
→ writes code
QA manager
→ write & execute test cases
write code in a file
Stop

# Where It Fits in Agentic AI Design Patterns



- In the broader landscape of agentic patterns, self-consistency sits in the **reasoning/reflection family** alongside:
  - Chain-of-Thought: Single linear reasoning trace
  - Self-Consistency: Multiple sampled reasoning traces + majority vote
  - Tree-of-Thought: Explicit branching + search/backtracking over reasoning states
  - ReAct: Interleaved reasoning + tool-use actions
  - Reflection / Self-Refine: Iterative self-critique and revision of one trace
  - Debate: Multiple agent personas argue and cross-examine
- Self-consistency is distinct from Tree-of-Thought in that it does **not** require explicit search, backtracking, or evaluation of intermediate states
- The paths run to completion independently and are only compared at the final-answer stage
- It's cheaper and simpler to implement than ToT, but less powerful for problems needing mid-reasoning correction

# Where It's Most Effective



- Arithmetic reasoning (word problems)
- Commonsense reasoning
- Symbolic reasoning tasks with a discrete, verifiable answer space
- Tasks where the answer space is small and discrete (multiple choice, numeric, short factual) — voting is straightforward
- Less effective / harder to apply for open-ended generation (long-form writing, code generation with many valid solutions) because "majority vote" over free text is ill-defined — though variants exist (e.g., clustering similar outputs, or using execution-based voting for code, like running test cases)

# Cost / Latency Trade-offs



- N× the inference cost and latency of a single CoT call (you're running N samples instead of 1)
- Diminishing returns: accuracy gains typically saturate around N = 10–40 samples depending on task difficulty
- Common mitigation in agentic systems
  - Run samples in parallel (not sequential) to bound latency
  - Use a cheaper/smaller model for the N sampled paths and a larger model only for final verification/tie-breaking
  - Adaptive sampling: start with a small N, only sample more if there's no clear majority (early-stopping / dynamic self-consistency)

# Variants and Extensions

- **Universal Self-Consistency**
  - uses the LLM itself (rather than exact-match voting) to select the best answer among the N generations, useful when answers are free-form text rather than short discrete outputs
- **Self-Consistency + ToT**
  - sample multiple full trees / search rollouts, then vote
- **Self-Consistency with tools**
  - each sampled path can include different tool-call sequences; vote over final grounded answers rather than raw reasoning
- **Confidence-weighted self-consistency**
  - incorporate the model's calibrated confidence per path to weight votes rather than treat all paths equally

# Key Limitations



- Assumes the correct answer is more "reachable" via diverse reasoning than incorrect ones — this can fail on adversarial or genuinely ambiguous problems
- Not useful when there's no well-defined ground truth answer to vote on (e.g., creative writing, subjective judgments)
- Compute-hungry — a real constraint in production agentic systems with cost/latency budgets
- Doesn't fix systematic bias — if the model has a consistent blind spot or misconception, all N samples may reproduce the same wrong answer (voting won't help against correlated errors)

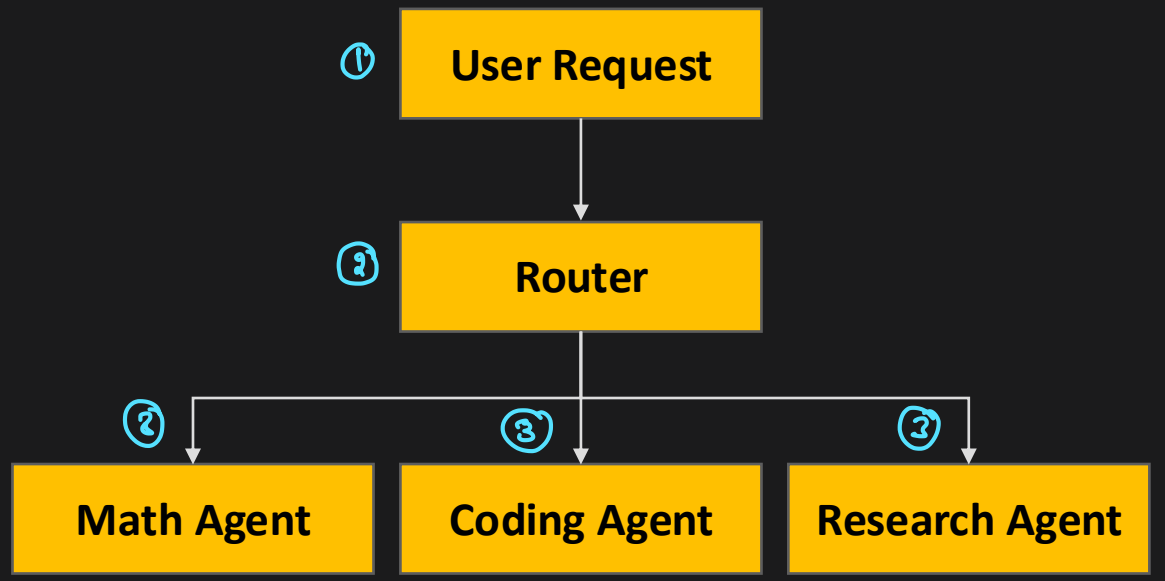
# Self Consistency vs Reflection

| Dimension | Self-Consistency | Reflection |
|---|---|---|
| **Primary goal** | Reduce variance / improve accuracy | Improve quality / fix errors |
| **Mechanism** | Sample N paths → vote | Generate → critique → revise |
| **Number of passes** | Multiple *parallel* samples | Multiple *sequential* iterations |
| **Direction** | Horizontal (breadth) | Vertical (depth) |
| **Key operation** | Aggregation (majority vote) | Critique + revision |
| **Changes the answer?** | Selects best from candidates | Transforms the answer |
| **Needs a critic?** | No | Yes (self or external) |
| **Needs sampling?** | Yes (temperature > 0) | Not necessarily |
| **Compute cost** | K× per query (parallel) | N× per query (sequential) |
| **Latency** | Parallelizable | Inherently sequential |
| **Determinism** | Reduces non-determinism | Can increase it (more rounds = more drift) |
| **Best for** | Discrete answers (math, logic, classification) | Open-ended outputs (code, writing, plans) |
| **Weakness** | Can't fix a systematically wrong model | Can loop, over-critique, or drift |

# Routing

# Introduction

- The **Routing Design Pattern** is an Agentic AI pattern in which an incoming task is first classified or analyzed by a router, and then the task is dynamically sent to the **most appropriate** agent, model, tool, or workflow
- Instead of using one agent to handle every type of request, routing allows a system to have **specialized agents** and intelligently decide which one should handle each request
- In a traditional application, routing usually means
  - URL → Controller → Service
- In Agentic AI, routing is more dynamic
- The decision can be based on:
  - User intent
  - Task type
  - Domain
  - Complexity
  - Required tools
  - User permissions



# Why Do We Need Routing

- **The "One Model for Everything" Problem**
  - Without routing, a single do-everything prompt forces one model on every input
  - This creates a permanent tradeoff
    - Use an expensive frontier model for everything → you overpay on trivial queries (e.g., "capital of France?")
    - Use a cheap small model for everything → you underperform on hard tasks (e.g., complex multi-step proofs)
  - Routing **breaks this bind** by separating "what kind of question is this?" from "answer it," allowing each component to be optimized independently—including the choice of model, prompt, and tools
- **The Heterogeneity Assumption**
  - Extensive research validates that **no single agent or backbone uniformly dominates** across tasks
  - Performance rankings vary substantially across benchmarks and backbones, even under identical prompt templates
  - The best agent on one benchmark is often suboptimal on another
  - This heterogeneity is the empirical justification for routing: a principled, task-aware mechanism outperforms a one-size-fits-all approach

# Router vs Agent

- A common misconception is: The router itself is an agent
- A router can be implemented using
  - Rule-based logic
  - Machine learning classifier
  - LLM-based routing
  - Embedding similarity
  - Hybrid routing

# LLM-Based Routing



- One of the most common implementations in modern Agentic AI is an LLM acting as the router
- For example:
  - **User request**
    - "Analyze Apple's latest earnings and explain how they might affect the stock."
- The router might produce:

```
{
  "route": "financial_research",
  "confidence": 0.94
}
```

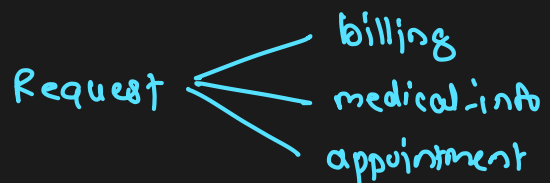
- The system then invokes: Financial Research Agent
- instead of the general-purpose agent

# Routing Strategies



- Routing Based on Intent → LLM as a Router
- Routing Based on Domain
- Routing Based on Task Complexity
- Routing Based on Cost
- Routing Based on Latency
- Routing Based on Available Tools

# Common Routing Topologies




- **Flat (Single-Level) Routing**
  - One router classifies into N categories, each with a dedicated handler
  - Simplest and most common
  - Works well when the number of categories is manageable (roughly 3 – 10)
  - **Example:** Customer service queries split into refunds, technical support, and general questions
- **Hierarchical Routing**
  - A **router of routers**: a top-level triage agent picks a domain, then a domain-level router picks the specialist
  - This tames large systems where a flat router would face too many destinations at once
  - Cost: more hops and more places a decision can go wrong
- **Auction-Based Routing**
  - Instead of a central router pushing work to an agent, candidate agents bid on a request, and the best-fit bid wins
  - Suits dynamic agent pools where capabilities change frequently
  - Bidding adds coordination overhead, rarely worth it for small, fixed teams
- **Context-Aware Routing (Framework-Specific)**
  - The router reads the request, classifies it, and returns a Handoff(target=specialist) directly from the classify tool
  - The framework resolves the routing target from the tool result, bypassing transition graph conditions entirely
  - This avoids a known stall issue where ContextEquals conditions only evaluate on substantive packets

# When to Use Routing

- **Strong Fit**
  - **Inputs come in distinguishable types** that warrant materially different handling
  - **Classification can be handled accurately** by an LLM or traditional algorithm
  - **Complex tasks with distinct categories** are better handled separately
  - **Cost/quality optimization** matters—you want to spend expensive model calls only where they help
  - **Specialization is needed**—optimizing one input type shouldn't hurt performance on others
- **Weak Fit / Anti-Patterns**
  - **Truly uniform inputs** with no meaningful categories—routing adds overhead with no benefit
  - **Categories are not reliably distinguishable**—if the classifier can't pick accurately, routing errors compound
  - **Very small number of simple paths**—a simple conditional may suffice without a full routing layer

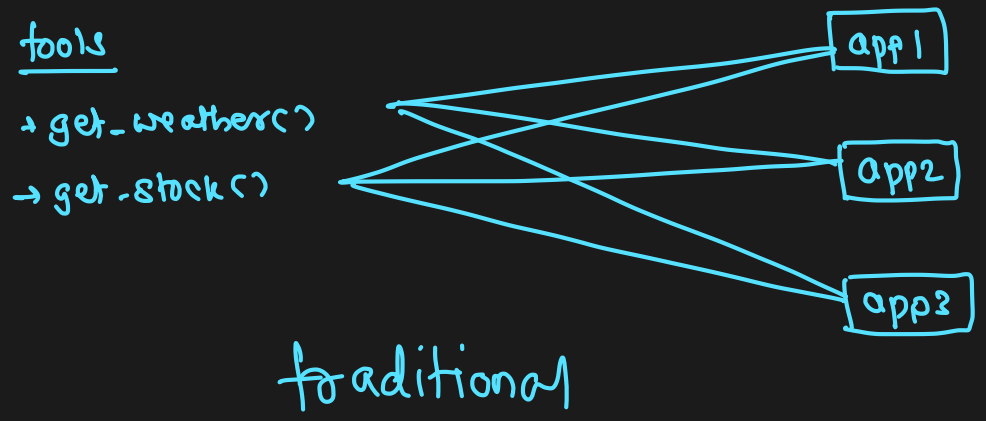
# MCP

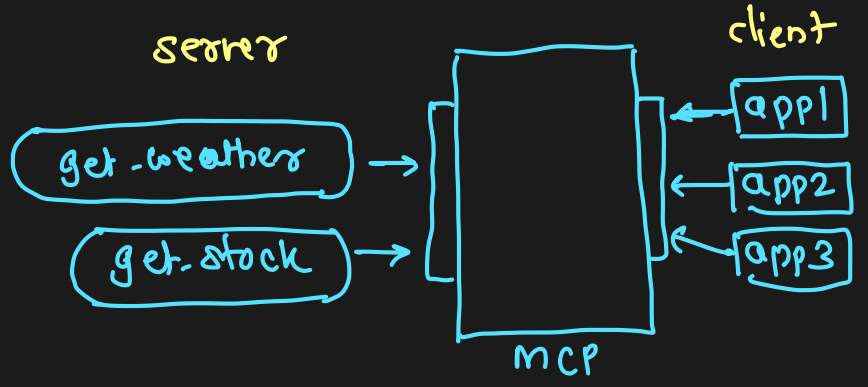
model context protocol

# MCP

prompts & resources

- MCP is an open standard that enables AI applications to connect seamlessly with external data sources and tools
- It is often described as a **"USB-C port for AI applications"**—a universal connector that eliminates the need for custom integrations for every tool or data source
- Introduced by Anthropic in November 2024
- In December 2025, Anthropic donated MCP to the **Linux Foundation**, with co-founders including Anthropic, Block, and OpenAI
- Platinum members include AWS, Google, Microsoft, and Cloudflar

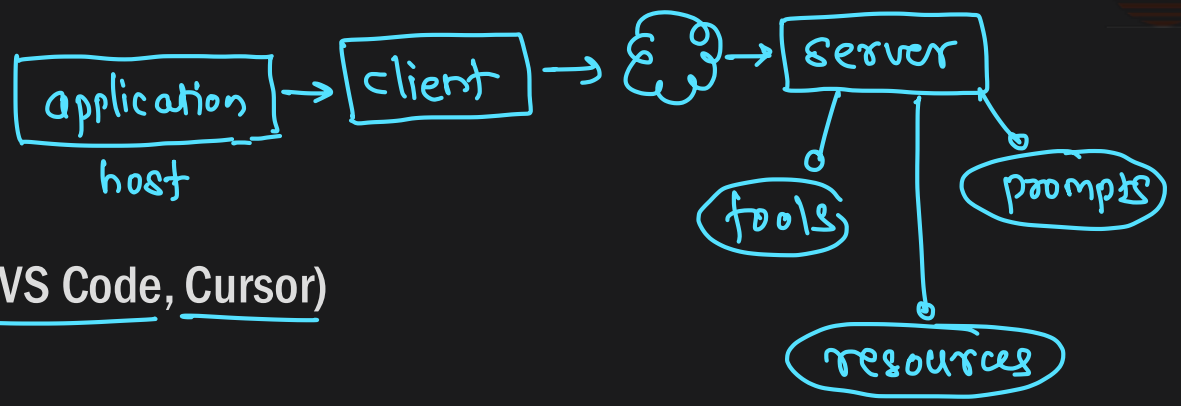


# The "N x M" Problem

- Before MCP, connecting **N** AI models to **M** external tools required **N×M** custom integrations
- MCP reduces this to an **N+M** model
  - build one MCP server, and any compatible AI application can use it
- **Benefits**
  - Developers: Reduces development time; "integrate once" instead of rebuilding for every model
  - AI Applications: Access to an ecosystem of tools and data sources without custom code
  - End Users: More capable AI that can access personal data and take actions

# Architecture

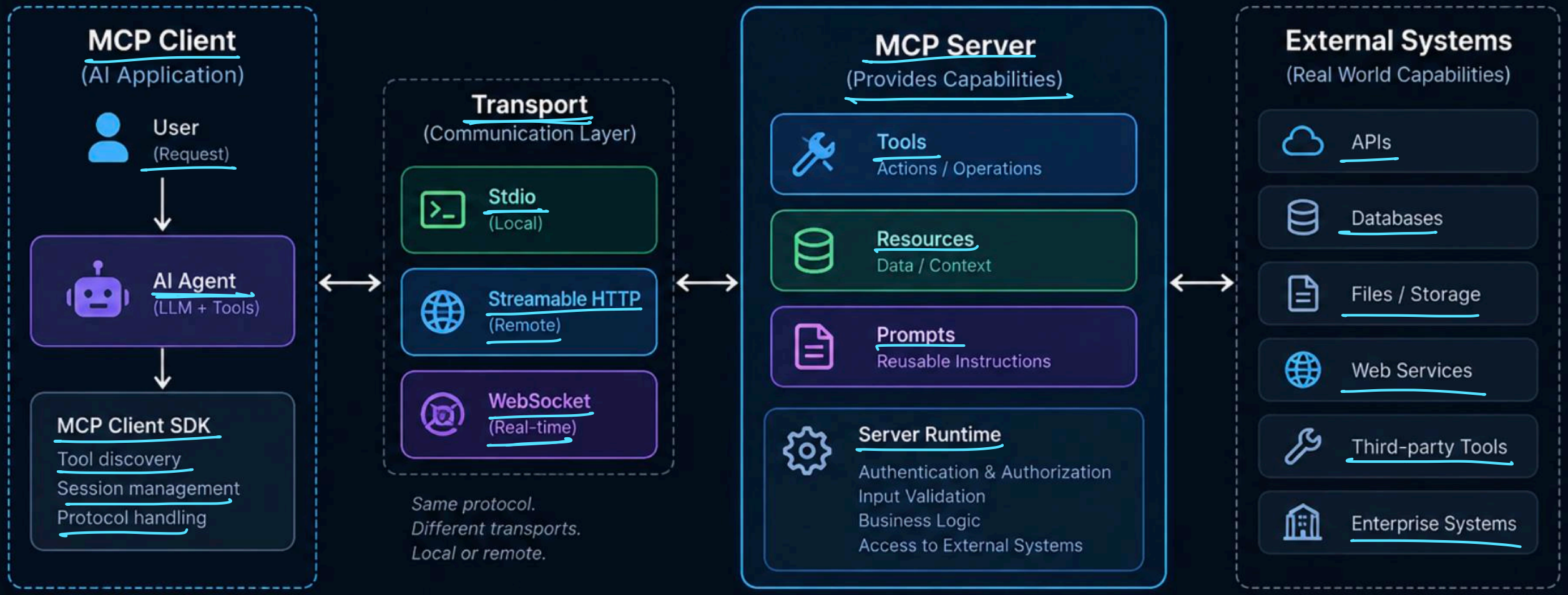



- MCP follows a client-host-server architecture
- **Host**
  - The **host** is the AI application itself (e.g., Claude Desktop, VS Code, Cursor)
  - It manages the overall session
  - Creates and manages multiple client instances
  - Enforces security policies and consent requirements
  - Coordinates AI/LLM integration and sampling
  - **Critical Security Boundary**
    - The host ensures servers cannot read the full conversation or "see into" other servers
- **Client**
  - A **client** is a connector inside the host
  - Each client maintains a **1:1 relationship** with exactly one server
  - Attaches protocol version and capabilities to every request
  - Routes messages bidirectionally
  - Maintains isolation between servers

# Architecture

- **Server**
  - A **server** provides the actual capabilities (tools, data, prompts)
  - Servers are designed to be **extremely easy to build** and **highly composable**
  - Can be local processes (STDIO) or remote services (HTTP)
  - Focused on specific, well-defined responsibilities
  - Servers can offer three main types of features to clients
    - Resources
      - Context and data for the user or AI model to read
      - E.g. A database record, a file, or a document
    - Prompts
      - Templated messages and workflows
      - E.g. A pre-defined "summarize this article" workflow
    - Tools
      - Functions for the AI model to execute
      - E.g. get_weather, send_email, query_database

# Architecture



# Protocol Mechanics

- **Base Protocol: JSON-RPC 2.0**
  - All MCP messages use **JSON-RPC 2.0** formatting
  - Key message types
    - **Requests:** Include a unique ID; expect a response.
    - **Responses:** Successful results or errors.
    - **Notifications:** One-way messages with no ID and no expected response .
- **Stateful vs. Stateless Evolution**
  - **Early Spec (2025-03-26):** Stateful connections with persistent sessions
  - **Current Spec (2026-07-28):**
    - **Stateless, self-contained requests**
    - Every request carries its own protocol version and capabilities in the _meta field
  - **Why it matters:** Statelessness enables trivial horizontal scaling, load balancing, and failover for enterprise deployment
- **Capability Negotiation**
  - Clients and servers declare supported features during initialization
  - **Servers declare:** Tool support, resource subscriptions, prompt templates
  - **Clients declare:** Sampling support, elicitation handling, roots
  - **Extensions:** Optional features (e.g., Tasks for async operations, MCP Apps for interactive UI) are opt-in and negotiated

# Security Framework



- **User Consent and Control**
  - Users must explicitly consent to and understand all data access and operations
- **Data Privacy**
  - Hosts must obtain consent before exposing user data to servers
- **Tool Safety**
  - Tools represent arbitrary code execution
  - Tool descriptions should be considered **untrusted** unless from a trusted server
- **Sampling Controls**
  - Users must approve LLM sampling requests, including the prompt and what results the server can see

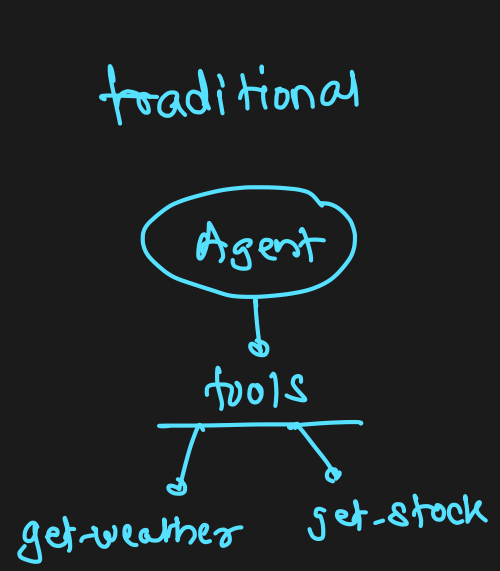
# Benfits

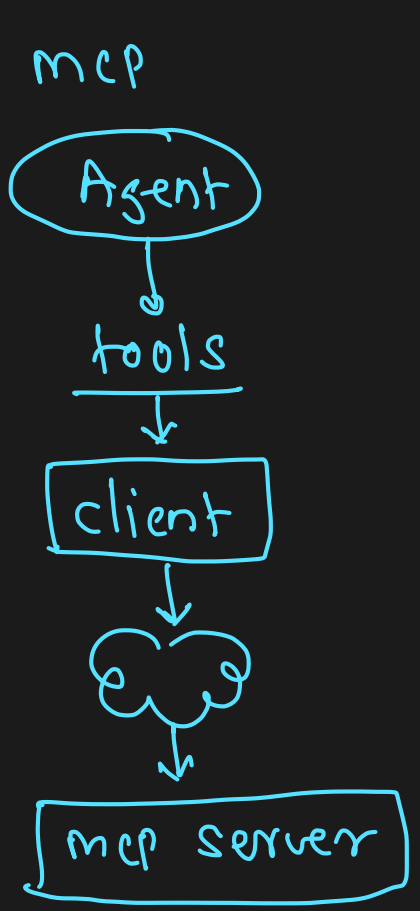


- **Standardized integration**
  - The biggest advantage is that MCP provides a **standard protocol** for connecting AI applications with external capabilities
- **Reusable integrations**
  - An MCP server can potentially be used by multiple MCP-compatible applications
- **Tool discovery**
  - MCP allows clients to discover capabilities dynamically
- **Separation of concerns**
  - MCP creates a clean architectural boundary
- **Supports different capabilities**
  - MCP isn't limited to tools, it can also be used to share dynamic prompts as well as resources
- **Framework independence**
  - An MCP server doesn't inherently need to be built with LangChain
- **Local and remote deployment**
  - MCP supports different transport mechanisms

# Disadvantages

- Additional architectural complexity
- Another protocol to understand
- Additional latency
- Security becomes important
- Tool overload
- Debugging can become harder
- Not every application needs MCP
- **Version compatibility can be a challenge**



# When to use and when not to use



- When to use MCP
  - Company-wide AI platform
  - Shared database access
  - Shared GitHub integration
  - Shared enterprise systems
  - Multiple AI agents
  - Multiple AI clients
  - Remote AI services
  - Reusable business capabilities
- When not to use MCP
  - Simple chatbot
  - One application
  - One or two internal functions
  - No external integrations
  - No need to reuse the integration

# Implementation

# Server

```python
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("Demo MCP Server")


@mcp.tool()
def add_numbers(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@mcp.resource(
    "knowledge://python",
    mime_type="text/plain"
)
def python_knowledge() -> str:
    """Basic information about Python."""

    return """
Python is a high-level programming language.

Python is commonly used for:
- Artificial Intelligence
- Machine Learning
- Data Science
"""

@mcp.prompt()
def explain_topic(topic: str) -> str:
    """Generate a prompt for explaining a technical topic."""

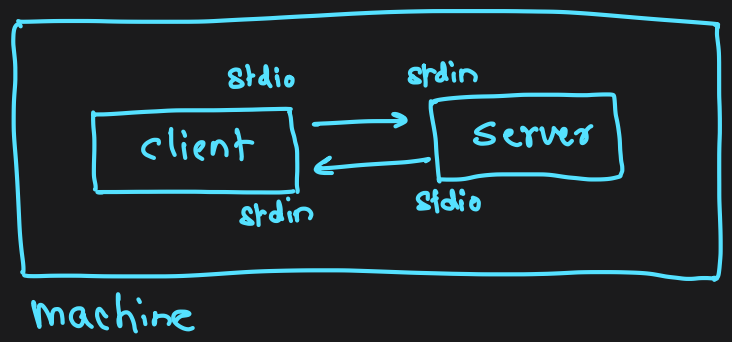
    return f"""
Explain {topic} in simple terms.

Include:
1. Definition
2. How it works
"""


if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8000)
```

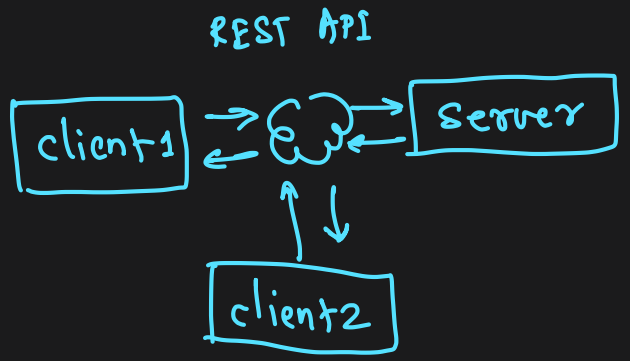
# Transport

- In MCP, a **transport** defines **how the MCP client and MCP server communicate**
- The MCP protocol defines the messages exchanged; the **transport determines how those messages physically move between client and server**
- **There are 3 options: stdio, streamable http and SSE**
- **Stdio** → (Inter Process Communication)
  - This transport tunnels MCP messages over the standard input and output streams of a subprocess
  - The client launches the server as a local subprocess. Messages are exchanged via the server's stdin and stdout
  - **When to use it**
    - **Client and server are on the same machine**
    - **You are developing/testing an MCP server**
    - **An AI desktop application launches local MCP servers**
    - **You don't need a network server**



# Transport: Streamable HTTP

- This is the recommended transport for remote, networked deployments
- It exposes the MCP server over a single HTTP endpoint (like http://localhost:8000/mcp) that supports both POST and GET requests
- The server runs as an independent process
- Clients send requests via HTTP POST
- The server can respond either with a single JSON object or by opening a Server-Sent Events (SSE) stream for streaming multiple messages back
- **When to use it**
  - MCP server is remote
  - Multiple clients need to access the server
  - Server is deployed in cloud
  - You need network-based access
  - You're building a production MCP service



# Transport: SSE



- You might see references to an sse transport in older tutorials
- This was the **original HTTP transport** but has since been **deprecated**
- It was replaced by Streamable HTTP in a 2025 protocol revision
- It exists only for backward compatibility with clients that haven't migrated yet
- **Recommendation**
  - **Do not use it** for any new projects. Always choose streamable-http for web-based deployments

# Client



```python
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

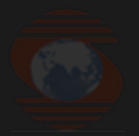
            tools = await session.list_tools()
            for tool in tools.tools:
                print(f"{tool.name}: {tool.description}")

            result = await session.call_tool(
                "add_numbers",
                {"a": 10, "b": 20}
            )
            print(result)


if __name__ == "__main__":
    asyncio.run(main())
```

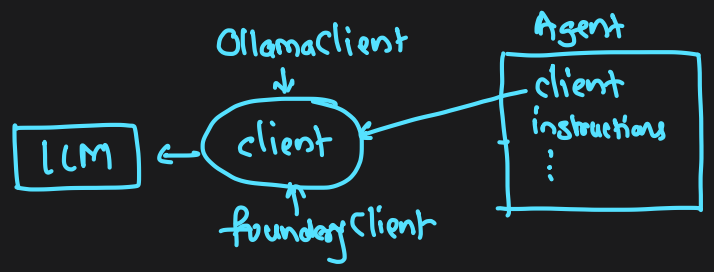
# Microsoft Agent Framework

# Introduction



- **Microsoft Agent Framework** is an open-source SDK and runtime for building AI agents and multi-agent workflows in .NET and Python
- It is the **direct successor** to Semantic Kernel and AutoGen, created by the same engineering teams
- **Why Agent Framework Exists**
  - Semantic Kernel
    - Enterprise connectors, type safety, telemetry, filters
    - Complex multi-agent abstractions
  - AutoGen
    - Simple agent abstractions, research-friendly patterns
    - Lacked enterprise state management and observability
- Agent Framework eliminates this trade-off
- It provides one framework spanning single-agent scenarios to complex multi-agent orchestration, with production-grade features built in from the start

# Core Architecture: Four Primary Areas




- **Agents**
  - Individual agents that use LLMs to process inputs, call tools and MCP servers, and generate responses
  - Supported model providers include **Microsoft Foundry, Anthropic, Azure OpenAI, OpenAI, Amazon Bedrock, Google Gemini, Ollama**, and any OpenAI-compatible endpoint
  - **Key architectural point**: All agents derive from a unified base class (Agent in Python, AIAgent in .NET), providing a consistent interface regardless of the underlying model provider
- **Harness Agent** ↳ Extensible
  - An opinionated agent with "batteries-included" capabilities for long, multi-step tasks
  - It wraps the model in a runtime that handles planning, todo tracking, context compaction, file access, memory, tool approval, and observability
- **Workflows**
  - Functional and graph-based workflows that connect agents and functions through **explicit execution paths**
  - Supports sequential, concurrent, handoff, group chat, and Magentic-One orchestration patterns
- **Integrations**
  - Connections to model providers, agent services, tools, context providers, middleware, evaluation services, and UI frameworks, organized by provider and component

# Foundational Building Blocks

- **Model clients:** Unified interface for chat completions and responses across providers
  - → Ollama | AWS Bedrock | Anthropic
- **Agent session:** Native state management for persistent multi-turn conversations → short term memory
- **Context providers:** Memory components that surface relevant information dynamically → long term mem
  - ↳ sqlite | json
- **Middleware:** Hooks to intercept, log, or modify agent actions before and after execution
- **MCP clients:** Built-in Model Context Protocol support for dynamic tool discovery at runtime
- **Function tools:** Custom functions automatically registered with schema generation handled by the framework
  - ↳ tool calling

# Basic Agent Examples



```python
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(
        project_endpoint="https://your-foundry-service.services.ai.azure.com/api/projects/your-foundry-project",
        model="gpt-5.4-mini"
    ),
    name="HelloAgent",
    instructions="You are a friendly assistant. Keep your answers brief.",
)
result = await agent.run("What is the largest city in France?")
```

# Key Capabilities and Features



- **Multi-Turn Conversations:** Agents maintain chat history either locally or via service-managed history (Foundry, Azure OpenAI Responses, OpenAI Responses)
- **Structured Outputs:** Generate responses with type safety and schema validation
- **Streaming Responses:** Receive results incrementally as they are generated
- **Middleware:** Intercept agent actions before and after execution for logging, modification, or observability
- **MCP Support:** Built-in MCP client support enables dynamic tool discovery and integration at runtime
- **A2A Protocol:** Cross-runtime interoperability with other agent frameworks via the A2A protocol
- **Durable Workflows:** Support for long-running, human-in-the-loop scenarios with robust state management

# Agent Framework vs. Foundry Agent Service

| Consideration | Microsoft Agent Framework | Foundry Agent Service |
|---|---|---|
| **Performance** | Fast (runs locally) | Slower (managed, remote service) |
| **Development** | Full code, maximum control | Low-code, rapid integration |
| **Testing** | Manual/unit tests in code | Built-in playground for quick testing |
| **Scalability** | App-managed | Azure-managed, auto-scaling |
| **Safety Guardrails** | Custom implementation required | Built-in safety and content moderation |
| **Identity** | Custom implementation required | Built-in agent ID and authentication |
| **Enterprise** | Custom integration required | Built-in Microsoft 365/Teams deployment |

# Handling Memory



- Agent Framework distinguishes between two memory horizons:
- Short-term memory
  - Conversation history and recent context within a session
  - AgentSession + ChatMessageStore
- Long-term memory
  - Facts, preferences, and knowledge extracted across sessions
  - ContextProvider (memory providers)

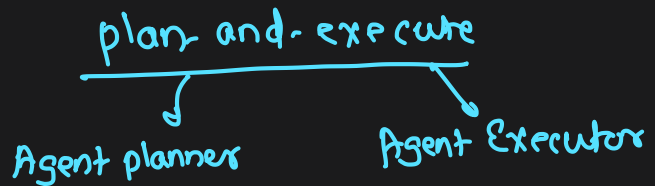
# Multi-Agent Collaboration

Agent orchestration

microservices architecture

# Introduction

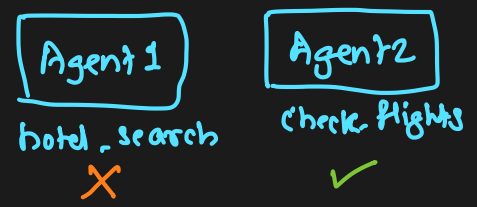
- Multi-Agent Collaboration is a design pattern in Agentic AI where **multiple autonomous, specialized agents work together** — through communication, coordination, and negotiation — to solve a problem that a single agent cannot handle effectively alone
- Each agent typically has:
  - A **role** (e.g., researcher, coder, critic, planner)
  - Its own **tools** and **memory**
  - A **goal** or sub-goal
  - The ability to **communicate** with other agents




- The core idea here is to **divide a complex task into sub-tasks, assign them to specialized agents, and orchestrate their interaction** to produce a coherent final result

# Why Use Multi-Agent Collaboration?

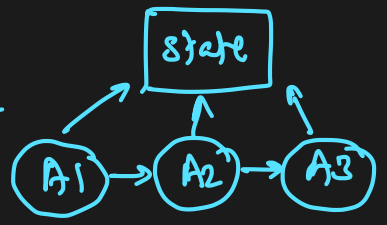
- **Specialization:** Each agent focuses on one domain (e.g., retrieval, coding, validation), improving quality
- **Modularity:** Easier to build, test, and replace individual agents
- **Scalability:** Add more agents to handle more workload or new capabilities
- **Robustness:** If one agent fails, others can compensate or retry
- **Parallelism:** Independent sub-tasks run concurrently, reducing latency
- **Emergent Intelligence:** Collective reasoning often outperforms a single LLM
- **Separation of Concerns:** Planning, execution, and critique are decoupled



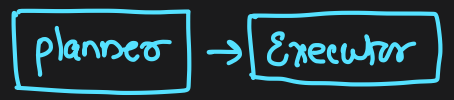
# Key Components

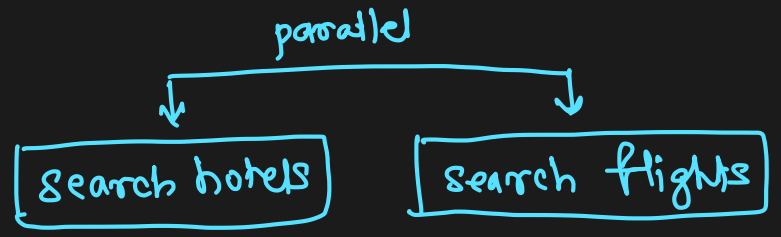
- **Agents** — Autonomous units with role, goal, tools, memory, and policy
- **Communication Channel** — How agents exchange messages (shared memory, message bus, chat)
- **Orchestrator / Coordinator** — Decides who acts when (static or dynamic)
- **Task Decomposer** — Splits the main goal into sub-tasks
- **Shared State / Blackboard** — Common workspace for intermediate results
- **Environment / Tools** — External APIs, databases, code executors
- **Evaluation / Critic** — Validates outputs and triggers refinement
- **Termination Condition** — When the task is considered complete

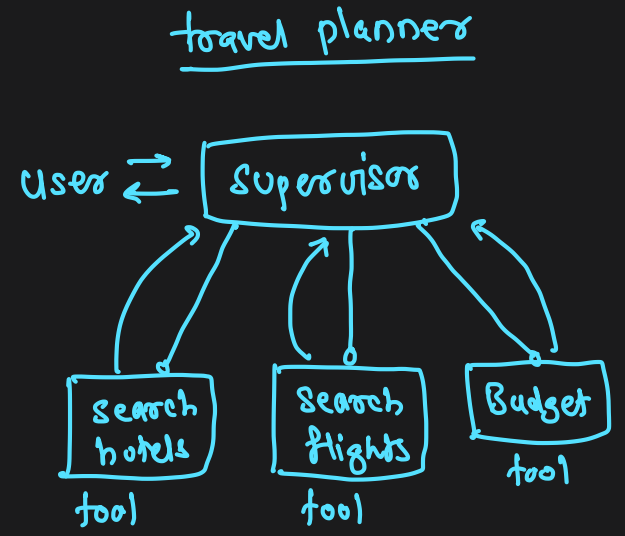
# Major Orchestration Architectures




- Sequential Orchestration
- Concurrent / Broadcast (Parallel) Orchestration
- Hierarchical Orchestration
- Handoff / Routing Pattern
- Group Chat / Debate Pattern
- Blackboard Pattern
- Market / Auction-Based
- Competitive / Adversarial
- Supervisor Pattern



# Communication Mechanisms



- **Message Passing:** Direct agent-to-agent messages (like actors)
- **Shared Memory / Blackboard:** Common store all agents read/write
- **Publish–Subscribe:** Agents subscribe to topics of interest
- **Structured Protocols:** JSON schemas, FIPA-ACL-like contracts
- **Natural Language Chat:** Free-form LLM dialogue (AutoGen, CrewAI)
- **Message content types:** requests, informs, proposals, critiques, confirmations, refusals

# Coordination and Control Strategies

- **Centralized:** One orchestrator decides everything
- **Decentralized:** Agents self-organize via rules or negotiation
- **Hybrid:** Central planner + autonomous workers
- **Contract Net Protocol:** Manager announces task → agents bid → best bidder wins
- **Voting / Consensus:** Majority or weighted vote for decisions
- **Role-based:** Fixed roles (Planner, Executor, Critic, Verifier)

# Design Considerations and Trade-offs

- **Cost:** More agents = more LLM calls = higher cost
- **Latency:** Parallelism helps; coordination adds overhead
- **Complexity:** More agents = harder debugging
- **Reliability:** Redundancy helps; cascading failures hurt
- **Autonomy vs. Control:** Free agents risk drift; tight control limits flexibility
- **Communication Overhead:** Too many messages slow the system
- **Convergence:** Debate/chat patterns may never terminate
- **State Consistency:** Shared memory needs synchronization

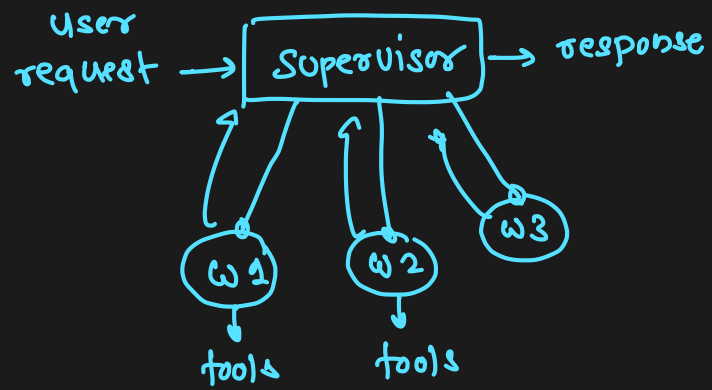
# Frameworks and Implementations

- **AutoGen :** Conversational multi-agent, group chat → MS agents framework
- **CrewAI:** Role-based crews with sequential/hierarchical processes
- **LangGraph:** Graph-based orchestration with state
- **MetaGPT:** SOP-driven software team agents
- **CAMEL:** Role-playing agent dialogue
- **Swarm (OpenAI):** Lightweight handoff-based agents
- **AgentVerse:** Simulation of collaborative agents
- **ChatDev:** Multi-agent software development

- Google ADK →
- OpenAI Agent framework

# Real-World Use Cases

- **Software development:** Planner + coder + tester + reviewer
- **Research:** Retriever + summarizer + critic
- **Customer support:** Triage → specialist → escalation
- **Content creation:** Researcher → writer → editor → fact-checker
- **Finance:** Analyst + risk + compliance agents
- **Robotics:** Swarm coordination
- **Game AI:** Adversarial and cooperative teams
- **Healthcare:** Diagnostic + specialist + safety agents




# Supervisor

# Introduction

→ group efforts

- The supervisor design pattern is an orchestration architecture for multi-agent systems
- It contains central coordinating agent—the supervisor—receives user requests, delegates subtasks to specialized worker agents, and aggregates their outputs into a final response
- Rather than having a single monolithic agent handle every aspect of a complex task, this pattern distributes work across focused specialists while maintaining centralized control over routing, sequencing, and result synthesis
- The supervisor does not perform specialist work itself when worker agents are available
- Its role is fundamentally about orchestration: understanding intent, breaking requests into subtasks, selecting the appropriate worker for each subtask, passing necessary context, waiting for results, and deciding whether additional delegation is needed before producing a final coordinated answer

# Architecture: Structural Components

- A supervised agent team consists of three core elements:
- **Supervisor Agent:** The orchestrator that receives the user request, makes routing decisions, and maintains the overall task context
- **Worker Agents:** Specialized agents with focused responsibilities—such as querying databases, performing calculations, retrieving documents, or executing transactions—each with their own configured tools
- **Delegation Mechanism:** A tool-based handoff system that allows the supervisor to transfer control to a worker and receive results back
- In implementation frameworks like LangGraph, the supervisor creates handoff tools automatically for each worker agent (e.g., handoff_to_math, handoff_to_research)
- These tools encapsulate the transfer of task context to the appropriate specialist

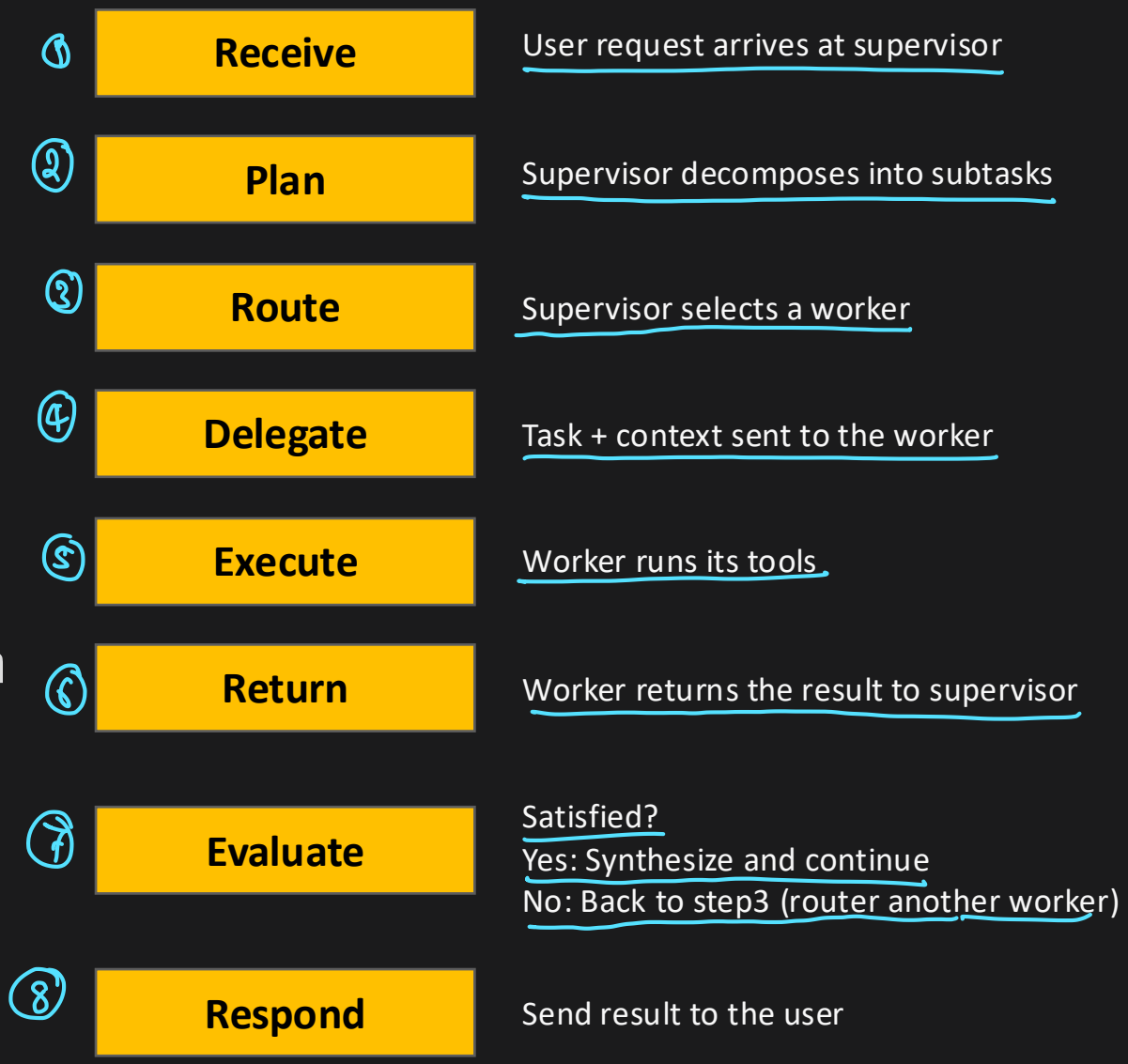
# Component Roles

- Supervisor (Manager) → brain
  - Understand the user's intent
  - Break the task into subtasks → routing
  - Decide which worker handles each subtask
  - Pass the right context to each worker
  - Evaluate whether results are sufficient
  - Re-route or retry when needed
  - Combine outputs into a final answer
  - Does NOT
    - Execute domain-specific work itself (worker agent)
    - Hold specialist tools
    - Override worker expertise

- Worker (Specialist)
  - Handle one narrow domain
  - Own a focused set of tools
  - Return clean, structured results
  - Does NOT
    - Route to other workers (unless explicitly allowed)
    - Synthesize final answers
    - Handle tasks outside their domain

# Sequential Task Processing

- The subtask process is sequential: a worker subtask must complete before the supervisor delegates the next one
- The supervisor waits for the worker's result, adds it to its context, and then decides whether to call another worker, request more information, explain a failure, or return the final response
- Worker agents typically use separate conversation threads, which means the supervisor must include sufficient context in each delegated request
- This isolation prevents cross-contamination of context but requires deliberate context management by the supervisor



# Design Considerations

- **Routing and Delegation Strategy**
  - The supervisor's routing rules should be concrete and predictable
  - Effective supervisor instructions specify which agent or tool handles which type of request
  - For example:
    - Use the SQL agent for structured data questions, the HTTP agent for external API lookups, and the RAG agent for document-based questions
    - This explicitness reduces ambiguity in routing decisions and improves system reliability
- **Context Passing and Isolation**
  - A critical design decision concerns how much context to pass to worker agents
  - Frameworks support multiple modes
    - **Full history:** The worker receives the complete conversation history from the supervisor's context
    - **Last message only:** The worker sees only the immediate task assignment
    - **Fresh context:** Each worker receives only the specific task, not the full conversation history
- **Iteration Limits and Error Handling**
  - Supervisor implementations typically include maximum iteration limits to prevent infinite delegation loops
  - Worker retries can also be configured to handle transient failures gracefully

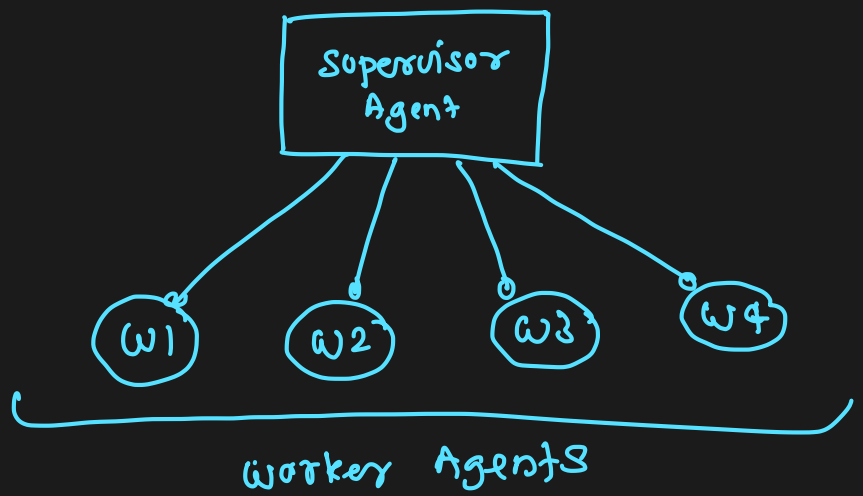
# Advanced Patterns

- **Human-in-the-Loop Integration**
  - Supervisor systems can incorporate approval gates for sensitive operations
  - When a worker attempts a high-stakes action—such as transferring funds or allocating budget—an interrupt is triggered and the system pauses for user approval
  - The workflow resumes from the exact point of interruption after the user responds
- **Nested and Hierarchical Supervisors**
  - Multi-level hierarchies allow supervisors to manage other supervisors : supervisor of supervisors
  - For example, a top-level supervisor might coordinate a research team supervisor and a writing team supervisor, each of which manages their own worker agents
  - This enables complex organizational structures that mirror human team hierarchies
- **Plan-Execute-Replan Sub-Agents**
  - A supervisor can delegate to a complex sub-agent that itself follows a Plan-Execute-Replan pattern
  - Creating a multi-step plan, executing each step, and adjusting the plan based on intermediate results
  - This allows the supervisor to delegate entire complex workflows while maintaining high-level oversight

# Topology Options

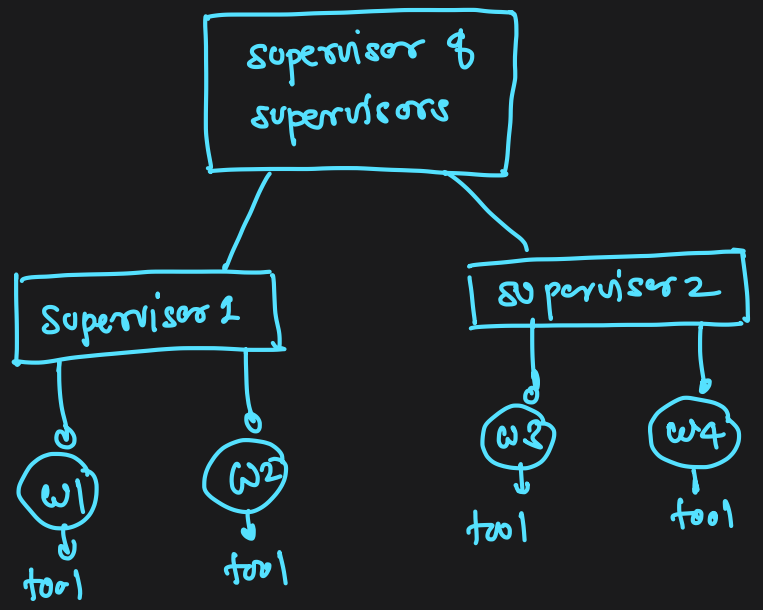
# Flat / Star Topology

- One supervisor, N workers, all communication flows through the center
- Workers never talk to each other
- **Pros**
  - Simplest to build and reason about
  - Centralized logging and observability
  - Easy to add/remove workers
- **Cons**
  - Supervisor is a bottleneck (all traffic passes through it)
  - No parallelism unless the supervisor explicitly fans out
  - Single point of failure
- **Best for:** Small teams (2–5 workers), demos, MVPs, well-defined domains



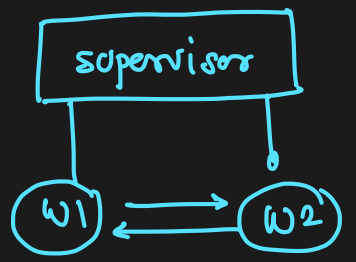
# Hierarchical (Supervisor of Supervisors)

- Top supervisor manages sub-supervisors; each sub-supervisor manages its own workers
- Two or more levels of coordination
- **Pros:**
  - Scales to very large teams (dozens of agents)
  - Mirrors real organizational structures
  - Each sub-team has focused responsibilities
  - Top supervisor sees a simplified view (teams, not individuals)
- **Cons:**
  - Higher latency (multiple coordination hops)
  - More complex to debug
  - Risk of the top supervisor being too abstract
  - Sub-supervisors add token cost
- **Best for:** Enterprise-scale systems with clearly separated business domains (e.g., sales team, support team, engineering team)



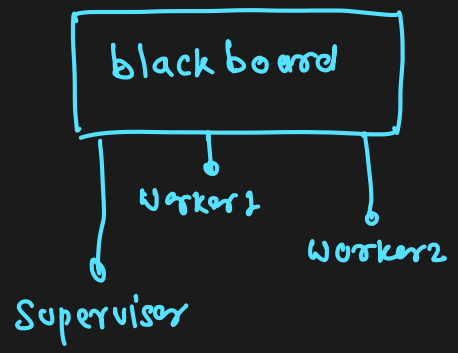
# Supervisor + Mesh (Hybrid)

- Supervisor handles initial routing and final synthesis, but workers can hand off to each other directly for specific tasks
- e.g., a research worker passing data to an analysis worker without supervisor involvement
- **Pros:**
  - Faster for worker-to-worker handoffs
  - Reduces supervisor load
  - Keeps centralized control for high-level decisions
- **Cons:**
  - Harder to trace (traffic doesn't always flow through the center)
  - Workers need to know about each other
  - Risk of bypassing supervisor guardrails
- **Best for:** Pipelines where workers have natural handoff relationships (retrieve → analyze → write)

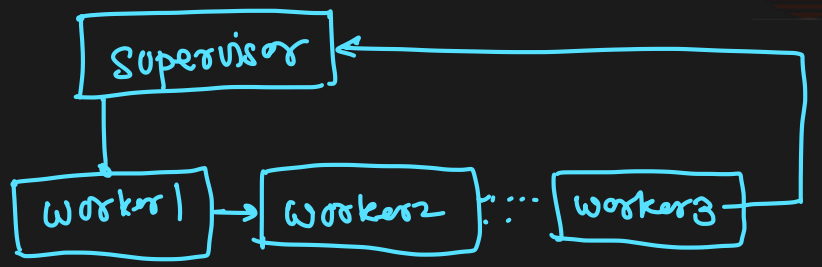


# Blackboard (Shared State)

- All agents read from and write to a shared data store
- The supervisor publishes tasks; workers pick them up and post results
- **Pros:**
  - Highly decoupled — workers don't need to know each other
  - Natural fit for async / long-running workflows
  - Easy to add workers (they just subscribe to relevant events)
- **Cons:**
  - Race conditions and ordering issues
  - Harder to enforce strict sequencing
  - Debugging requires tracing the shared state, not just messages
- **Best for:** Event-driven systems, long-running workflows, agents that react to conditions rather than requests



# Pipeline (Linear Supervisor)




- Workers are chained in a fixed order
- Supervisor enforces the sequence and passes outputs forward
- **Pros:**
  - Deterministic and reproducible
  - Easy to test and debug
  - Low coordination overhead
- **Cons:**
  - No dynamic routing
  - Can't skip or reorder steps
  - Fails if a step produces bad output
- **Best for:** Well-defined workflows (ETL, document processing, multi-stage analysis).

# Use of Pattern



- When to use
  - Workflows span distinct business or technical domains requiring different expertise
  - Requests require different agents, tools, or data sources in varying orders
  - The team needs a single entry point for routing and coordination
  - The order of execution cannot be predetermined and must be decided at runtime based on user intent and intermediate results
- When to avoid
  - The pattern is less suitable when a single agent with tool access can handle the full scope of tasks, or when strict deterministic workflows with no runtime routing decisions are required.

# Best Practices



- **Keep supervisor instructions concrete:** Specify routing rules explicitly so decisions are predictable and debuggable
- **Define clear worker specialties:** Each worker agent should have a focused responsibility with its own configured tools
- **Manage context deliberately:** Choose isolation levels based on continuity needs versus privacy and prompt-injection risk
- **Set iteration limits:** Prevent infinite loops with reasonable maximum iteration counts
- **Use meaningful names:** Descriptive agent names aid debugging and trace interpretation
- **Add guardrails:** Consider PII filtering, content moderation, and prompt-injection detection before and after the supervisor

# Pros and Cons



- Pros
  - Centralized Control and Observability
  - Clear Separation of Concerns
  - Dynamic Routing at Runtime
  - Easier to Add or Remove Specialists
  - Natural Fit for Human-in-the-Loop
  - Context Isolation
  - Supports Hierarchies
- Cons
  - Single Point of Failure
  - Supervisor Becomes a Bottleneck
  - High Token Cost
  - Requires Reliable Structured Output
  - Prompt Engineering Burden
  - Hard to Make Deterministic

# Design Approaches

# Routing Logic



- **LLM-as-Router (most common)**
  - Give the supervisor a prompt listing each worker's name, description, and capabilities
  - The supervisor's LLM reads the user request and emits a tool call to the matching worker
  - Works well when routing rules are semantic and fuzzy (e.g., "billing question" vs "technical question")
- **Rule-Based / Deterministic Router**
  - Use keyword matching, regex, or intent classifiers to route requests
  - Faster and cheaper than LLM routing, but brittle for ambiguous queries
  - Good for high-volume, well-defined domains (e.g., "refund" → billing agent)
- **Hybrid Router**
  - First pass: cheap rules handle obvious cases
  - Fallback: LLM decides for ambiguous cases
  - Reduces cost and latency while keeping flexibility
- **Capability-Registry Router**
  - Each worker registers a set of "skills" or "tool signatures."
  - The supervisor matches the task to skills rather than to named agents
  - Makes adding/removing workers easier—no prompt changes needed

# Delegation Mechanism

- **Tool-Call Handoff**
  - Each worker is exposed as a callable tool to the supervisor
  - Supervisor calls tool and receives a result
  - Works with any LLM that supports tool calling
- **Message-Passing / Shared State**
  - All agents read/write to a shared blackboard or state object
  - Supervisor writes a task; the target worker picks it up
  - Useful for async or parallel execution
- **Explicit Control Transfer (Handoff)**
  - Supervisor fully yields control to the worker, which then returns control when done
  - Common in graph-based frameworks (LangGraph nodes and edges)
  - Enables worker-to-worker handoffs without supervisor in the loop
- **Queue-Based Delegation**
  - Supervisor pushes tasks onto per-worker queues
  - Workers pull, process, and push results back
  - Good for scaling, retries, and observability

# Context Management



- **Full History Sharing**
  - Worker sees the entire conversation
  - Pros: maximum continuity, worker understands nuance
  - Cons: token cost, prompt-injection risk, context bloat
- **Summary + Task Only**
  - Supervisor summarizes prior context and passes only the summary + current task
  - Balances continuity with cost and isolation
- **Isolated Task Context**
  - Worker sees only its assigned task, nothingelse
  - Pros: clean, secure, parallelizable
  - Cons: worker may lack needed background
- **Scoped Memory**
  - Each worker has its own memory store; supervisor has its own
  - Shared facts go in a global store; private reasoning stays local
- **Structured Handoff Object**
  - Pass a typed object: {task, constraints, prior_results, expected_output_format}
  - Forces the supervisor to be explicit; makes workers easier to test

# Task Decomposition



- **Static Playbooks**
  - Predefined workflows (e.g., "for a refund request: verify → check policy → issue refund")
  - Predictable, testable, but inflexible
- **Dynamic Planning (Plan-and-Execute)**
  - Supervisor generates a plan at runtime, then executes step by step
  - Handles novel requests; requires good planning prompts
- **ReAct-Style Interleaving**
  - Supervisor thinks → acts (delegates) → observes result → thinks again
  - Simple to implement; good for exploratory tasks
- **Recursive Decomposition**
  - Supervisor splits a task, then delegates sub-splits to sub-supervisors
  - Enables hierarchies for very complex workflows
- **Dependency Graph**
  - Supervisor builds a DAG of subtasks with dependencies
  - Independent branches run in parallel
  - Good for research + analysis + writing pipelines

# Result Aggregation



- **Sequential Chaining**
  - Worker A's output becomes Worker B's input
  - Good for pipelines (retrieve → analyze → write)
- **Merge & Summarize**
  - Workers run in parallel; supervisor summarizes/merges results
  - Good for multi-perspective tasks (research + critique + fact-check)
- **Voting / Consensus**
  - Multiple workers answer the same question; supervisor picks the majority or best
  - Useful for high-stakes decisions
- **Conflict Resolution**
  - When workers disagree, supervisor asks a tie-breaker worker or applies rules
  - Explicit "critic" agent can adjudicate
- **Format Normalization**
  - Supervisor forces each worker to return a typed schema (JSON) for easy merging

# CrewAI

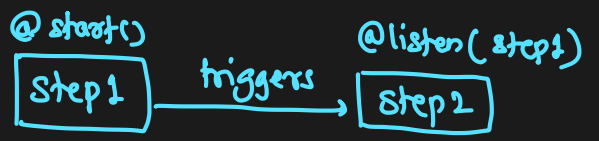
multi-Agent apps

# Introduction

- CrewAI is an open-source Python framework for orchestrating autonomous AI agents and building complex, production-ready workflows
- Its core design philosophy balances **autonomy** with **precise control**: it provides the collaborative intelligence of role-playing agent teams, while also offering the structured, stateful orchestration needed for reliable production systems
- The framework is built around two complementary abstractions
  - **Crews**
  - **Flows**

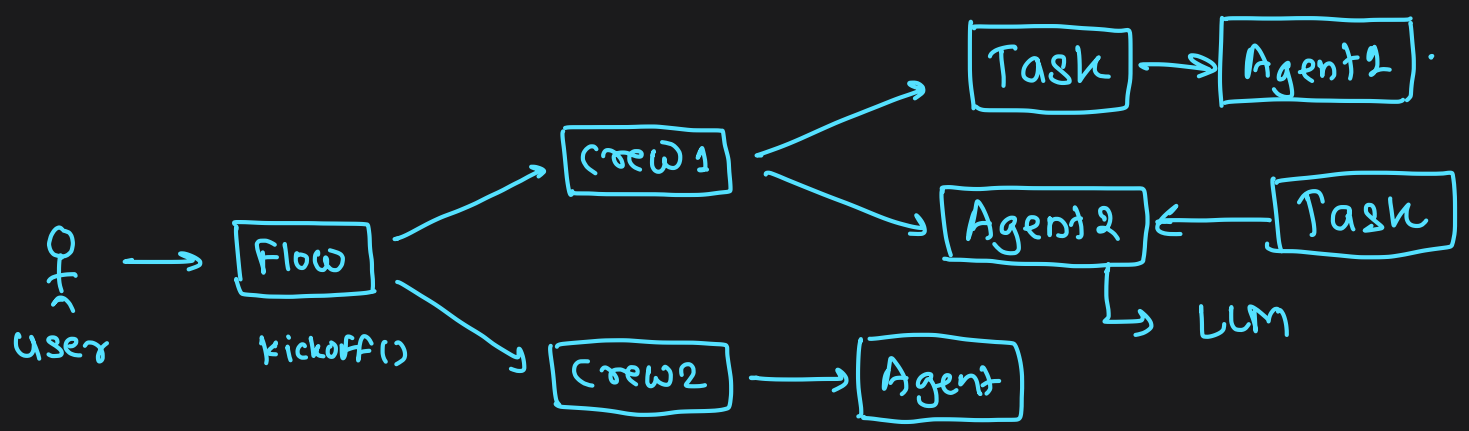
# Flows – The Backbone

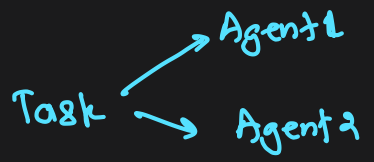
- A **Flow** is the structural foundation of a CrewAI application
- Think of it as the "manager" or "process definition" that defines the steps, logic, and data movement through your system → workflow
- Flows provide three critical capabilities:
  - **State Management**
    - Flows persist data across steps and executions, allowing you to pass information between different Crews, maintain context, and handle user inputs
    - State can be either unstructured (a flexible dictionary) or structured (using Pydantic models for type safety and validation)
  - **Event-Driven Execution**
    - Flows trigger actions based on events or external inputs
    - The @start() decorator marks entry points, while @listen() decorators define steps that execute in response to other steps completing
  - **Control Flow**
    - Flows support conditional logic, loops, and branching—essential for handling edge cases and ensuring predictable application behavior



# Crews: The Intelligence

- A **Crew** is a team of autonomous agents that collaborate to solve a specific task delegated by a Flow
- Crews are the "units of work" that do the heavy lifting requiring creativity, exploration, and collaboration
- Each Crew consists of **Agents** (specialized team members with defined roles, goals, and tools) and **Tasks** (discrete work units)
- The Crew defines how these agents collaborate through its **Process** (Sequential or Hierarchical)



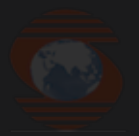



# How They Work Together



- The interaction pattern is straightforward
  - a Flow starts and manages state, then delegates a complex task to a Crew
  - The Crew's agents collaborate autonomously to complete the task, returning the result to the Flow, which continues execution based on that result
- **The recommended production pattern is "Flow-first"**
- Even if you only need a single Crew, wrapping it in a Flow provides state management, observability, and the structure needed for robust deployment

# Agents: The Team Members



- It is a specialized AI team member defined by three required attributes: **role**, **goal**, and **backstory**
- The **role** defines what the agent is (e.g., "Senior Data Researcher") → system prompt
- The **goal** defines what the agent aims to achieve → deliverable
- The **backstory** provides situational context that shapes the agent's behavior and expertise → context
- **Key Agent Parameters**
  - **LLM configuration:** The llm parameter determines which language model the agent uses (default: OpenAI's GPT-4o-mini). You can pass a model name string (e.g., 'gpt-4', 'claude-2') or an LLM class instance for advanced configuration
  - **Memory:** Setting memory=True enables the agent to maintain conversation history and context across multiple interactions, which is useful for complex, multi-step tasks → short term memory
  - **Context window management:** The respect_context_window parameter (default True) automatically summarizes content when the conversation exceeds the model's token limit. Setting it to False causes execution to stop with an error instead, preserving all context for cases where information loss is unacceptable
  - **Tools:** Agents are equipped with tools to interact with external systems. Tools can be specified at the agent level, and task-level tool specifications can override the agent's defaults for fine-grained control

# Tasks: The Work Units

- A **Task** represents a discrete unit of work
- Each task has a **description** (what to do), an **expected_output** (what the result should look like), and an assigned **agent**
- **Task Configuration**




  - **Dependencies:** Tasks can declare dependencies on other tasks, forming a structured execution order. The output of a preceding task can serve as context for subsequent tasks
  - **Output schemas:** Tasks support output_pydantic and output_json parameters for structured outputs. This is critical for production applications: structured outputs prevent parsing errors and ensure type safety when passing data between tasks or to your application
  - **Guardrails:** Task Guardrails allow you to validate task outputs before they are accepted. A guardrail function returns a tuple of (bool, Any)—if the first element is False, the second element provides feedback for the agent to retry
  - **Callbacks:** task_callback executes after each task completes; step_callback executes after each step in an agent's execution

# Processes: How Crews Collaborate

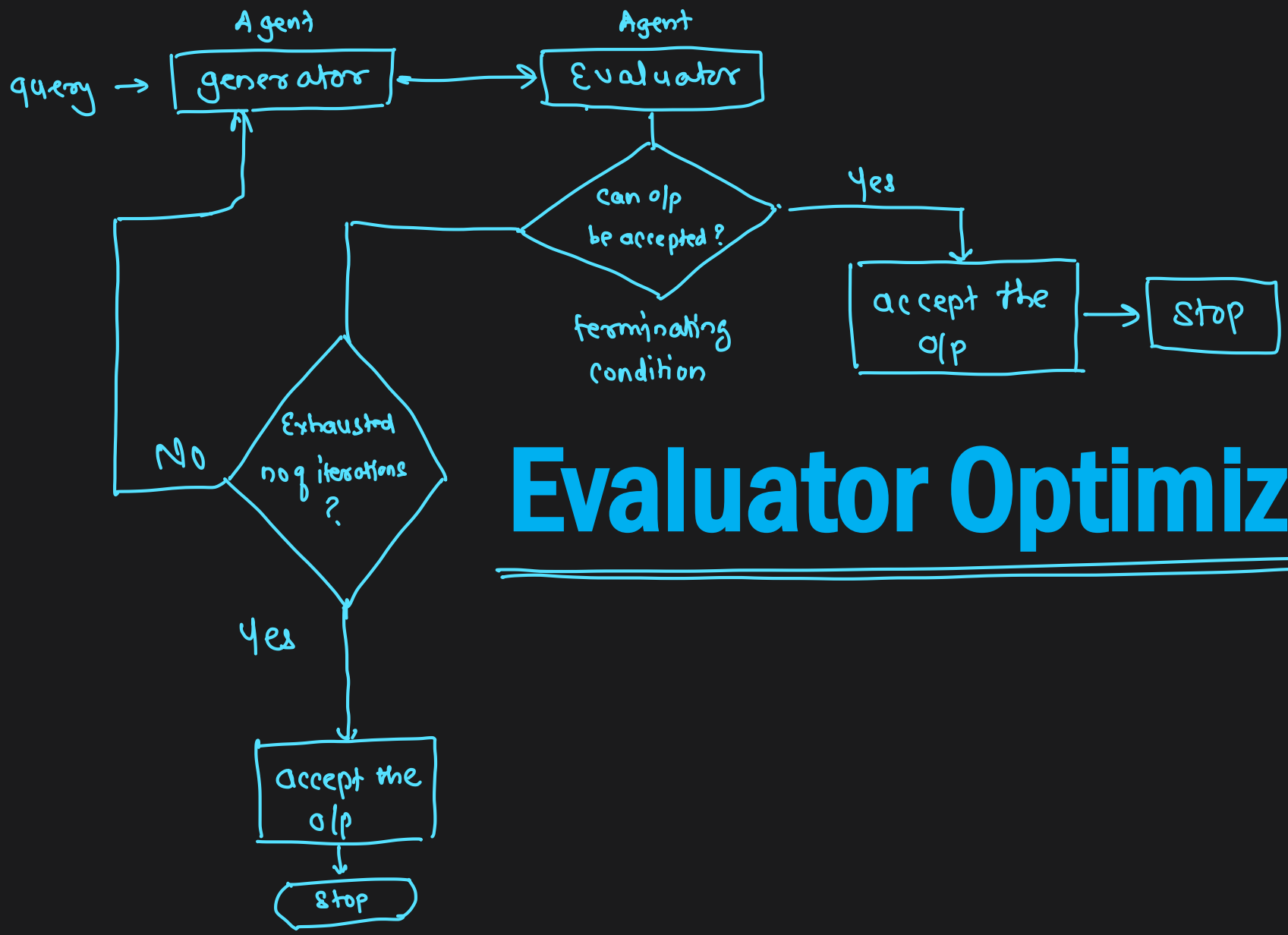
- The **Process** defines the execution strategy for a Crew's tasks
- CrewAI supports two process types, specified via the Process enum
- **Sequential Process (Default)** → deterministic
  - Tasks execute in the exact order they appear in the task list
  - The output of one task serves as context for the next
  - This is ideal for linear workflows where each step builds on the previous one
  - **Key characteristics:** Simple, easy to monitor, and requires that every task has an assigned agent
- **Hierarchical Process** → supervisor → orchestration
  - **A manager agent coordinates the workflow:** it delegates tasks to appropriate crew members based on their roles, validates outcomes, and ensures quality. This simulates a traditional organizational hierarchy
  - **The manager agent** can be created automatically by CrewAI or defined explicitly with custom attributes
  - A custom manager gives you control over the manager's role, goal, backstory, and capabilities
  - **Workflow:** The manager assigns tasks strategically, agents execute with oversight, and the manager validates results before proceeding. Despite the hierarchical structure, tasks still follow a logical order

# Tools: Extending Agent Capabilities

- Tools allow agents to interact with external systems—APIs, databases, search engines, file systems
- **Creating Custom Tools**
  - There are two primary approaches for custom tools
  - **Subclassing BaseTool:** Define a Pydantic input schema (args_schema), set the tool's name and description, and implement the _run method
  - **Using the @tool decorator:** For simpler tools, decorate a function with @tool("Tool Name"). The function's docstring serves as the description
- **Typed Outputs (Best Practice)**
  - When a tool returns structured data, define a Pydantic output model
  - This allows the agent to read the result as clear fields rather than guessing from plain text.
  - CrewAI infers the output schema from the Pydantic return annotation on a BaseTool subclass, or you can set result_schema explicitly with the decorator
  - Typed outputs are essential for results with stable fields like IDs, statuses, scores, or lists. They eliminate parsing ambiguity and improve agent reliability
- **Tool Caching and Async**
  - Custom caching strategies can be defined via cache_function
  - Async tools are supported by using async def with the @tool decorator, or by implementing both _run and _arun methods in a BaseTool subclass

# Summary

| Concept | Purpose | Key Configuration |
|---|---|---|
| Flow | Application backbone; state and control | Flow[StateModel], @start, @listen, @persist |
| Crew | Team of agents executing a task | agents, tasks, process, manager_llm |
| Agent | Specialized team member | role, goal, backstory, llm, tools, memory |
| Task | Discrete work unit | description, expected_output, agent, context, guardrails |
| Process | Collaboration strategy | Sequential (default) or Hierarchical |
| Tool | External system interface | BaseTool subclass or @tool decorator |
| LLM | Language model connection | Native SDK (OpenAI/Anthropic/etc.) or LiteLLM fallback |




# Evaluator Optimizer

# Introduction



- The **Evaluator-Optimizer pattern** is a fundamental agentic workflow where two distinct LLM roles operate in an iterative loop
  - a **generator** produces an output
  - an **evaluator** critiques it against defined criteria and feedback is fed back to the generator for refinement
- The cycle repeats until the output meets a quality threshold or a maximum iteration limit is reached
- The key insight is that **generation and evaluation are different cognitive tasks**
- Separating them allows each agent to specialize: the generator focuses on producing content, while the evaluator focuses on applying consistent quality criteria

# Structure and Components

- **Generator (Writer)**
  - Produces the initial output and subsequent revisions
  - Takes the original task plus any accumulated feedback as input
  - Can be the same model as the evaluator (with different system prompts) or a different model
- **Evaluator (Critic/Editor)**
  - Applies predefined criteria to assess the generator's output
  - Returns structured feedback: a **verdict** (e.g., PASS / NEEDS_IMPROVEMENT) and **specific issues** to address
  - Must produce **machine-checkable** or at least clearly actionable feedback for the loop to converge
- **Termination Condition**
  - **Primary:** Evaluator returns PASS (quality threshold met)
  - **Fallback:** Maximum iteration count reached, then escalate or return best-effort output
  - A starting limit of **3 iterations** is common, though the right cap depends on task complexity and cost budget

# How It Works



- **Initial Generation:** The writer creates a draft based on the given topic or task
- **Evaluation:** The evaluator reviews the draft against criteria such as clarity, engagement, structure, or domain-specific requirements
- **Iterative Refinement:** If improvements are needed, the feedback is incorporated, and the writer generates a revised draft
- **Loop Until Approved:** The process repeats until the content meets the desired quality standards or a maximum number of iterations is reached
- A concrete implementation might enforce **at least one feedback cycle** to demonstrate the pattern, even when the first draft appears acceptable

# Evaluator Design Choices



- The evaluator role can take one of two forms, with distinct trade-offs
- Same model, different system prompt
  - Cost: Lower
  - Independence: Limited
  - Risk: Evaluator may inherit generator's blind spots
- Different model
  - Cost: Higher
  - Independence: Full
  - Risk: Independent perspective; useful when roles require different strengths

# When to Apply



- The pattern produces measurable improvement under three conditions:
- **"Good output" can be described precisely enough** that the evaluator can score it consistently
- **Iterative refinement genuinely improves quality** — the generator can act on the evaluator's feedback to produce better results
- **The task does not have a single correct answer** that would make iteration redundant
- **Strong Use Cases**
  - **Code generation** with specific requirements (security standards, performance benchmarks, style guidelines)
  - **Professional communications** where tone and precision matter
  - **API documentation generation** (generator writes docs, evaluator checks for completeness, clarity, and accuracy against the codebase)
  - **SQL query production** (generator writes query, evaluator checks for efficiency and security issues)
  - **Literary translations** capturing nuanced meanings
  - **Any scenario where first-draft quality consistently falls short of requirements**

# When the Pattern Backfires

- Shared Blind Spots
- Vague Criteria
- Non-Actionable Feedback
- Tasks with a Single Correct Answer
- Resource-Constrained or Real-Time Scenarios



# Evaluator Optimizer vs Critic

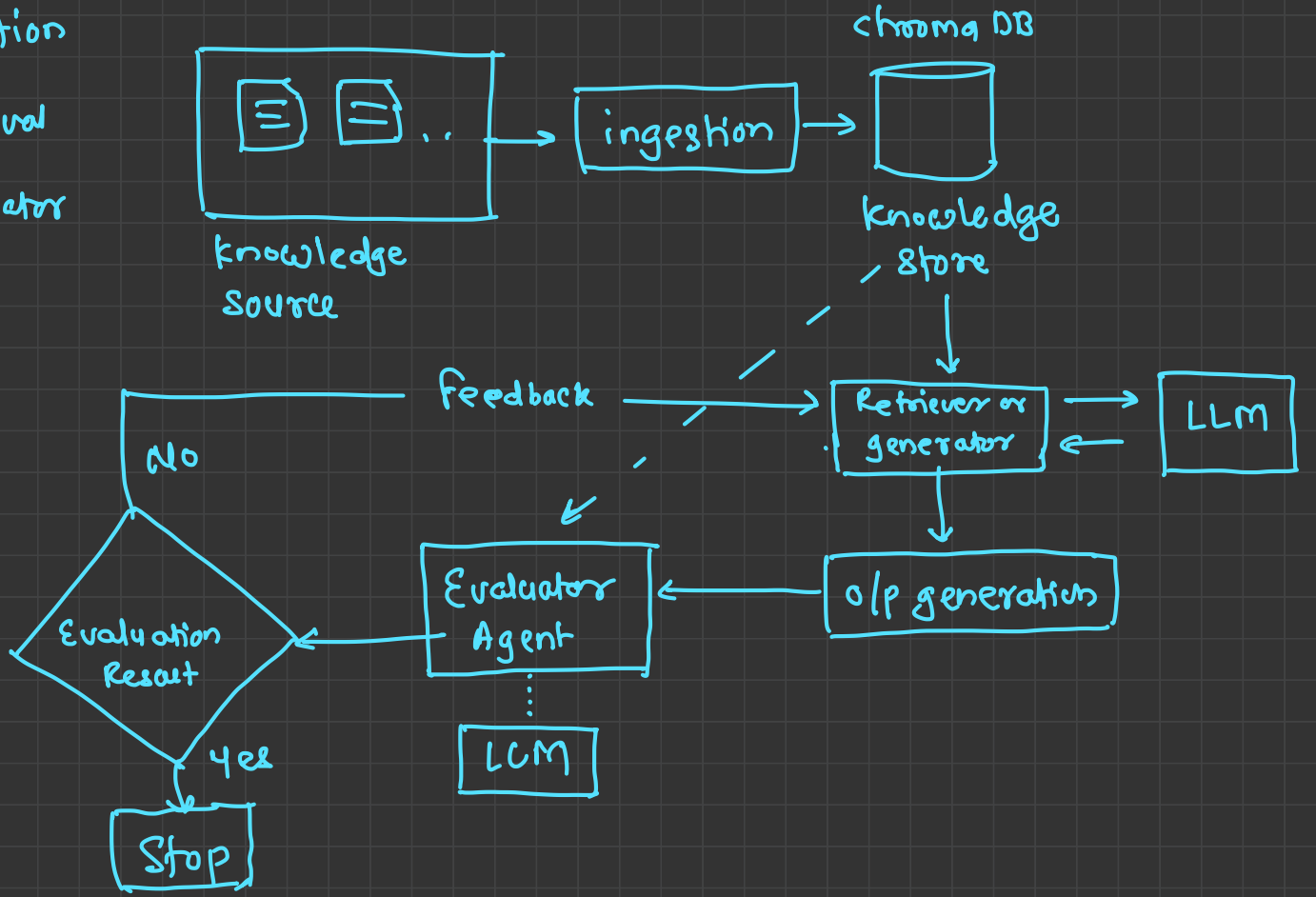
- The **Evaluator-Optimizer** pattern is defined by its **loop structure**
  - A generator produces output, an evaluator critiques it, and the generator revises repeating until quality passes
  - It is fundamentally about **iterative refinement over time**
- The **Critic** pattern is defined by its **role separation**
  - A dedicated agent whose sole job is to find flaws in another agent's work, operating with its own system prompt, context, and possibly a different model
  - It is fundamentally about **independent verification**

| Dimension | Evaluator-Optimizer | Critic Pattern |
|---|---|---|
| **Primary axis** | Temporal (iteration over rounds) | Spatial (separation of roles) |
| **Core mechanism** | Feedback loop with revision | Adversarial review without necessarily iterating |
| **Required components** | Generator + Evaluator + Loop controller | Critic agent (can exist standalone) |
| **Termination** | Explicit (PASS verdict or round cap) | Implicit (critique delivered) |
| **Feedback flow** | Critique → Generator revises → Re-evaluate | Critique → (optional) Human or downstream decision |

① ingestion

② Retrieval

③ Evaluator






# Hierarchical Agents

# What is the Hierarchical Agents Pattern?

- Hierarchical Agents is an important multi-agent design pattern used in Agentic AI systems
- In this pattern, multiple AI agents are organized into different levels of responsibility
- A higher-level agent is responsible for understanding the overall objective, planning the work, delegating tasks, monitoring progress, and integrating results
- Lower-level agents are responsible for executing specific tasks (tools)
- The fundamental principle is delegation of responsibility
- Instead of asking one AI agent to solve a large and complicated problem, the problem is divided into smaller tasks and assigned to specialized agents

# What is Hierarchical Agents?

- A Hierarchical Agent system is a multi-agent architecture in which agents have different levels of authority and responsibility
- A higher-level agent acts as a supervisor or manager
- It decides what needs to be done and which lower-level agent should perform each task
- Lower-level agents execute the assigned tasks and return their results to the parent agent
- The parent agent then evaluates the results and decides what should happen next
- This creates an organizational structure similar to a human organization where executives, managers, and employees have different responsibilities

# Core Idea

- The core idea is to separate planning from execution
- The top-level agent concentrates on the overall objective
- Manager agents concentrate on specific areas of the objective
- Worker agents concentrate on individual tasks
- This separation allows each agent to focus on a smaller and more clearly defined problem
- For example, if the objective is to build an e-commerce application, the top-level agent does not need to write every piece of code
- It can delegate frontend development to one agent, backend development to another agent, database design to another agent, and testing to another agent

# Why is it required ?

- Modern Agentic AI applications often involve complex objectives
- A single task may require research, reasoning, planning, coding, database operations, testing, document processing, and interaction with external systems
- As the number of responsibilities increases, a single agent becomes difficult to manage
- The agent may receive too much context, use inappropriate tools, make poor decisions about task priorities, or lose track of intermediate results
- Hierarchical architecture addresses these problems by distributing responsibility across multiple specialized agents

# Problem with single Agent

- A single general-purpose agent may be expected to perform every activity
- It may need to understand requirements, search for information, write code, access databases, execute tests, analyze errors, deploy infrastructure, and prepare documentation
- Although modern LLMs are capable of performing many of these tasks, combining all responsibilities into one agent increases complexity
- The prompt becomes larger
- The context becomes harder to manage
- Tool selection becomes more difficult
- The agent has more opportunities to make mistakes
- Debugging the system also becomes more difficult
- Hierarchical agents address this by distributing responsibilities

# The Three-Tier Architecture

- Most hierarchical agent systems follow a structure inspired by human organizational models
- The hierarchy typically consists of three functional layers:
  - **High-Level (Strategic) Agents**
    - Responsible for the "big picture"
    - They interpret user requests, formulate overall plans, and decompose those plans into actionable tasks
    - These agents manage the system in alignment with broader goals
  - **Mid-Level (Coordination) Agents**
    - Receive directives from high-level agents and refine them into specific instructions
    - They further decompose tasks when needed, coordinate teams of low-level agents, and serve as communication facilitators by converting broad goals into specific directives while consolidating reports from below
  - **Low-Level (Execution) Agents**
    - The "doers" of the system
    - These are specialized agents that perform specific tasks as directed
    - They are often narrow in scope and may use rule-based logic, APIs, or task-specific algorithms
    - They report results back up the chain, informing the next steps of the process

high → planning
↓
mid → co-ordination
↓
low → doer

# Agent communication

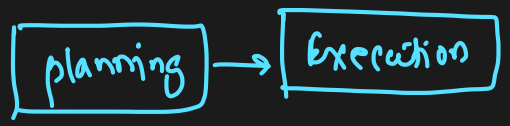


- Agents need mechanisms for exchanging information
- Common approaches include:
  - Direct agent-to-agent messages
  - Shared state → Short term memory
  - Databases → long term memory
  - Message queues → Kafka, RabbitMQ
  - Event streams
  - Graph state
- The communication mechanism should be designed so that agents receive only the information relevant to their task

# Pros and Cons

- **Advantages**
  - **Handles Complexity:**
    - Ideal for open-ended goals with no fixed sequence of steps
    - The manager agent can build a plan before execution starts, making it suitable for ambitious tasks
  - **Scalability:**
    - The architecture supports scaling by allowing new agents to be added at appropriate tiers and distributing workload across the hierarchy
  - **Quality Oversight:**
    - Higher tiers can review lower-tier work, providing built-in quality control and knowledge transfer
- **Limitations**
  - **Cost and Latency:**
    - Hierarchical orchestration is the most expensive and least predictable pattern
    - The manager's planning overhead can add 15–20% of total runtime, and a three-level system can add 6+ seconds of latency before any worker starts
  - **Single Point of Failure:**
    - The manager agent is a critical vulnerability
    - If it misjudges the plan or delegates to the wrong worker, every downstream agent inherits that mistake, and the added coordination layer makes errors harder to trace

# When to choose ?



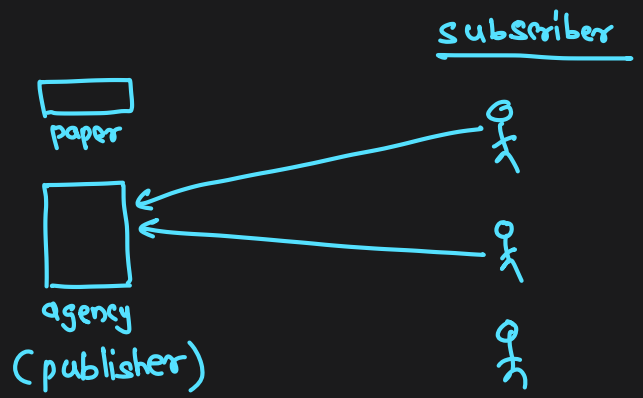

- Task requires planning before execution. The sequence of steps is unknown upfront, and a manager must build a plan before any worker starts.
- Long-horizon tasks with error compounding risk. A small mistake early on could cascade into major failure across many steps. Hierarchy provides oversight to catch and correct it.
- Need explicit quality gates. A supervisor can review worker outputs before passing them up, catching errors at each tier.
- Conditional branching and loop-back required. The workflow needs "if this, then that" decisions or iterative refinement based on intermediate results.
- Security or compliance boundaries exist. Regulations mandate strict data isolation, and one agent must prepare work while another validates it (separation of duties).
- Multiple teams own distinct knowledge domains. Each team maintains a domain-specific agent without waiting for others; explicit interfaces simplify governance.
- Task scale exceeds what one agent can handle. The problem is large enough that specialization across many domains is genuinely necessary.
- Planning is the core challenge, not execution. The hard part is deciding what to do, not doing it.

# When not to choose

- Role-based behavior can be simulated with prompting. Distinct roles (planner, reviewer, executor) do not automatically justify multiple agents. A single agent with persona switching often suffices.
- Cost is a primary constraint. Hierarchical orchestration is the most expensive pattern. Redundant context processing and communication overhead multiply with agent count.
- Latency is a primary constraint. Every handoff adds delay. A three-level system can add 6+ seconds before any worker starts.
- Task is simple or short-horizon. For single-hop tasks, marginal benefits of additional agents diminish quickly. Two agents often achieve the best accuracy-efficiency trade-off.
- Coordination overhead exceeds task value. Beyond ~15% of total token budget, hierarchical coordination erodes its planning advantage.
- Task can be described in a few steps without branching. If there is no conditional logic, a hierarchy adds cost without value.
- You are choosing hierarchy because it sounds impressive. Complexity is not a feature. The fewest agents capable of completing the task is the right default.

# Use Cases

- **Supply Chain and Logistics**
  - High-level agents plan inventory strategy, mid-level agents break it into localized implementations, and low-level agents handle warehouse operations and shipping routes
- **Manufacturing**
  - High-level agents ensure production meets demand, mid-level agents convert directives into tasks, and low-level agents control machines and production lines
- **Agentic RAG**
  - A manager agent coordinates the workflow, while specialized agents handle retrieval, reranking, and synthesis

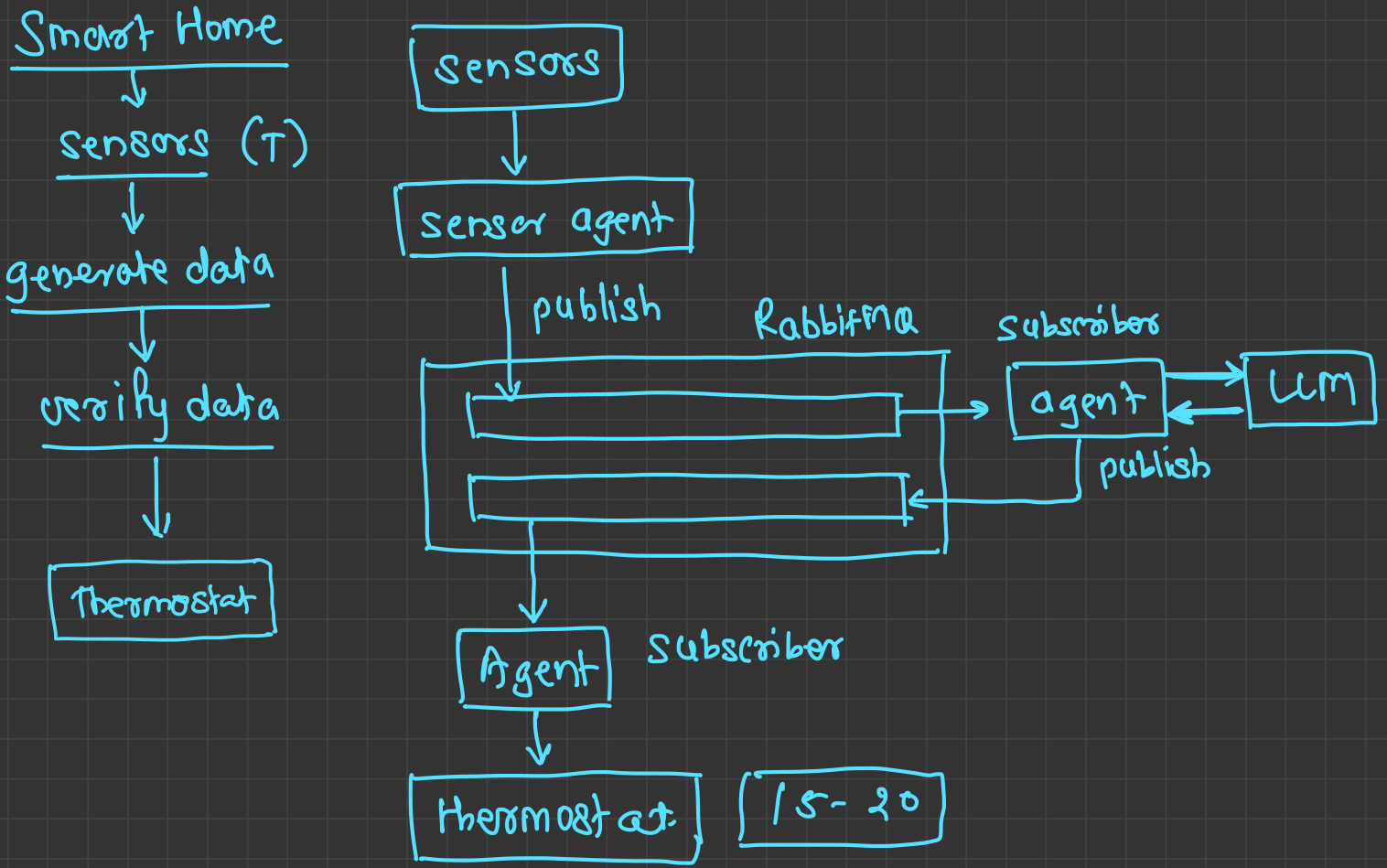






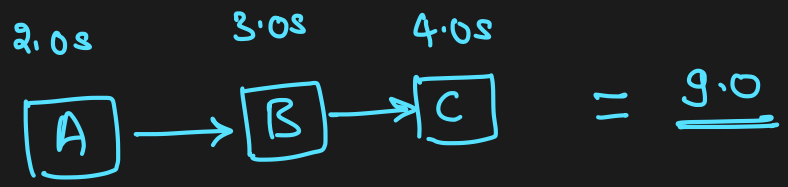
# Event Driven Agents

publisher - subscriber pattern

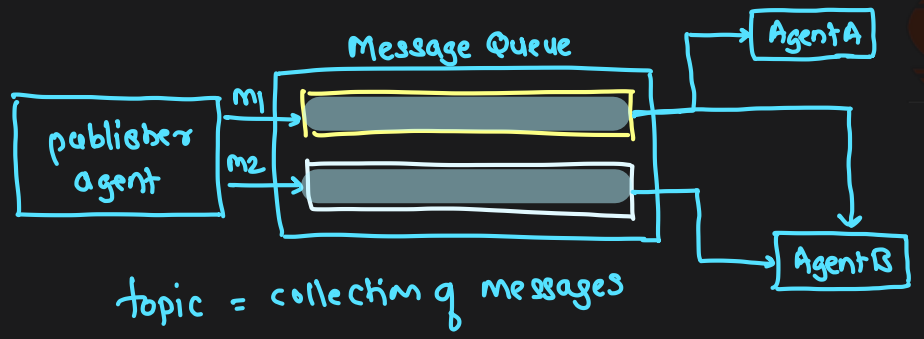

Smart Home
sensors (T)
generate data
verify data
Thermostat
sensors
sensor agent
publish
Rabbitmq
subscriber
agent
LLM
publish
Agent
subscriber
Thermostat
15-20

# Sequential Pipeline Trap

- Each agent holds the execution stack open while waiting
- Agent C cannot begin work until B returns, even if C's task is independent
- The fastest agent remains bottlenecked behind the slowest
- Failure in any agent cascades up the entire chain
- Adding a new agent requires modifying the caller
- No parallelism possible within a single request
- Throughput dictated by the slowest agent
- Debugging requires tracing a single monolithic stack



# Event Bus Architecture




- Agents publish typed events to named topics
- Agents subscribe to topics they care about
- No agent holds the execution stack open waiting for another
- When an agent completes work, it **publishes an event** rather than returning a value to a caller
- Downstream agents are **woken by their topic subscriptions**, not by explicit handoffs
- The broker handles routing, durability, and delivery guarantees
- Multiple agents can subscribe to the **same topic** and all receive the event simultaneously
- Publishers do not know who consumes their events
- Subscribers do not know who produces the events
- The bus becomes the single integration point
- New subscribers can be added without touching publishe

# Core Characteristics

# Asynchronous

- Agents operate independently, responding to triggers as they arrive
- The publisher does **not wait** for a response
- An agent can be offline; the event remains in the broker until it returns
- A single agent going down does **not halt the ecosystem**
- Publisher and subscriber lifecycles are independent
- Events can be processed at different times
- No shared execution thread between agents

| Sequential | Event-Driven |
|---|---|
| Caller blocks until response | Publisher fires and continues |
| Failure cascades up the chain | Failure isolated to one subscriber |
| Tight temporal coupling | Loose temporal coupling |
| Both must be alive simultaneously | Either can be offline |
| Single point of failure | Redundant consumers possible |
| Synchronous error propagation | Errors handled per-subscriber |
| Limited concurrency | Natural concurrency |

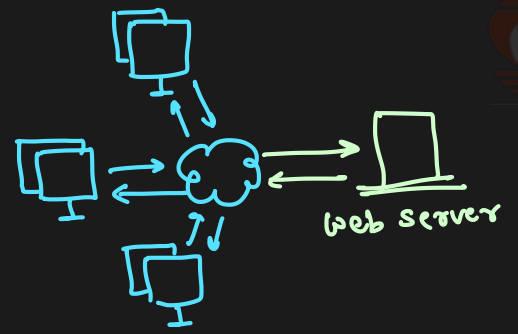
# Decoupled

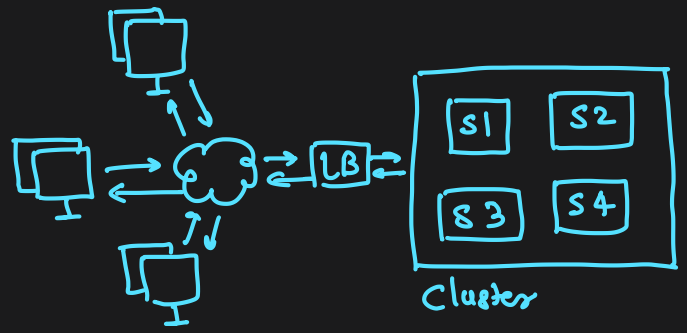


| Dimension | Sequential | Event-Driven |
|---|---|---|
| **Temporal** | Must be alive simultaneously | Publisher/subscriber need not overlap |
| **Spatial** | Must know each other's address | Only need to know the topic name |
| **Referential** | A must know B's interface | A only knows event schema |
| **Evolutionary** | Changing B breaks A | Adding subscribers does not affect publisher |
| **Protocol** | Shared RPC contract | Shared event contract |
| **Team** | Cross-team coordination needed | Teams own their agents independently |
| **Deployment** | Coordinated releases | Independent deployment |
| **Testing** | Integration tests required | Contract tests sufficient |

# Fault Tolerance

- **Durability:** Events persist in the broker until acknowledged
- **Replay:** Event log is the source of truth; state can be reconstructed
- **Store-and-forward:** Agents with intermittent connectivity can catch up
- **Isolation:** One agent's failure does not cascade
- **Retry:** Failed processing can be retried without affecting other subscribers
- **Dead Letter Queue:** Failed events captured for later analysis
- **Idempotency:** Events can be processed multiple times safely
- **Checkpointing:** Progress tracked per consumer (subscriber)
- **Circuit Breakers:** Prevent cascading failures
- **Bulkheads:** Isolate failures to specific agent pools




Horizontal scaling

# Scalability Through Parallelism



- Each agent type scales independently based on its own event load
- Horizontal scaling: add more instances of a slow agent
- System operates only as slowly as its single longest task
- Backpressure handled by broker (queue depth, consumer lag)
- Consumer groups enable load balancing across instances
- Partitioning enables ordered processing where needed
- Auto-scaling based on lag metrics
- No single point of contention

# Implementation: Message Broker

| Broker | Best For | Key Features |
|---|---|---|
| Apache Kafka | Enterprise scale, high throughput | Durability, replay, partitioning, exactly-once |
| AWS EventBridge | AWS-native, serverless | Routing rules, schema registry, SaaS integration |
| Google Pub/Sub | GCP-native, global scale | At-least-once, ordering, dead letter |
| Solace Agent Mesh | Multi-agent AI specifically | Built for agent transport, event mesh |
| asyncio queues | Python-native, lightweight | Minimal viable event bus, per-agent workers |
| Redis Streams | Low-latency, moderate scale | Consumer groups, persistence, simple ops |
| RabbitMQ | Complex routing, AMQP | Exchanges, queues, flexible routing |
| NATS | Cloud-native, lightweight | Fast, simple, JetStream for persistence |
| Azure Service Bus | Azure-native, enterprise | Sessions, transactions, dead letter |
| Pulsar | Multi-tenant, geo-replication | Tiered storage, functions, multi-protocol |

# Implementation: Dynamic Routing



- **Rule-Based Routing:** Event bridge style
- **Semantic Routing:** LLM-based routing
- **Content-based:** Route by payload fields
- **Header-based:** Route by metadata
- **Topic-based:** Route by event type
- **Priority-based:** Route urgent events first
- **Round-robin:** Distribute load evenly
- **Sticky:** Same correlation ID to same agent
- **Broadcast:** All subscribers receive
- **Fan-out/fan-in:** Parallel then aggregate

# Common Pitfalls to Avoid

| Pitfall | Symptom | Fix |
|---|---|---|
| **Event explosion** | Thousands of trivial events | Batch, aggregate, or use coarser event types |
| **Schema drift** | Subscribers break on new fields | Versioned schemas, backward compatibility |
| **Lost correlation** | Cannot trace flow across agents | Mandatory correlation_id in envelope |
| **Blocking subscribers** | One slow agent backs up the bus | Async processing, separate consumer groups |
| **Infinite loops** | Agent A triggers B triggers A | Idempotency keys, cycle detection, TTL |
| **No dead letter queue** | Failed events disappear silently | DLQ + alerting + replay tooling |
| **Over-engineering** | Kafka for 10 events/day | Match broker to scale; start simple |
| **Poison messages** | One bad event crashes consumer | Validation, DLQ, circuit breaker |
| **Ordering assumptions** | Events processed out of order | Partitioning by key, sequence numbers |
| **Duplicate processing** | Same event processed twice | Idempotency keys, deduplication |
| **Unbounded queues** | Memory exhaustion | TTL, max queue depth, backpressure |
| **Missing observability** | Cannot debug production | Traces, metrics, correlation IDs |

# When to Use

- Multiple agents can work in parallel on the same trigger
- Agents have different processing tempos
- Fault tolerance and replay are required
- System must scale horizontally
- New agents will be added over time without modifying existing ones
- Human-in-the-loop governance is needed
- Audit and compliance trails are required
- Agents are owned by different teams
- System must handle intermittent connectivity
- Workload is bursty or unpredictable

# When to avoid

- Strict sequential ordering is mandatory (e.g., financial transaction steps)
- Ultra-low latency is required (<10ms) and broker overhead is prohibitive
- Team lacks operational maturity for distributed systems
- Simple 2–3 agent pipeline with no growth expected
- Strong consistency across agents is required (eventual consistency is unacceptable)
- Debugging simplicity is paramount
- Transactional guarantees span multiple agents
- Total system state fits in a single process



# Checkpoint

# Introduction

remembers the context

- The checkpoint design pattern is a durability mechanism used in agentic AI systems
- It saves the complete execution state of an agent or workflow at defined points during execution
- Each saved snapshot is called a checkpoint
- A checkpoint allows an agent to resume from an exact position after a crash, interruption, or deliberate pause
- It transforms an agent from a fragile, single-process script into a resilient, long-running service
- The pattern is analogous to save points in video games or database transaction logs in distributed systems

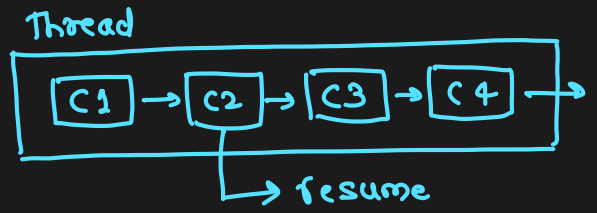
# The Problem It Solves

- Agentic workflows are often long-running and multi-step
- They involve expensive LLM calls, external tool invocations, and complex branching logic
- A failure at step 99 of a 100-step process, without checkpointing, forces a complete restart
- This wastes time, tokens, money, and API rate limits
- Human-in-the-loop workflows require the ability to pause indefinitely and resume later
- Multi-turn conversations require persistent memory across separate process invocations
- Checkpointing addresses all of these requirements with a single architectural mechanism
  - ↳ persists state in checkpoint file

# Core Concepts and Terminology

→ can be saved in the form of SQLite DB, json file, postgreSQL

- **Checkpoint:** A serialized snapshot of execution state at a specific point in time
- **Thread:** A logical conversation or workflow instance identified by a unique ID, used to group related checkpoints
- **Superstep:** A batch of execution in which multiple nodes or executors run before the next checkpoint is taken
- **Checkpointer:** The component responsible for saving and loading checkpoints
- **RunState:** An object representing the full, serializable state of an in-progress run
- **Interrupt:** A deliberate pause in execution that surfaces a request for external input → HITL → approve / reject
- **Resume:** The act of reconstructing state from a checkpoint and continuing execution



# What Gets Captured in a Checkpoint

- System → Tool Call
- Human
- AI

- **Conversation State:** The full message history that reconstructs the agent's context window
- **Execution State:** The current iteration count, execution phase, and pending node or tool calls
- **Executor States:** Custom state maintained by individual agents, nodes, or workflow executors
- **Shared State:** Variables and data shared across the entire workflow or graph
- **Pending Messages:** Messages queued for delivery in the next superstep
- **In-Flight Requests:** Outstanding requests awaiting responses, such as human approvals
- **Metadata:** Step numbers, timestamps, source of the checkpoint, and version information
-



# When Checkpoints Are Created

- **Superstep Boundaries:** After all executors in a workflow step complete their work
- **Phase Transitions:** Before an LLM call, after an LLM response, or after tool execution
- **Human Approval Points:** When the workflow pauses to wait for external input
- **Interval-Based:** Every N iterations or after processing a defined batch of items
- **Manual Triggers:** When the application explicitly requests a checkpoint
- **Entry Points:** At the start of a workflow to capture the original input for replayability
- Checkpoints are taken at deterministic boundaries to guarantee consistency across components

# Resume and Recovery



- The saved checkpoint is loaded from storage into memory
- The agent or workflow context is reconstructed from the serialized state
- Pending nodes, tool calls, or approval requests are identified
- Execution continues from the exact point of interruption
- In some frameworks, message history is replayed into a new instance to rebuild context
- Stable identifiers are required to match saved state to the correct executors across process restarts
- Resume can occur in the same process, a new process, or even a different machine

# Human-in-the-Loop Interruptions



- Checkpointing is the foundation for human-in-the-loop agent workflows
- A node or tool can explicitly request approval before performing a sensitive action
- The workflow pauses and saves a checkpoint at that exact moment
- The pending request is surfaced to a human reviewer through an API or UI
- The human provides a decision: approve, reject, or edit
- The workflow resumes from the checkpoint with the human's response injected
- Side effects before an interrupt must be idempotent, since the node may re-execute from the start on resume

# Storage Backends and Durability



- **In-Memory Storage:** Fast and simple, suitable only for development and testing; loses state on restart
- **File-Based Storage:** Persists to disk, suitable for single-machine deployments and demos
- **SQLite Storage:** Lightweight, file-backed, and durable; good for small to medium projects
- **Relational Databases:** PostgreSQL, MySQL, and similar systems for production workloads
- **NoSQL Databases:** MongoDB, Redis, and similar systems for distributed or low-latency scenarios
- **Cloud Storage:** Managed services such as Cosmos DB for enterprise-grade durability
- Storage choice determines durability, scalability, and recovery guarantees

# Best Practices

- Choose a persistent checkpointer for any production workload; never rely on in-memory storage
- Assign stable, unique identifiers to all executors and nodes to ensure reliable resume
- Design nodes so that side effects before interrupts are idempotent
- Place side effects after interrupts whenever possible, or isolate them in dedicated nodes
- Implement checkpoint pruning and retention policies to control storage costs
- Version your state schemas and executor definitions to maintain compatibility
- Test resume paths explicitly, including crash recovery and cross-process resumption
- Log checkpoint creation and resume events for observability and debugging



# Production Deployment

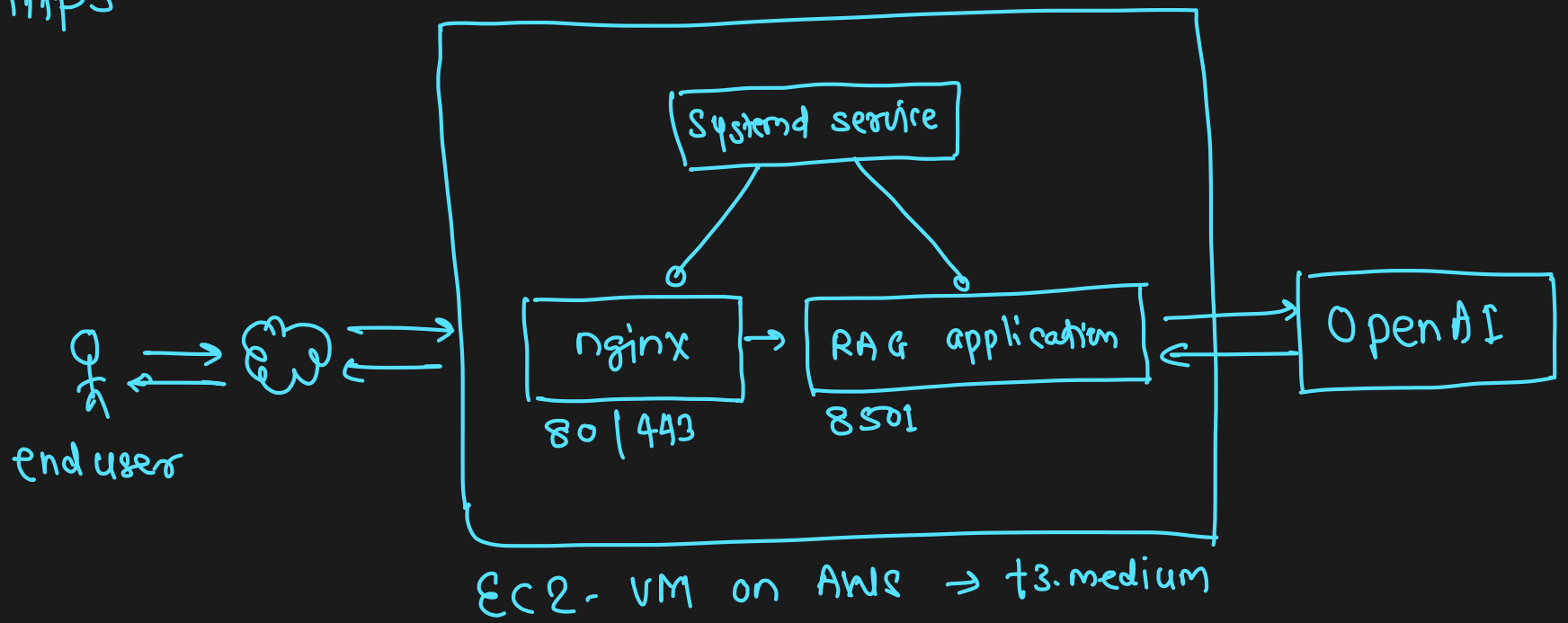
making the application available to end users

# Architecture Overview

① nginx → frontend deployment

② cloud → AWS

③ http | https



# Cost Considerations

- **EC2 instance costs (approximate):**
  - t3.small: ~$0.02/hour (~$15/month)
  - t3.medium: ~$0.04/hour (~$30/month)
- **API costs dominate:**
  - OpenAI embedding: $0.02/1M tokens
  - GPT-4o-mini: $0.15/1M input, $0.60/1M output
- **Managed alternative comparison:**
  - Bedrock Managed Knowledge Base: ~$350/month fixed floor (OpenSearch Serverless 2 OCU minimum)
  - Self-managed on EC2: Near-zero at low volume, scales with usage
- **Decision rule:** EC2 self-hosting wins at low-to-medium volume or when you need full control

# Instance Selection

- Development / Test
  - Type: t3.small (2 vCPUs / 2 GiB)
  - Minimum viable
- Production CPU
  - Type: t3.medium or c7g.xlarge (2-4 vCPU / 4-8 GiB)
  - 4 GiB RAM recommended
- GPU Inference
  - g4dn.xlarge: (4 vCPUs/ 16 GiB + T4)
  - For local model serving
- Storage: 20 GB EBS minimum
- Security Groups:

Firewall

| Port | Protocol | Source | Purpose |
|---|---|---|---|
| 22 | TCP | Your IP only | SSH access |
| 80 | TCP | 0.0.0.0/0 | HTTP (redirects to HTTPS) |
| 443 | TCP | 0.0.0.0/0 | HTTPS traffic |

# Systemd Service



```
[Unit]
Description=Agentic RAG Service
After=network.target

[Service]
User=ubuntu
Group=ubuntu
WorkingDirectory=/home/ubuntu/rag
ExecStart=/home/ubuntu/.local/bin/streamlit run app.py --server.address=0.0.0.0
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

# Nginx Reverse Proxy



```
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

server {
    listen 80;

    location / {
        proxy_pass http://127.0.0.1:8501;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }
}
```



# Optimizing Production Environment

# Optimization Triangle



- **Efficiency**
  - Right-sized resources, no over-provisioning
  - Prevents resource contention that cascades across workloads
- **Resilience**
  - Components fail independently
  - Limits failure propagation to one component
- **Operability**
  - Automated, auditable management
  - Eliminates human error as a failure vector

# Blast Radius for Agentic RAG

- **Definition:** The scope of resources, data, and capabilities affected if a component fails or is compromised
- For an Agentic RAG system, blast radius components include

| Component | Risk Factor | Blast Radius Amplifier |
|---|---|---|
| **LLM API credentials** | Compromised key = unlimited spend | High |
| **Vector store** | Corruption = all retrieval fails | High |
| **Agent orchestration** | Infinite loop = resource exhaustion | Medium |
| **EC2 instance** | Single point of failure = total outage | Critical |
| **Security group** | Overly permissive = lateral movement | Critical |

# Instance Right-Sizing (Efficiency + Blast Radius)



- **AWS right-sizing guidance:** Identify instances with **maximum CPU and memory utilization below 40%** over 4 weeks — these are candidates for downsizing
- **Monitoring approach:**
  - Focus on recent data (not historical)
  - Only evaluate instances running at least half the time
  - Ignore T-series burst instances for CPU-based sizing decisions
- **Blast radius benefit:**
  - A right-sized instance has predictable failure modes
  - An over-provisioned instance creates the illusion of capacity, hiding bottlenecks until they cascade

# Security Group Hygiene (Blast Radius Containment)

- **AWS best practice:** Apply **least-privilege rules** to security groups .
- **Anti-pattern (high blast radius):**
  - Inbound: 0.0.0.0/0 → Port 22 (SSH)
  - Inbound: 0.0.0.0/0 → Port 8000 (FastAPI direct)
- **Critical rule:**
  - EC2 Security Group and on-host firewall (UFW/iptables) are **separate layers**
  - Both must enforce least privilege
- **Blast radius impact:**
  - Removing SSH eliminates the most common lateral movement vector
  - An attacker cannot pivot to the instance even if other credentials leak

# Production Checklist



- **Efficiency:**
  - Instances right-sized based on 4-week max CPU/memory < 40%
  - Spot instances used only for interruption-tolerant workloads
  - Auto Scaling for stateless components
- **Blast Radius Containment:**
  - Security groups follow least-privilege (no 0.0.0.0/0 on SSH)
  - SSH disabled; Systems Manager used for access
  - Separate credentials per service/component
  - Vector store not publicly accessible
  - Multi-AZ for stateful components
- **Resilience:**
  - Automated health checks and remediation
  - Gradual deployment with canary stages
  - Agent loop iteration limits enforced
  - API spend limits configured
- **Observability:**
  - All layers monitored (infrastructure → agent behavior)
  - Alerts tuned for early detection, not noise
  - Change dependency graph traced before modifications

# Evaluation

# RAGAS



- RAGAS (Retrieval Augmented Generation Assessment) is an evaluation framework for LLM applications that uses LLM-as-a-judge to score quality without requiring massive human-annotated datasets
- **The problem it solves**
  - Traditional metrics like BLEU or ROUGE fail to capture what matters in RAG systems—factual consistency, retrieval quality, and answer relevance
  - RAGAS provides **quantifiable scores** for each failure point in the RAG pipeline
- **Why it matters for production**
  - Without systematic evaluation, you ship blind
  - RAGAS turns "answers feel better" into measurable deltas—faithfulness up, retrieval stable, cost per query down

# Core RAGAS Metrics

| Metric | What It Evaluates | Failure Mode Detected |
|---|---|---|
| **Faithfulness** | Is the answer grounded in retrieved context? | Hallucination—model freelancing |
| **Answer Relevancy** | Does the answer address the query? | Off-topic or incomplete responses |
| **Context Precision** | Are relevant documents ranked higher? | Poor retrieval ranking |
| **Context Recall** | Does context contain all needed information? | Missing key facts from retrieval |

**Recommended production targets : Faithfulness ≥ 0.9, Answer Relevancy ≥ 0.85, Context Precision ≥ 0.8**